A BEGINNER COURSE IN PROGRAMMING

JavaScript Foundations

Ten chapters of clear lessons, guided practice, quizzes, challenges, and real projects - from your very first line of code to saving files on disk.

```
let name = "friend";
let total = 0;

for (let i = 1; i <= 5; i++) {
  total = total + i;
}

console.log(`${name}, the total is ${total}`);
// friend, the total is 15
```

Written for school students and self-learners with zero experience. Every lesson ends with a check, every chapter with practice, a quiz, challenges, and a full answer key.

AUDIENCE	LEVEL	EDITION
Students & Self-Learners	Beginner	First Edition

LEARN · PRACTICE · BUILD

About This Book

Licence, support, and feedback

Free to read

JavaScript Foundations is a free book. You never need to pay to read it, print it, or hand it to a friend, a classroom, or a study group.

Licence

© 2026 Mohamed Ahmed Fayad. This work is licensed under the Creative Commons Attribution-NonCommercial 4.0 International licence (CC BY-NC 4.0). You are welcome to share and adapt it for any non-commercial purpose, as long as you give credit and say if you made changes. Full licence text: creativecommons.org/licenses/by-nc/4.0

Support the next book

> If this book helped you learn, a small donation helps keep it updated and pays for more free lessons. Donating is always optional.
>
> Donate at: **ko-fi.com/1to10**

Found a mistake?

> No book is perfect. If you spot a typo, an example that does not run, or a question with the wrong answer, please tell us so the next version can fix it.
>
> Report it here: **moh.fayad14@gmail.com**

Edition

First Edition · Version 1.0 · October 2026. A newer version may be available where you downloaded this book.

All code examples run in a browser console or in Node.js. This book is an independent educational resource and is not affiliated with or endorsed by any company or product named in it.

Table of Contents

How to Use This Book 4

Chapter 1 Hello, JavaScript 6

Lesson 1.1 What Is Programming? 6

Lesson 1.2 Running Your First Code 7

Lesson 1.3 Statements Run in Order 8

Lesson 1.4 Comments and Clean Code 9

Chapter 2 Variables and Values 17

Lesson 2.1 Labels for Values: let and const 17

Lesson 2.2 Numbers and Arithmetic 18

Lesson 2.3 Strings and Concatenation 19

Lesson 2.4 Booleans, typeof, and Good Names 21

Chapter 3 Working with Strings 28

Lesson 3.1 Template Literals 28

Lesson 3.2 Escape Sequences 29

Lesson 3.3 String Methods: Measure and Change Case 30

Lesson 3.4 Slicing, Replacing, and Immutability 32

Mini-Project 1 Mad Libs Story Machine 39

Chapter 4 Numbers and Math 41

Lesson 4.1 The Math Object: Rounding and Powers 41

Lesson 4.2 Limits, Precision, and toFixed 42

Lesson 4.3 Random Numbers for Games 43

Lesson 4.4 Converting Text to Numbers (and NaN) 45

Chapter 5 Making Decisions 52

Lesson 5.1 Comparisons Produce Booleans 52

Lesson 5.2 if, else, and else if 53

Lesson 5.3 Logical Operators: &&, ||, ! 55

Lesson 5.4 switch: Choosing Among Many Options 56

Chapter 6 Loops 64

Lesson 6.1 The while Loop . . . 64
Lesson 6.2 The for Loop . . . 65
Lesson 6.3 break and continue . . . 66
Lesson 6.4 Nested Loops and Patterns . . . 68

Mini-Project 2 Number Guessing Game . . . 76

Chapter 7 Functions . . . 78
Lesson 7.1 Defining and Calling Functions . . . 78
Lesson 7.2 Parameters, Arguments, and return . . . 79
Lesson 7.3 Scope and Arrow Functions . . . 80
Lesson 7.4 Documenting and Defending: JSDoc and try/catch . . . 82

Chapter 8 Arrays . . . 91
Lesson 8.1 Creating Arrays and Reading Items . . . 91
Lesson 8.2 Adding and Removing: push, pop, shift, unshift . . . 92
Lesson 8.3 Walking Through Arrays . . . 93
Lesson 8.4 Searching, Joining, and Slicing . . . 95
Lesson 8.5 Transforming Arrays: map, filter, reduce, splice . . . 96

Chapter 9 Objects . . . 104
Lesson 9.1 Properties: Dot and Bracket Access . . . 104
Lesson 9.2 Arrays of Objects: Real Records . . . 105
Lesson 9.3 Objects with Methods . . . 107
Lesson 9.4 JSON: Objects as Text . . . 108
Lesson 9.5 Looping Over Objects: keys, values, entries . . . 109

Mini-Project 3 Student Gradebook . . . 118

Chapter 10 Files, Modules, and Real Programs . . . 120
Lesson 10.1 Modules: Sharing Code Between Files . . . 120
Lesson 10.2 Reading Files with readFileSync . . . 121
Lesson 10.3 Writing and Appending Files . . . 122
Lesson 10.4 Organizing a Real Program . . . 124

Final Project Study Log Manager . . . 132

Glossary . . . 136

Answer Key **139**

Lesson Check Answers 139

Practice Answers 165

Mixed Quiz Answers 175

Challenge Solutions 192

Index **209**

What's Next **214**

How to Use This Book

Welcome! This book was written for school students and self-learners who have never programmed before. You do not need any special software, any mathematics beyond everyday arithmetic, or anyone's permission to start. All you need is curiosity and a little patience.

The book has ten chapters. Most chapters contain four lessons, and chapters 8 and 9 close with a fifth lesson on modern power tools - the array and object methods that real-world JavaScript uses every day. Every lesson teaches one small, complete idea. Lessons move slowly on purpose: first you read a short explanation, then you see the idea as runnable code inside a tinted box, and then you read a tip, a note, or a warning that shows you how the idea is used in the real world.

One honest note about scope: this book teaches the JavaScript LANGUAGE itself, and every example runs in the console or in Node.js. It does not cover HTML, CSS, web-page design, or browser events - that is the step AFTER this book, and a friendly one. Learn the language first and the web becomes easy, because every interactive web page is powered by exactly the ideas you will practice here: values, decisions, loops, functions, arrays, and objects.

After every lesson comes a **Lesson Check**: four multiple choice questions that take about two minutes each. Some questions look similar on purpose - read every word before answering. After each chapter you will meet four bigger sections. **Practice Questions** (ten per chapter) mix six different styles, from simple multiple choice to writing your own code. The **Mixed Quiz** (fifteen questions) covers the whole chapter at once, just like a school test. The five **Challenges** are hard thinking problems; each one has a hint, and the Answer Key contains a full step-by-step solution. Finally, the **Chapter Recap** sums up the key ideas, and **Common Mistakes** shows the errors almost every beginner makes - and how to fix them.

After chapters 3, 6, and 9 you will build a **Mini-Project** - a complete, runnable program with a line-by-line explanation and ideas for making it your own. After chapter 10, the **Final Project** combines everything you have learned into one real program that saves information in files, exactly the way serious programs do.

The back of the book holds three study tools. The **Glossary** defines eighty-four terms in plain language. The **Answer Key** explains the correct answer to every Lesson Check, Practice, and Mixed Quiz question - including why the wrong options are wrong - and solves every challenge step by step. The **Index** at the very end tells you exactly which lesson introduced any topic, function, or keyword, so you can jump back for a reminder.

A friendly suggestion: type every example by hand, break it on purpose, and fix it again. Programming is a skill of the hands as much as of the eyes. Work through the checks and quizzes honestly before looking at answers, and you will be amazed how quickly the ideas become your own.

What you will find in every chapter

Lessons	Four or five short lessons that each teach one complete idea, with runnable code examples.
Lesson Check	Four quick multiple choice questions after each lesson.
Practice Questions	Ten mixed exercises: multiple choice, true/false, output prediction, bug hunting, short answers, and code writing.
Mixed Quiz	Fifteen questions covering the whole chapter.
Challenges	Five hard problems with hints and full solutions in the Answer Key.
Chapter Recap	A bullet list of the key ideas to review.
Common Mistakes	Beginner traps shown with the fix for each one.

What to Learn Next

Finish the Final Project and you will hold the complete language core: values, strings, numbers, decisions, loops, functions, arrays, objects, and files. The natural next step is the web itself. HTML gives a page its structure, CSS gives it style, and the DOM (document object model) is how JavaScript reaches into that page to change text, react to clicks, and respond to typing. Those three, built on everything you practiced in this book, are what turn a console programmer into a web programmer.

After the DOM, the usual road continues with fetch - asking servers on the internet for data - then a deeper dive into Node.js and its enormous library catalog, and eventually a framework such as React for building full applications. You will also meet the tools from Lessons 8.5 and 9.5 in nearly every tutorial you open: map, filter, reduce, Object.entries, and optional chaining were added to this edition for exactly that reason. Take the next step at your own pace - everything you built here travels with you.

CHAPTER 1

Hello, JavaScript

Welcome to the very first step of your programming journey. In this chapter you will learn what a program really is, how JavaScript fits into the world of technology around you, and how to run your very first lines of code. Nothing here requires any previous experience - if you can type a sentence, you can write a program.

By the end of this chapter you will be able to open a JavaScript console, print messages with console.log(), control the order in which your statements run, and make your code friendly to read with comments. These small skills are the foundation for everything that follows, so take your time and enjoy the process.

Lesson 1.1 - What Is Programming?

A computer program is a list of instructions that tells a computer exactly what to do. The computer follows your instructions one at a time, in order, and it does precisely what you say - nothing more and nothing less. If the instructions say "add these two numbers", the computer adds them; if the instructions are unclear, the computer stops and reports a problem instead of guessing what you meant.

Writing those instructions is called programming, and the language we write them in is called a programming language. Humans speak English, Arabic, Spanish, and hundreds of other languages; computers speak languages such as JavaScript, Python, and Java. A programming language has strict rules about spelling and punctuation, called syntax, so that the computer can read your instructions without confusion.

JavaScript is one of the most popular programming languages on Earth. It was created in 1995 so that web pages could respond to the user, and today it runs everywhere: inside every web browser, on servers that power websites, in mobile apps, and even in robots and games. When you press a like button, see a countdown timer, or drag a map on a phone, JavaScript is usually the language doing the work.

Your first taste of JavaScript

```
// A tiny JavaScript program
console.log("Hello, world!");
console.log("I am learning JavaScript.");
```

Do not worry about the details of that program yet. For now, just notice that it contains two instructions, one per line, and that each instruction ends with a semicolon. When this program runs, it prints two short messages on the screen. You will be able to run it yourself in the next lesson, and by the end of this book you will understand every symbol in it.

NOTE

JavaScript has no connection to the Java language despite the similar name. The name was a marketing decision from 1995, and it has confused students ever since. JavaScript and Java are two completely different languages.

Lesson Check 1.1

Choose the best answer. Answers are in the Answer Key.

1. What is a computer program?

A. A picture stored on the internet

B. A machine that stores files

C. A list of instructions that tells a computer what to do

D. A type of computer memory

2. Which statement about JavaScript is true?

A. It is the same language as Java

B. It runs in web browsers and also outside them, for example with Node.js

C. It only runs inside the game industry

D. It was invented before the internet existed

3. The strict spelling and punctuation rules of a programming language are called...

A. grammar school

B. alphabet

C. keyboards

D. syntax

4. What does the computer do if an instruction is unclear?

A. It stops and reports a problem

B. It guesses what you probably meant

C. It skips the instruction and continues

D. It asks another computer what to do

Lesson 1.2 - Running Your First Code

JavaScript code needs a place to run, and you already own two of them. The first is your web browser. Browsers such as Chrome, Edge, and Firefox include a JavaScript console: a small window where you can type code and see results immediately. You can usually open it by pressing the F12 key, or by right-clicking a web page and choosing "Inspect" and then the "Console" tab.

The second place is Node.js, a free program that runs JavaScript on your computer without any browser at all. Node.js lets you build larger programs that you save in files, and later in this book it will let your programs read and write files on your computer. For the first chapters, the browser console is the quickest way to experiment, because there is nothing to install and nothing to save.

The most useful tool you will meet today is console.log(). It is a built-in function, which means it is a piece of ready-made behavior that JavaScript provides for you. You hand it a message inside parentheses, and it prints that message on the screen. Note that console.log is a function you call - it is not a keyword, and it is not part of the language grammar itself.

Printing with console.log()

```
console.log("Cookies are delicious.");
console.log(2 + 3);
```

That second line may surprise you. When you give console.log() a small calculation instead of a message, JavaScript first works out the answer and then prints the result. So the program above prints the text about cookies and then the number 5. A computer is at its heart a very fast calculator, and you will use that fact constantly.

TIP

Type code by hand instead of copying and pasting it. Your fingers learn the punctuation, and the small mistakes you make and fix will teach you more than any perfect copy-paste ever could.

WARNING

The console is case-sensitive. console.log with a lowercase c works, but Console.log or Console.Log does not exist and will produce an error. Almost everything in JavaScript is lowercase, so type carefully.

Lesson Check 1.2

Choose the best answer. Answers are in the Answer Key.

1. Which tool lets you type JavaScript and see results immediately inside a browser?

A. A calculator app
B. The developer console
C. The address bar
D. The bookmarks menu

2. console.log() is best described as...

A. a comment
B. a web browser
C. a built-in function you can call
D. a keyword of the language grammar

3. What does console.log(2 + 3); print?

A. 2 + 3
B. 23
C. Nothing, it is an error
D. 5

4. You type Console.log("hi"); and get an error. Why?

A. JavaScript is case-sensitive and the name is spelled wrong
B. Semicolons are not allowed there
C. The message is too short
D. console.log needs two inputs

Lesson 1.3 - Statements Run in Order

A statement is a single complete instruction, and in JavaScript statements usually end with a semicolon. A program is simply a list of statements, and the computer normally runs them from the top of the file to the bottom, one statement at a time. This top-to-bottom order is called the flow of the program, and understanding it is the first step toward predicting what a program will do.

Order matters

```
console.log("First");
console.log("Second");
console.log("Third");
```

The program above always prints its three messages in the same order: First, then Second, then Third. If you swap two lines, the output changes in exactly the same way. This may look obvious, but it is a powerful idea: every program you will ever write, including huge ones, is built from this simple rule of running statements in order.

Because the computer moves strictly downward, a statement can only use things that the computer has already seen. It is like reading a recipe: you cannot stir the batter before the ingredients exist. Later chapters will show you techniques such as functions that let you organize code so that it can be reused, but the flowing, top-to-bottom execution never goes away.

What happens when something goes wrong? If JavaScript cannot understand a statement - because of a missing quote, a missing parenthesis, or a misspelled word - it reports an error and the program stops at that point. Statements before the broken line have already run; statements after it never run at all. Reading error messages calmly is a core skill for every programmer.

NOTE

Some programmers omit semicolons and rely on automatic semicolon insertion. That style works, but while you are learning it is safer to end every statement with a semicolon so your intention is always visible.

Lesson Check 1.3

Choose the best answer. Answers are in the Answer Key.

1. A program has three console.log statements in this order: "A", then "B", then "C". What prints first?

A. All three at exactly the same moment

B. A, because statements normally run from top to bottom

C. C, because the console shows the newest line on top

D. The order is random each run

2. Which symbol normally ends a JavaScript statement?

A. A semicolon (;)

B. A period (.)

C. A colon (:)

D. An exclamation mark (!)

3. An error happens on line 3 of a 5-line program. What has run by then?

A. Lines 1 and 2 only

B. Lines 1, 2, 4, and 5

C. Nothing at all

D. All five lines

4. Why can a statement normally not use something the computer has not seen yet?

A. Because later lines are always comments

B. Because the console hides future lines

C. Because JavaScript downloads them slowly

D. Because the computer runs top to bottom and has not reached that part

Lesson 1.4 - Comments and Clean Code

Comments are notes that you write inside your program for humans to read. The computer completely ignores them, so they never change what the program does. A comment starts with two forward slashes and continues to the end of the line. Programmers use comments to explain why the code exists, to leave reminders, and to temporarily switch lines off while testing.

Writing comments

```
// This program greets the user three times.
console.log("Welcome!");   // a note at the end of a line

// console.log("This line is switched off.");
console.log("Goodbye!");
```

For longer explanations you can write a multi-line comment between a slash-star and a star-slash pair. Everything between those two markers is ignored, even if it covers many lines. Multi-line comments are also handy for switching off a whole block of code while you test an alternative version.

A multi-line comment

```
/*
This program was written by a student
as part of Chapter 1 practice.
*/
console.log("Comments are ignored by the computer.");
```

Clean code is code that other humans can read easily, and the "other human" is often you in three weeks. Use spaces and blank lines to separate ideas, keep each statement on its own line, and choose comments that add real meaning instead of repeating the code. A comment that says "print the message" above a console.log line helps nobody; a comment that explains a tricky calculation helps everybody.

TIP

A good habit: before solving a problem, write a short comment that describes the plan in plain words, then fill in the code underneath. The comment becomes your checklist, and your future self will thank you.

Lesson Check 1.4

Choose the best answer. Answers are in the Answer Key.

1. What does the computer do with a comment?

A. It prints it on the screen

B. It treats it as an error

C. It translates it into another language

D. It ignores it completely

2. Which line correctly starts a single-line comment?

A. // note here

B. ** note here

C. ## note here

D. <!-- note here -->

3. What is a good use of a multi-line comment?

A. Making the program run faster

B. Storing a password safely

C. Explaining a longer idea or switching off a block of code while testing

D. Printing several messages at once

4. Read carefully. A student writes this program:
```
// print the greeting
console.log("Hi");
```
Which of the following is TRUE?

A. The comment replaces the console.log line, so nothing prints

B. The program prints "Hi" only, because the comment is ignored

C. The program prints "// print the greeting" and then "Hi"

D. The program shows an error because of the comment

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. Which of these is NOT a place where JavaScript normally runs?

A. Inside a web browser

B. On a server with Node.js

C. In mobile apps built with JavaScript tools

D. Inside a refrigerator that was never programmed

2. What does the statement console.log("7"); print on the screen?

A. It prints nothing, because numbers are invisible

B. It prints an error message

C. It prints the text 7, without the quotation marks

D. It prints the number 14

3. Which order of statements prints "Morning", "Noon", "Night"?

A. console.log("Morning"); then console.log("Noon"); then console.log("Night");

B. console.log("Noon"); then console.log("Morning"); then console.log("Night");

C. console.log("Morning"); then console.log("Night"); then console.log("Noon");

D. console.log("Night"); then console.log("Morning"); then console.log("Noon");

4. A friend claims comments make programs run faster. What is the truth?

A. Comments are ignored, so they make no difference to speed

B. Comments always make programs much faster

C. Comments make programs run much slower

D. Only long comments make programs slower

5. Which program prints two separate lines of output?

A. console.log("One"); and then console.log("Two"); on the next line

B. console.log("One Two");

C. console.log(One); and then console.log(Two);

D. console.log "One"; and then console.log "Two";

6. In JavaScript, console.log is a keyword of the language.

☐ True ☐ False

7. What does this program print? Write the exact output lines.

```
console.log("Top");
// console.log("Middle");
console.log("Bottom");
```

Output: _____

8. This program should print two messages but stops with an error on line 1. Find the bug.

```
consol.log("Welcome");
console.log("Good luck");
```

Bug: _____

9. In your own words, explain why statements run in order matters when you write a program. Give one example.

_____

_____

_____

10. Write a three-line program that prints your name, your favorite color, and the number of days in a week, each with its own console.log statement.

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. JavaScript was created so that web pages could...

A. replace HTML completely
B. run only inside databases
C. respond to the user
D. stay offline forever

2. Which is the correct way to write the print statement?

A. Console.log("Hi");
B. console.log(Hi);
C. print.log("Hi");
D. console.log("Hi");

3. How many statements are in this program?

```
console.log("A");
// console.log("B");
console.log("C");
```

A. 1
B. 2
C. 3
D. 4

4. What happens when JavaScript meets a statement it cannot understand?

A. It asks the user to fix it before stopping
B. It reports an error and the program stops at that point
C. It rewrites the statement automatically
D. It skips it and continues normally forever

5. Which line is a multi-line comment?

A. // like this
B. # like this
C. <!-- like this -->
D. /* like this */

6. A comment is best described as...

A. a way to print hidden text
B. a note for humans that the computer ignores
C. an instruction the computer runs quietly
D. a type of error message

7. What does this print?

```
console.log("Book");
console.log("Pen");
```

A. Book on one line, then Pen on the next line
B. BookPen on one line
C. An error, because order is wrong
D. Pen on one line, then Book on the next line

8. Which habit makes code easiest for humans to read?

A. Deleting all comments to keep files small
B. Mixing capital and small letters randomly
C. One statement per line, helpful comments, consistent spacing
D. Writing everything on one long line

9. A program with an error on line 3 still runs lines 1 and 2 before stopping.

☐ True ☐ False

10. The developer console can be opened in most browsers by pressing F12.

☐ True ☐ False

11. Multi-line comments must fit on exactly two lines.

☐ True ☐ False

12. Explain the difference between a statement and a comment. Use one example of each.

_____

_____

_____

13. Why do you think programmers are told to type code by hand instead of copy-pasting while learning? Give two reasons.

_____

_____

_____

14. Write a complete two-line program that prints the word JavaScript and the number 1995.

_____

_____

_____

15. Write a program with one comment and two console.log statements that print "Day 1" and "Day 2".

_____

_____

_____

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. Write a program that prints a countdown: 3, 2, 1, and then the word "Go!", using four console.log statements. Then explain in a comment why the order of your lines matters.

Hint: Each number needs its own statement, and they must appear in top-to-bottom order.

Challenge 2. Predicting output. Without running it, write down exactly what this program prints, including which lines do nothing:

Hint: Two of the lines are comments. Comments produce no output at all.

Challenge 3. Fix and improve. The program below has three bugs (two spelling problems and one missing quote). Find and fix them all:

Hint: Check the capital C, the missing l in log, and count the quotation marks on each line.

Challenge 4. Write a program that draws a simple flag or picture using console.log output only. For example, print a triangle using the * character over three lines. Then add a comment explaining your picture.

Hint: Each console.log prints one line, so think of the picture as rows of text, one row per statement.

Challenge 5. The read-aloud test. Write a four-line program for your morning routine, then swap two of its lines. Write down both versions and explain what changed in the output.

Hint: Output order always matches statement order. Swapping statements swaps their output lines exactly.

Chapter Recap

- A program is a list of instructions; programming is writing those instructions in a language with strict syntax rules.
- JavaScript runs in web browsers (developer console) and outside them with Node.js.
- console.log() is a built-in function that prints values; it is a function you call, not a keyword.
- Statements normally run from top to bottom, one at a time.
- A statement that the computer cannot understand causes an error and stops the program at that point.
- Comments start with // for one line or sit between /* and */ for many lines; the computer ignores them.
- JavaScript is case-sensitive: console.log works, Console.log does not.
- Clean code uses one statement per line, consistent spacing, and comments that explain reasons, not the obvious.

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Wrong capitalization

```
Console.log("Hi");
```

Fix: JavaScript is case-sensitive. The function is console.log with a lowercase c. Always match names letter for letter.

```
console.log("Hi");
```

Mistake 2: Missing quotes around text

```
console.log(Hello);
```

Fix: Text (called a string) must be wrapped in quotes. Without them, the computer looks for a name called Hello and reports an error.

```
console.log("Hello");
```

Mistake 3: Forgetting the parentheses

```
console.log"Ready";
```

Fix: console.log is a function, so the input goes inside parentheses immediately after the name.

```
console.log("Ready");
```

Mistake 4: Typing a comment as if it were code

```
// console.log("Run");
```

Fix: This line is commented out, so nothing prints. Remove the slashes when you want the statement to run - or use them on purpose to switch a line off.

```
console.log("Run");
```

Mistake 5: Putting two statements on one crowded line

```
console.log("A"); console.log("B");
```

Fix: One statement per line makes programs easy to scan and debug. Split the line into two lines, one statement each.

```
console.log("A");
console.log("B");
```

CHAPTER 2

Variables and Values

Programs would be almost useless if they could only print fixed messages. Real programs remember things: a score that grows, a name that the user types, a temperature that changes. In this chapter you will meet variables - the labeled boxes that let a program remember - and the main kinds of values that go inside them.

You will learn how to create variables with let and const, how to do arithmetic with numbers, how to join text together, and how to ask JavaScript what type of value you are holding. These four skills appear in every single program you will ever write, so master them here and everything after this chapter becomes easier.

Lesson 2.1 - Labels for Values: let and const

A variable is a named storage place for a value. Imagine a cardboard box with a label on it: the label is the variable name, and what you put inside is the value. You create a variable with the keyword let, followed by a name, an equal sign, and the starting value. The equal sign here does not mean "equals" as in mathematics; it means "store the right side into the left side".

Creating variables with let

```
let score = 0;
let playerName = "Maya";
console.log(playerName);
console.log(score);
```

Once a variable exists, you can change what is inside it simply by assigning again. The old value disappears and the new value takes its place. This is why they are called variables - their contents can vary while the program runs. Reading a variable name in your code always gives back the value currently stored there.

Changing what is inside

```
let score = 0;
console.log(score);
score = 10;
console.log(score);
score = score + 5;
console.log(score);
```

The const keyword creates a variable that cannot be reassigned after its first value. Use const for values that should stay the same from start to finish, such as a maximum number of players or a tax rate. If you try to assign a new value to a const, JavaScript reports a TypeError and stops. Choosing const by default makes your intentions visible: readers instantly know this value never changes.

const protects the value

```
const DAYS_IN_WEEK = 7;
console.log(DAYS_IN_WEEK);
// DAYS_IN_WEEK = 8;  // TypeError: assignment to constant
```

TIP

A popular style: use const for everything that never changes and let only for values that really do change. This tiny habit prevents a whole family of "who changed this?" bugs.

NOTE

You may also meet the older keyword var in code you find online. Modern JavaScript rarely needs it: var has surprising scoping rules that caused so many bugs that let and const were added to replace it. This book uses let and const only.

Lesson Check 2.1

Choose the best answer. Answers are in the Answer Key.

1. Which line correctly creates a variable named city with the value "Cairo"?

A. city let = "Cairo";
B. let city = "Cairo";
C. const = "Cairo";
D. let "city" = Cairo;

2. What does this program print?

```
let lives = 3;
lives = 2;
console.log(lives);
```

A. 32
B. 3
C. 2
D. An error

3. When should you choose const instead of let?

A. When the value is a number
B. When the variable is used many times
C. When the program is long
D. When the value should never be reassigned

4. What happens if a program assigns a new value to a const?

A. The const becomes a let automatically
B. The old value is stored twice
C. JavaScript reports a TypeError and stops
D. The value quietly changes

Lesson 2.2 - Numbers and Arithmetic

JavaScript handles numbers without any fuss: whole numbers like 42 and decimals like 3.5 are both simply numbers. You can add with +, subtract with -, multiply with *, and divide with /. There are two extra operators that beginners find delightful: the remainder operator %, which gives what is left over after a division, and the exponent operator **, which raises a number to a power.

The arithmetic operators

```
console.log(9 + 4);    // 13
console.log(9 - 4);    // 5
console.log(9 * 4);    // 36
console.log(9 / 4);    // 2.25
console.log(9 % 4);    // 1  (9 = 2*4 + 1 left over)
console.log(2 ** 5);   // 32
```

The remainder operator % (often read as "mod") is perfect for questions like "is this number even?" - an even number has remainder 0 when divided by 2. It also answers "how many cookies are left after sharing them equally", which is exactly the kind of everyday problem programs solve all the time.

When an expression mixes several operators, JavaScript follows the same precedence rules as school mathematics: first **, then * / %, then + and -. Parentheses override the default order and also make your intention obvious to human readers. When in doubt, add parentheses - they cost nothing and prevent confusion.

Precedence and parentheses

```
console.log(2 + 3 * 4);      // 14, not 20
console.log((2 + 3) * 4);    // 20
console.log(2 ** 3 * 2);     // 16 (** runs first)
```

WARNING

The = sign stores a value; it never checks equality. Writing score = 10 puts 10 into score. Testing whether two values are equal is done with the triple equal sign ===, which you will meet in Chapter 5. Mixing these up is a classic beginner bug.

Lesson Check 2.2

Choose the best answer. Answers are in the Answer Key.

1. What is the value of the expression 17 % 5?

A. 2 **B.** 3

C. 3.4 **D.** 12

2. Which operator raises a number to a power?

A. % **B.** **

C. ^ **D.** *

3. What does console.log(2 + 3 * 4); print?

A. 10 **B.** 14

C. 20 **D.** 24

4. What is the value of 2 ** 3 * 2?

A. 12 **B.** 16

C. 64 **D.** 128

Lesson 2.3 - Strings and Concatenation

A string is a piece of text wrapped in quotes, such as "code" or 'hello'. Both single and double quotes work; this book uses double quotes for consistency. The quotes tell JavaScript "this is text, not a variable name or a command", and the quotes themselves are not part of the text. A string can be empty ("") or hold thousands of characters.

Joining strings together is called concatenation, and in JavaScript you do it with the same + sign that adds numbers. When both sides are strings, + glues them together end to end. You can concatenate any number of

pieces, including variables that hold strings.

Concatenation with +

```
let first = "choco";
let second = "late";
let word = first + second;
console.log(word);
console.log("I love " + word + "!");
```

Something interesting happens when you mix strings and numbers with +. Because the left side is a string, JavaScript converts the number to text and concatenates instead of adding: "Page " + 3 gives "Page 3". But an expression is evaluated left to right, so "3" + 4 + 5 gives "345", while 3 + 4 + "5" gives "75". This left-to-right behavior surprises many beginners, so watch for it in the exercises.

Mixing strings and numbers with +

```
console.log("Page " + 3);   // "Page 3"
console.log("3" + 4 + 5);   // "345"
console.log(3 + 4 + "5");   // "75"
```

TIP

Whenever you see + between values, ask yourself: is either side a string? If yes, the whole chain becomes text joining from that point onward. Chapter 3 shows template literals, a cleaner way to mix text and values.

Lesson Check 2.3

Choose the best answer. Answers are in the Answer Key.

1. What does "sun" + "flower" evaluate to?

A. An error, because two strings cannot be added
B. "sun flower"
C. "flower sun"
D. "sunflower"

2. What does "3" + 4 + 5 evaluate to?

A. "12"
B. 12
C. "345"
D. An error

3. What does 3 + 4 + "5" evaluate to?

A. "75"
B. "345"
C. 12
D. "7" + "5" as an error

4. What does console.log("Page " + 3); print?

A. NaN
B. Page 3
C. Page3
D. "Page 3" including quotes

Lesson 2.4 - Booleans, typeof, and Good Names

The third core value type is the boolean: a value that is either true or false, nothing else. Booleans look tiny but they are the fuel of every decision a program makes. Chapter 5 is dedicated to using them; for now, meet them and learn how to ask JavaScript about types in general.

The three core types

```
let age = 12;              // number
let name = "Omar";         // string
let isStudent = true;      // boolean
console.log(age, name, isStudent);
```

The typeof operator tells you the type of any value as a string: "number", "string", "boolean", and more. It is mainly a debugging tool - when a program behaves strangely, printing typeof for a variable often reveals that a "number" is secretly a string. One famous quirk: typeof null reports "object" even though null means "no value"; this is a historical mistake that JavaScript keeps for compatibility.

Asking about types

```
console.log(typeof 12);        // "number"
console.log(typeof "12");      // "string"
console.log(typeof true);      // "boolean"
console.log(typeof undefined); // "undefined"
```

Choosing good variable names is a real skill. Names may contain letters, digits, underscores, and dollar signs, but they cannot start with a digit and cannot be reserved words such as let or const. JavaScript programmers use camelCase: the first word is lowercase and every later word starts with a capital, like totalPrice or playerHighScore. A name should describe what the value is - six months from now, "x" will tell you nothing but "remainingLives" will tell you everything.

Names: legal, illegal, and clear

```
let totalPrice = 19.99;      // good camelCase
let user2 = "Sam";           // legal, digit not first
// let 2user = "Sam";        // SyntaxError: starts with
                             // a digit
// let let = 5;              // SyntaxError: reserved word
```

WARNING

JavaScript treats a variable that was never created as undefined, and using a misspelled name is the most common way that happens. If you see "undefined" where you expected a value, check the spelling of your variable name first.

Lesson Check 2.4

Choose the best answer. Answers are in the Answer Key.

1. What does the expression typeof 42 evaluate to?

A. "digit"

B. 42

C. "number"

D. "integer"

2. Which variable name is illegal in JavaScript?

A. totalPrice **B.** user2
C. _hidden **D.** 2ndPlayer

3. What is the value stored in a variable that was never created?

A. undefined **B.** 0
C. null **D.** An empty string

4. Which statement about typeof null is true?

A. It reports "object", a famous historical quirk **B.** It reports "null"
C. It reports "undefined" **D.** It causes a SyntaxError

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. A program keeps a player's maximum allowed score fixed at 100. Which declaration fits best?

A. let MAX_SCORE = 100; **B.** MAX_SCORE = 100;
C. const "MAX_SCORE" = 100; **D.** const MAX_SCORE = 100;

2. What does console.log(10 % 3); print?

A. 0 **B.** 1
C. 3 **D.** 3.33

3. Which expression equals 45?

A. 6 + 4 * 10 **B.** (5 + 4) * 10 / 2
C. 5 * 4 + 10 **D.** 54 - 10

4. What does console.log("a" + 1 + 2); print?

A. 3 **B.** An error
C. "a12" **D.** "a3"

5. Which name follows camelCase and clearly describes a value?

A. TP **B.** remaininglives
C. Remaining Lives **D.** remainingLives

6. The statement let 3cats = 3; is valid JavaScript.

☐ True ☐ False

7. What does this program print? Write every output line.

```
let x = 5;
x = x + 2;
console.log(x);
x = x % 3;
console.log(x);
```

Output: _____

8. The programmer expected 12 but the program prints something else. Find the bug.

```
let total = "1" + 2;
console.log(total);
```

Bug: _____

9. Explain the difference between let and const in your own words, and give one example situation where each is the better choice.

_____

_____

_____

10. Write a program that creates two variables (apples = 3, bananas = 4), prints the total number of fruits, then prints a sentence using concatenation, like "I have 7 fruits".

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. Which statement creates a variable correctly?

A. let temperature = 21;
B. temperature let = 21;
C. let = temperature 21;
D. let temperature: 21;

2. What does 23 % 10 evaluate to?

A. 2
B. 2.3
C. 3
D. 23

3. What does console.log("5" * 2); print? (Hint: * behaves differently from +.)

A. "55" **B.** 10

C. "10" **D.** An error

4. After these statements, what is in score?

```
const score = 50;
// score = 60;
```

A. undefined **B.** A TypeError stops the program

C. 50 **D.** 60

5. Which expression uses parentheses to force addition before multiplication?

A. 2 * 3 + 4 **B.** 2 + (3 * 4)

C. (2 + 3) * 4 **D.** 2 + 3 * 4

6. What is the type of the value "true" (with quotes)?

A. boolean **B.** string

C. undefined **D.** number

7. What does this print?

```
let msg = "Go";
msg = msg + "!";
console.log(msg); (assume a normal semicolon)
```

A. Go! **B.** Go

C. "Go!" **D.** An error

8. Why do programmers prefer descriptive names like remainingLives instead of short ones like x?

A. Longer names make programs run faster **B.** Short names are illegal in JavaScript

C. The computer requires at least 5 letters **D.** Descriptive names make code easier for humans to understand and maintain

9. The line score = 10; changes a variable's value only if score was created earlier with let.

☐ True ☐ False

10. The expression "10" + 5 evaluates to the number 15.

☐ True ☐ False

11. typeof "hello" evaluates to "string".

☐ True ☐ False

12. A classmate wrote: const total = 10; total = total + 5; Explain exactly what happens when this runs and how to fix it.

13. What is the difference between "7" (in quotes) and 7 (without quotes)? Mention both type and one behavior difference with +.

14. Write a program that stores 12 in a variable rows and 8 in columns, then prints the area (rows times columns) and a message like "Area: 96".

15. Write a program that prints the last digit of 12345 using the % operator, and the rest of the number using division.

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. The swap problem. Create two variables, a = 1 and b = 2. Using a third variable, swap their values so a becomes 2 and b becomes 1, then print both.

Hint: You need a temporary box to hold one value while you move the other.

Challenge 2. Digit extractor. For the number 987, print its hundreds digit (9), tens digit (8), and ones digit (7) using only arithmetic - no string tricks.

Hint: Dividing by 100 and using % will isolate each digit. Remember % gives the remainder.

Challenge 3. Minute converter. A movie lasts 137 minutes. Print how many whole hours and leftover minutes that is (2 hours and 17 minutes) using / with Math.floor and %.

Hint: Math.floor(137 / 60) gives whole hours; 137 % 60 gives the leftover minutes.

Challenge 4. Receipt line. Create variables item = "Notebook", price = 4.5, and quantity = 3. Print exactly: 3 x Notebook = 13.5 - using concatenation and the * operator, without typing the 13.5 yourself.

Hint: Compute price * quantity inside the concatenation and watch where spaces go.

Challenge 5. Type detective. Without running the code first, predict the output of these four lines, then run and check yourself:

Hint: Track the type of the running total in line 3. Once a string joins, + switches to gluing.

Chapter Recap

- A variable is a named storage place; create one with let or const followed by a name, =, and a value.
- let allows reassignment; const forbids it and fails with a TypeError if you try.
- Arithmetic operators: + - * / plus % for remainders and ** for powers.
- Operator precedence: ** first, then * / %, then + and -; parentheses override and clarify.
- Strings are quoted text; + concatenates strings, and once a string appears in a + chain, everything joins as text.
- Evaluation of + chains runs left to right, so 3 + 4 + "5" is "75".
- Booleans are true/false values; typeof reports any value's type as a string.
- Use camelCase names that describe the value; names cannot start with a digit or be reserved words.
- var is legacy JavaScript; prefer let and const everywhere.

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Using = to compare

```
if (score = 10) { /* meant to compare */ }
```

Fix: The single = assigns; it does not compare. Comparisons use ===. This mistake is covered fully in Chapter 5.

```
if (score === 10) { /* compares properly */ }
```

Mistake 2: Forgetting that "3" + 4 is text

```
let total = "3" + 4;
console.log(total);  // "34"
```

Fix: A leading string turns + into concatenation. Remove the quotes when you want arithmetic.

```
let total = 3 + 4;
console.log(total);  // 7
```

Mistake 3: Reassigning a const

```
const speed = 50;
speed = 80;
```

Fix: const values cannot be reassigned. Use let for values that change.

```
let speed = 50;
speed = 80;
```

Mistake 4: Starting a name with a digit

```
let 1stPlace = "Ann";
```

Fix: Names cannot begin with a digit. Move the digit or spell the word.

```
let firstPlace = "Ann";
```

Mistake 5: Expecting a space in concatenation

```
console.log("Hello" + "World");
```

Fix: Concatenation adds nothing between pieces. Include the space inside one of the strings.

```
console.log("Hello" + " " + "World");
```

CHAPTER 3

Working with Strings

Text is everywhere in programs: names, messages, labels, stories, search boxes. Chapter 2 introduced strings as quoted text; this chapter turns you into a string specialist. You will learn the modern way to build sentences with template literals, how to embed invisible characters like new lines and tabs, and how to use the built-in string methods that every JavaScript programmer uses daily.

By the end you will be able to measure a string, change its case, trim unwanted spaces, search inside it, cut out pieces, and replace words - and you will understand the one rule that surprises every beginner: strings themselves never change. Every method gives you back a brand-new string.

Lesson 3.1 - Template Literals

Chapter 2 built sentences with + concatenation, which works but gets messy as sentences grow longer. Template literals are the modern, cleaner tool. A template literal is a string wrapped in backticks - the ` character, usually on the same key as the tilde ~ - and it can hold placeholders written as ${...}. Whatever expression you put inside the braces is evaluated and inserted into the text at that spot.

Building a sentence with a template literal

```
let name = "Amina";
let score = 95;
console.log(`${name} scored ${score} points!`);
```

Compare that with the concatenation version: name + " scored " + score + " points!". The template literal version reads almost like the final sentence, the quotation-mark juggling disappears, and the ${...} placeholders can hold any expression - arithmetic, variables, even method calls. JavaScript evaluates each placeholder and converts the result to text automatically.

Placeholders can hold expressions

```
let apples = 3;
let friends = 4;
console.log(`Each friend gets ${apples * friends / 12}`);
console.log(`Uppercase name: ${"amina".toUpperCase()}`);
```

Template literals have one more superpower: they can span several lines. A normal quoted string breaks with an error if you press Enter in the middle of it, but a template literal keeps whatever line breaks you type. This makes them perfect for printing small formatted blocks, such as a receipt or a poster.

A multi-line template literal

```
let item = "Poster";
let price = 8;
let receipt = `--- Receipt ---
Item: ${item}
Price: ${price} dollars`;
console.log(receipt);
```

TIP

Backticks look similar to single quotes on many keyboards but they are a different character. If a template literal shows up as an error, check that you really typed ` and not ' or ".

Lesson Check 3.1

Choose the best answer. Answers are in the Answer Key.

1. How do you write a template literal?

A. With asterisks: *text ${expr}*
B. With backticks: `text ${expr}`
C. With double quotes: "text ${expr}"
D. With single quotes: 'text ${expr}'

2. What does `2 + 2 = ${2 + 2}` evaluate to?

A. "2 + 2 = 4"
B. An error
C. "4"
D. "2 + 2 = ${2 + 2}"

3. Which feature do template literals have that quoted strings lack?

A. They can span multiple lines naturally
B. They run faster
C. They can contain variables without placeholders
D. They ignore spelling mistakes

4. What does `Sum: ${3 + 4}` evaluate to?

A. "Sum: 3 + 4"
B. "Sum: ${3 + 4}"
C. 7
D. "Sum: 7"

Lesson 3.2 - Escape Sequences

Some characters are hard to type directly inside a string: a new line, a tab, or a quotation mark that would end the string early. JavaScript solves this with escape sequences - a backslash followed by a character, which together mean "insert something special here". The backslash tells the computer: the next character is not what it normally means.

The everyday escape sequences

```
console.log("Line one\nLine two");
console.log("Name:\tMaya");
console.log("She said \"wow\" today");
console.log("A path: C:\\Users\\student");
```

The first and most important sequence is \n, which starts a new line wherever it appears. The string "Line one\nLine two" is stored as a single string, but when it is printed, the output appears on two lines. The \t sequence inserts a tab, useful for aligning short columns of text. Note that \n is one character as far as the string is concerned, even though you type two keys.

The other two sequences solve the quote problem. Since double quotes mark the ends of a string, a bare " inside would end the string early and cause a syntax error. Writing \" inserts an actual quotation mark without ending anything. Similarly, a single backslash in a string is written as two: \\, because one backslash starts an escape and the second one is the character being escaped.

Why the backslash is needed

```
// console.log("She said "wow"");   // SyntaxError!
console.log("She said \"wow\"");    // works fine
console.log("Count: \\n means new line");
```

WARNING

Inside double-quoted strings, single quotes need no escaping: "it's fine" works. But if your string uses double quotes inside and out, escaping is required. When a program reports a strange SyntaxError around a string, count your quotes and backslashes first.

Lesson Check 3.2

Choose the best answer. Answers are in the Answer Key.

1. What does console.log("A\nB"); print?

A. A on one line and B on the next
B. The single line: A\nB
C. An error
D. AB on one line

2. How do you put a real backslash inside a string?

A. \
B. \\
C. //
D. ~~

3. What does "Name:\tMaya" do when printed?

A. Prints the letter t after the colon
B. Removes the colon
C. Prints Name: Maya with a tab gap
D. Starts a new line before Maya

4. Which line prints: She said "hi" (with double quotes around hi)?

A. console.log("She said "hi"");
B. console.log('She said \'hi\');
C. console.log("She said \hi\");
D. console.log("She said \"hi\"");

Lesson 3.3 - String Methods: Measure and Change Case

Strings come with built-in functions called methods. You call a method by writing a dot after the string (or after a variable holding it), then the method name and parentheses. The first method everyone learns is .length, which reports how many characters the string contains - including spaces and punctuation.

Measuring strings

```
console.log("JavaScript".length);
let word = "hippopotamus";
console.log(word.length);
console.log("".length);
```

Case-changing methods come next. .toUpperCase() returns a copy of the string with every letter capitalized, and .toLowerCase() returns a copy with every letter turned lowercase. They are perfect for normalizing user input - for example, comparing "YES", "yes", and "Yes" fairly by converting all of them to lowercase first.

Changing case

```
let shout = "quiet please";
console.log(shout.toUpperCase());
console.log("LOUD NOISES".toLowerCase());
let answer = "YeS";
console.log(answer.toLowerCase() === "yes");
```

The .trim() method removes whitespace - spaces, tabs, and new lines - from both ends of a string, leaving the middle untouched. This is the standard first step when handling typed input, because accidental leading or trailing spaces are one of the most common reasons two strings that look equal fail a comparison.

Trimming spaces

```
let messy = "   banana   ";
console.log(messy.trim());
console.log("  a  b  ".trim());  // inner spaces stay
```

One more tool in this lesson: .indexOf() searches for text inside a string and reports the position where the first match starts. Positions count from 0, so "cat".indexOf("a") is 1. If the text is not found, indexOf returns the number -1, which programs use as a signal for "not present".

Searching with indexOf

```
let msg = "banana";
console.log(msg.indexOf("a"));
console.log(msg.indexOf("na"));
console.log(msg.indexOf("z"));
```

Lesson Check 3.3

Choose the best answer. Answers are in the Answer Key.

1. What is "open door".length?

A. 7 **B.** 8

C. 9 **D.** 10

2. What does "Mixed Case".toUpperCase() return?

A. "mixed case" **B.** "MIXED CASE"

C. "Mixed Case" **D.** undefined

3. What does " red apple ".trim() return?

A. "red apple" **B.** "redapple"

C. " red apple" **D.** "red apple "

4. What does "banana".indexOf("na") return?

A. -1 **B.** 0

C. 1 **D.** 2

Lesson 3.4 - Slicing, Replacing, and Immutability

The .slice(start, end) method cuts out a piece of a string. It copies characters from position start up to but not including position end. Positions start at 0, so "abcdef".slice(1, 3) gives "bc" - positions 1 and 2, stopping before 3. If you omit the end, slice runs to the end of the string; negative numbers count from the end, so slice(-3) grabs the last three characters.

Cutting with slice

```
let word = "abcdef";
console.log(word.slice(1, 3));
console.log(word.slice(0, 2));
console.log(word.slice(3));
console.log(word.slice(-3));
```

Two quick checkers round out the toolbox. .includes(text) answers a true/false question: does the string contain this text anywhere? And .charAt(i) returns the single character at position i. Meanwhile .replace(old, new) returns a copy of the string with the first occurrence of old swapped for new - note that it only replaces the FIRST match, not every match.

includes, charAt, and replace

```
let sentence = "I like tea much more than coffee";
console.log(sentence.includes("tea"));
console.log("hello".charAt(1));
console.log(sentence.replace("tea", "juice"));
```

Now the big rule. In JavaScript, strings are immutable: the characters inside an existing string can never be changed. Every method that seems to modify a string - toUpperCase, trim, slice, replace - actually builds and returns a brand-new string, leaving the original exactly as it was. That is why word.toUpperCase() does nothing useful on its own; you must catch the result, usually by storing it back into a variable.

Methods return NEW strings

```
let word = "cat";
word.toUpperCase();          // new string is lost!
console.log(word);           // still "cat"
word = word.toUpperCase();   // catch the result
console.log(word);           // "CAT"
```

WARNING

Calling a string method without using its result is a classic silent bug. The call runs, a shiny new string is created... and immediately thrown away. Always either store the result (word = word.toUpperCase();) or use it directly (console.log(word.toUpperCase());).

NOTE

Strings have many more methods - split, repeat, startsWith, and friends. You will meet a few later in the book; the six from this chapter cover most everyday needs.

Lesson Check 3.4

Choose the best answer. Answers are in the Answer Key.

1. What does "abcdef".slice(2, 4) return?

A. "cde" **B.** "bc"

C. "cd" **D.** "bcd"

2. What does "hello world".includes("lo w") return?

A. false **B.** "lo w"

C. 3 **D.** true

3. What does "cat".replace("a", "o") return?

A. "cat" **B.** "cot" and the original becomes "cot"

C. undefined **D.** "cot"

4. Strings in JavaScript are immutable. This means...

A. They cannot contain numbers **B.** Existing strings never change; methods return new strings

C. They cannot be used in variables **D.** They must stay under 26 characters

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. Which line prints three separate lines of output?

A. console.log("One\tTwo\tThree"); **B.** console.log("One" "Two" "Three");

C. console.log("One\nTwo\nThree"); **D.** console.log("One Two Three");

2. What does `Box: ${5 * 3}` evaluate to?

A. "Box: 15" **B.** "Box: ${5 * 3}"

C. "Box: 5 * 3" **D.** 15

3. What does "Conversation".slice(4, 7) return?

A. "ver" **B.** "vers"

C. "sa" **D.** "ers"

4. A variable holds " Paris ". Which expression gives exactly "Paris"?

A. " Paris ".length **B.** " Paris ".replace(" ", "")

C. " Paris ".trim() **D.** " Paris ".toUpperCase()

5. What does "moon".indexOf("o") return?

A. -1 **B.** 0

C. 1 **D.** 2

6. The expression "abc".toUpperCase() changes the original string to "ABC".

☐ True ☐ False

7. What does this program print? Show every line exactly.

```
let word = "book";
console.log(word.length);
console.log(word.charAt(0));
console.log(word.slice(1));
```

Output: _____

8. The programmer wanted to print HELP but the program prints help. Find the bug.

```
let shout = "help";
shout.toUpperCase();
console.log(shout);
```

Bug: _____

9. Explain in your own words what it means for strings to be immutable, and why word = word.toUpperCase(); works while word.toUpperCase(); alone does nothing useful.

_____

_____

_____

10. Write a program using a template literal that stores your name and age in variables and prints: I am NAME and I am AGE years old. (Replace NAME and AGE with the placeholder values.)

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. Which character starts and ends a template literal?

A. ` **B.** "

C. ' **D.** /

2. What does "tab\ttab" look like when printed?

A. tabttab **B.** tab tab (with a tab gap)

C. tab then a new line, then tab **D.** An error

3. What does "robots".length evaluate to?

A. 5 **B.** 6

C. "6" **D.** 7

4. What does "school".indexOf("o") return?

A. -1 **B.** 1

C. 2 **D.** 3

5. What does "abracadabra".includes("cad") return?

A. "cad" **B.** 4

C. true **D.** false

6. What does "abcdef".slice(-2) return?

A. "ef" **B.** "ab"

C. "de" **D.** "f"

7. What does "I like cake".replace("cake", "tea") return?

A. "I like cake" **B.** "I liketea"

C. true **D.** "I like tea"

8. Which statement about .trim() is TRUE?

A. It removes ALL spaces everywhere in the string **B.** It removes spaces from both ends only

C. It converts spaces to tabs **D.** It returns a number of spaces removed

9. In the source code "C:\\folder", the printed text contains exactly one backslash.

☐ True ☐ False

10. "A\nB" and "AB" have the same length.

☐ True ☐ False

11. "Sun".charAt(0) returns "S".

☐ True ☐ False

12. You want to print a two-column list: names on the left, scores on the right. Which escape sequence helps you align them, and why?

_____

_____

_____

13. Why does "It's" (with escaped quote) work inside double quotes, and when would you need to escape an apostrophe?

_____

_____

_____

14. Store the word "javascript" in a variable. Print it in ALL CAPS, print its length, and print the slice containing "script".

_____

_____

_____

15. Write a program that prints this exact output using one console.log and escape sequences:
Quote: "Learn"
Tab separated

_____

_____

_____

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. Initial machine. Store a full name like "Ada Lovelace" in one variable. Using charAt, slice, and indexOf (no hard typing), print the initials "A.L.".

Hint: The first initial is charAt(0). The second starts right after the space - and indexOf finds the space.

Challenge 2. Word counter. Without using split (not taught yet), count how many words are in the sentence "Practice makes progress every single day" by counting spaces and adding one.

Hint: Each space separates two words, so words = spaces + 1.

Challenge 3. Secret message. Take "escape sequences are fun" and print it so that each word appears on its own line, using ONE string with escape sequences - no extra variables.

Hint: Replace the spaces with \n inside one quoted string.

Challenge 4. Title fixer. The variable title holds " the HOBBIT ". Produce and print the tidy version "THE HOBBIT" - extra spaces removed and all letters capitalized. Do it in one expression.

Hint: Method calls chain left to right: clean first, then change case.

Challenge 5. Reverse thinking. The string is "stressed". Using slice and concatenation (no loops yet), build and print "desserts" - the same letters backwards.

Hint: Cut the word into small slices and glue them back in reverse order: last letter first.

Chapter Recap

- Template literals use backticks and ${...} placeholders; placeholders evaluate any expression and can span lines.
- Escape sequences: \n new line, \t tab, \" a quote, \\ one backslash.
- A string is stored as characters counted from position 0; .length reports the count including spaces.
- Methods return NEW strings - strings are immutable, so store the result to keep it.
- toUpperCase, toLowerCase, and trim each return a cleaned copy.
- indexOf reports the first position of a search text, or -1 when missing; includes answers true/false.
- slice(start, end) copies up to but not including end; negatives count from the string end.
- replace swaps only the first occurrence; charAt(i) returns one character.

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Using quotes instead of backticks for ${}

```
let name = "Ada";
console.log("Hi ${name}");
```

Fix: The ${...} placeholder only works inside backticks. With ordinary quotes it prints literally.

```
let name = "Ada";
console.log(`Hi ${name}`);
```

Mistake 2: Thinking \\n prints a backslash and a new line

```
console.log("A\\nB");  // prints A\nB literally
```

Fix: \\ is an escaped backslash, so \\n prints backslash-n as text. For a new line, drop the extra backslash.

```
console.log("A\nB");
```

Mistake 3: Calling a method and dropping the result

```
let word = "tea";
word.replace("t", "s");
```

Fix: The new string must be stored or used immediately; otherwise it vanishes.

```
let word = "tea";
word = word.replace("t", "s");
```

Mistake 4: Expecting indexOf to find every match

```
console.log("cocoa".indexOf("o"));  // 1, not all spots
```

Fix: indexOf stops at the first match. To find more, keep searching from later positions (a later-chapter trick).

```
console.log("cocoa".indexOf("o"));      // 1
console.log("cocoa".indexOf("oa"));     // later match
```

Mistake 5: Forgetting slice excludes the end position

```
let half = " ABCDEFG ".slice(0, 4);  // " ABC" not "ABCDE"
```

Fix: slice(start, end) stops BEFORE end. Add one to include the boundary character.

```
let letters = "ABCDEFG";
console.log(letters.slice(0, 5));  // ABCDE
```

Mini-Project 1: The Mad Libs Story Machine

Mad Libs is a classic word game: one player secretly collects words from friends - a name, a place, an animal - then drops them into a story and reads the hilarious result. Your program will do the same job: hold a few words in variables, then weave them into a story with template literals and print it, complete with a title line and a neatly formatted ending.

This project uses only Chapter 1-3 skills: variables, strings, template literals, escape sequences, and a few string methods. Type the whole program yourself, run it, and read the line-by-line notes below.

The complete program

```
// Mad Libs Story Machine
let hero = "Zaid";
let place = "the library";
let animal = "penguin";
let food = "pizza";
let number = 7;

let title = "THE TALE OF " + hero.toUpperCase();
console.log(title);
console.log("================");

console.log(`One quiet morning, ${hero} walked to
${place} carrying ${number} sandwiches.`);
console.log(`Suddenly, a ${animal} appeared and
said: "Those look delicious!"`);
console.log(`${hero} thought carefully, then shared
the food with the ${animal}.`);
console.log(`They ate ${food} together and became
best friends.`);

console.log("\nTHE END\nWritten by a JavaScript programmer");
```

Line-by-line explanation

Lines 2-6. Five variables hold the raw ingredients. Notice that number holds a real number - it will be converted automatically when placed in a template literal.

Lines 8. Concatenation joins the literal text THE TALE OF with the hero name converted to capitals by .toUpperCase(). The method returns a NEW string, which + glues in place.

Lines 9-10. The title prints, then a line of equal signs acts as a banner. Plain strings - nothing clever, and that is fine.

Lines 12-13. The first story sentence uses a multi-line template literal: the line break you see in the source appears in the output. The ${...} placeholders insert hero, place, and number.

Lines 14-16. Two more sentences follow the same pattern. Each template literal evaluates its placeholders and builds one final string.

Lines 18. The finale uses \n twice: one new line BEFORE THE END to separate it from the story, and one AFTER to space the credit line. Escape sequences format the page without extra statements.

How It Works

The program is a pipeline: raw words first, story second. Nothing is printed until every variable exists, which mirrors the top-to-bottom execution rule from Chapter 1 - the story statements can safely use the variables because the assignments run first.

Two string-building styles share the stage. Concatenation (+) suits short labels like the title; template literals shine for sentences with several insertions. Choosing the readable tool per job is a real-world habit, not a school rule.

The multi-line template literals are doing quiet work here. Because backtick strings keep your line breaks, the story wraps naturally instead of running off the edge - no \n needed inside those blocks.

Things to Try

- Add two new variables - a vehicle and a sound - and write a fourth story sentence that uses both.
- Make the animal name print in CAPITALS everywhere using a method call inside the placeholder.
- Change number to a calculation, such as 3 + 4, and prove to yourself that the placeholder evaluates expressions.
- Build a second story about the same characters using only escape sequences - no template literals - and compare which version was easier to write.
- Add a line that prints the length of the longest sentence you wrote, using .length.

CHAPTER 4

Numbers and Math

Computers began life as calculators, and they have never stopped being ones. Games count points, shops total bills, apps measure time and distance - all of it is arithmetic. JavaScript ships with a built-in helper called the Math object, a toolbox packed with rounding functions, limits, powers, square roots, and even random numbers.

In this chapter you will learn your way around that toolbox, discover why 0.1 + 0.2 is not exactly 0.3 (and what to do about it), generate random numbers for games, and convert text like "25" into real numbers. These are the skills behind score boards, timers, and every guess-the-number game you will build.

Lesson 4.1 - The Math Object: Rounding and Powers

The Math object is a built-in collection of number tools that is always available - no import needed. You write the word Math, a dot, and the tool you want. The most used tools are the rounding family: Math.round() rounds to the nearest whole number (halves round up), Math.floor() always rounds DOWN toward negative infinity, Math.ceil() always rounds UP, and Math.trunc() simply chops off the decimal part.

The rounding family

```
console.log(Math.round(4.5));
console.log(Math.round(4.4));
console.log(Math.floor(4.9));
console.log(Math.ceil(4.1));
console.log(Math.trunc(4.9));
```

The four functions agree on positive numbers most of the time but disagree in important edge cases. For 4.5, round gives 5 while floor gives 4. For negative numbers the difference grows: Math.floor(-4.1) is -5 (down means toward negative infinity), while Math.trunc(-4.1) is -4 (chopping the decimal moves toward zero). Choosing the right one is a real decision: ages, item counts, and page numbers usually want floor; scores displayed smoothly often want round.

Negative numbers split the family

```
console.log(Math.floor(-4.1));
console.log(Math.trunc(-4.1));
console.log(Math.round(-4.5));  // -5: half rounds up
console.log(Math.ceil(-4.9));
```

Math.abs() gives the absolute value - the number without its sign, which is the distance from zero. Math.pow(a, b) raises a to the power b (same as the ** operator), and Math.sqrt() returns the square root. These three cover most school-mathematics needs, from geometry homework to physics simulations.

Abs, power, and square root

```
console.log(Math.abs(-7));
console.log(Math.abs(7));
console.log(Math.pow(2, 10));
console.log(Math.sqrt(144));
```

TIP

When a question says "how many WHOLE boxes fit", that is Math.floor. When it says "at least how many boxes are needed", that is Math.ceil. Translating words into the right rounding function is half the job.

Lesson Check 4.1

Choose the best answer. Answers are in the Answer Key.

1. What does Math.round(7.5) return?

A. 0.75 **B.** 7

C. 7.5 **D.** 8

2. What does Math.floor(-3.2) return?

A. 3 **B.** -4

C. -3.2 **D.** -3

3. Which function simply chops off the decimal part?

A. Math.trunc **B.** Math.round

C. Math.floor **D.** Math.ceil

4. What does Math.sqrt(144) return?

A. 12 **B.** 14

C. 24 **D.** 72

Lesson 4.2 - Limits, Precision, and toFixed

Finding the biggest or smallest value in a group is a classic job for Math.min() and Math.max(). Hand them any number of arguments and they answer immediately. They pair beautifully with variables: Math.max(scoreA, scoreB) instantly names the champion.

Minimum and maximum

```
console.log(Math.min(4, 9, -2, 7));
console.log(Math.max(4, 9, -2, 7));
let a = 12;
let b = 8;
console.log(Math.max(a, b));
```

Now for a famous surprise. Try 0.1 + 0.2 in the console and you will get 0.30000000000000004. JavaScript stores numbers in binary, and fractions like one tenth cannot be written exactly in binary - the same way one third cannot be written exactly in decimal. The tiny error is normal for every mainstream language and is called floating-point imprecision. It is not a bug in your code.

The floating-point surprise

```
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 === 0.3);
console.log((0.1 + 0.2).toFixed(2));
```

The .toFixed(n) method is the everyday cure: called on a number, it returns a STRING rounded to n decimal places, perfect for displaying prices and averages. Notice that it returns a string - great for printing, but not for further math until converted back. For comparisons involving decimals, prefer comparing rounded values or using a small tolerance instead of demanding exact equality.

Displaying money cleanly

```
let total = 19.987;
console.log(total.toFixed(2));
console.log(total.toFixed(0));
console.log(typeof total.toFixed(2));
```

WARNING

Never build money logic that depends on exact decimal equality such as 0.1 + 0.2 === 0.3 - floating-point imprecision makes it false. Round for display with toFixed, or compare with a tolerance, or work in whole cents.

Lesson Check 4.2

Choose the best answer. Answers are in the Answer Key.

1. What does Math.max(3, -8, 5) return?

A. -8 **B.** 3
C. 5 **D.** 16

2. What does console.log(0.1 + 0.2 === 0.3); print?

A. true **B.** 0.3
C. NaN **D.** false

3. What does (3.14159).toFixed(2) return?

A. "3.15" **B.** 3.1
C. "3.14" as a string **D.** 3.14 as a number

4. What does Math.min(10, 2, 7) return?

A. 2 **B.** 7
C. 10 **D.** 19

Lesson 4.3 - Random Numbers for Games

Games need surprises: dice rolls, shuffles, mystery numbers. Math.random() delivers. Called with no arguments, it returns a random decimal from 0 up to but NEVER including 1 - a half-open range written math-style as [0, 1). Every call gives a fresh value, which is why you usually store it or use it immediately.

Math.random basics

```
console.log(Math.random());
console.log(Math.random());
console.log(Math.random() * 10);  // [0, 10)
```

Multiplying stretches the range: Math.random() * 6 gives decimals from 0 up to (but never reaching) 6. To turn that into a die roll you need whole numbers, so wrap it in Math.floor: Math.floor(Math.random() * 6) gives one of 0, 1, 2, 3, 4, 5. Dice start at 1, so add one at the end. This stretch-floor-shift pattern is THE standard recipe for random whole numbers in a range.

A fair six-sided die

```
let die = Math.floor(Math.random() * 6) + 1;
console.log(die);
// Random whole number in [min, max]:
let min = 3;
let max = 9;
let value = Math.floor(Math.random() * (max - min + 1))
    + min;
console.log(value);
```

Reading the general recipe piece by piece: (max - min + 1) counts how many different values you want; Math.random() * that count lands in [0, count); floor makes it whole; adding min shifts the range to start at min. Memorize the pattern once and you can roll dice, pick cards, or choose a random student - all with one line.

NOTE

Math.random() is not truly random - it is a fast mathematical sequence that looks random. For games and practice exercises that is exactly what you want. Security-grade randomness uses other tools you will meet much later in your programming life.

Lesson Check 4.3

Choose the best answer. Answers are in the Answer Key.

1. What is the range of Math.random()?

A. From 1 to 10
B. From 0 up to (but never including) 1
C. Any whole number
D. From -1 to 1

2. What does Math.floor(Math.random() * 6) return?

A. Always 0
B. A number from 0 to 5
C. A number from 1 to 6
D. A number from 0 to 6

3. How do you make a fair die roll from 1 to 6?

A. Math.floor(Math.random() * 6) + 1
B. Math.random(6)
C. Math.round(Math.random() * 6)
D. Math.floor(Math.random()) + 6

4. In the recipe Math.floor(Math.random() * (max - min + 1)) + min, what does (max - min + 1) count?

A. The smallest possible result
B. How many times to call random
C. How many different values the roll can be
D. The largest possible result

Lesson 4.4 - Converting Text to Numbers (and NaN)

Input usually arrives as text. The value from a form, a file, or a prompt looks like "25" - a string of two characters, not the number 25. Adding it to something would concatenate instead of calculate. JavaScript offers conversion tools: Number("25") converts text to a number when the whole text is numeric, parseInt("25 apples") reads the leading whole number and ignores trailing text, and parseFloat("3.5 km") keeps the decimals.

Three ways to convert

```
console.log(Number("42") + 1);
console.log(parseInt("18 years"));
console.log(parseFloat("2.75 meters"));
console.log(typeof Number("42"));
```

What if the text is not a number at all? Number("hello") returns a special value called NaN - "Not a Number". NaN is contagious: any arithmetic with it produces NaN again. It is even stranger than it looks - NaN does not equal anything, including itself! The reliable way to detect it is the built-in Number.isNaN() function.

NaN: the number that is not a number

```
console.log(Number("hello"));
console.log(Number("hello") + 5);
console.log(Number.isNaN(Number("hello")));
console.log(parseInt("apples 5"));  // NaN
```

A practical habit from real programs: convert first, check with Number.isNaN, then calculate. If the check fails, show a friendly message instead of letting NaN creep into your totals. You will use this pattern in the final project, where file data arrives as text and must be verified before it becomes scores.

The convert-check-calculate pattern

```
let raw = "abc";
let value = Number(raw);
if (Number.isNaN(value)) {
    console.log("Please type a number!");
} else {
    console.log(value * 2);
}
```

TIP

parseInt stops at the first non-digit, so parseInt("12abc") is 12 - but parseInt("abc12") is NaN because nothing numeric comes first. Leading spaces are fine; leading junk is not.

Lesson Check 4.4

Choose the best answer. Answers are in the Answer Key.

1. What does Number("42") return?

A. "42" as a string

B. NaN

C. undefined

D. The number 42

2. What does parseInt("18 years old") return?

A. NaN **B.** "18 years old"

C. 0 **D.** 18

3. What does Number("hello") return?

A. An error stops the program **B.** NaN

C. 0 **D.** "hello"

4. Which expression reliably detects NaN?

A. value == "NaN" **B.** typeof value == "nan"

C. Number.isNaN(value) **D.** value === NaN

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. A shop packs 17 muffins into boxes of 6. Which expression gives the number of FULL boxes?

A. Math.floor(17 / 6) **B.** 17 % 6

C. Math.ceil(17 / 6) **D.** Math.round(17 / 6)

2. What does Math.abs(-12) return?

A. -12 **B.** 0

C. 12 **D.** NaN

3. What does Math.ceil(2.01) return?

A. 2 **B.** 2.01

C. 2.1 **D.** 3

4. Which expression rolls a random whole number from 1 to 20?

A. Math.random() * 20 **B.** Math.floor(Math.random() * 20)

C. Math.floor(Math.random() * 20) + 1 **D.** Math.ceil(Math.random()) * 20

5. What does typeof NaN evaluate to?

A. "nan" **B.** "undefined"

C. "notanumber" **D.** "number"

6. Math.floor(9.99) and Math.trunc(9.99) both return 9.

☐ True ☐ False

7. What does this program print?

```
console.log(Math.max(3, 9, 4));
console.log(Math.min(-1, -5, 2));
console.log(Math.pow(3, 2));
```

Output: _____

8. This die roll never seems to roll a 6. Find the bug.

```
let die = Math.floor(Math.random() * 6);
console.log(die);
```

Bug: _____

9. Explain why 0.1 + 0.2 === 0.3 is false in JavaScript, and name one safe way to compare or display such values.

_____

_____

_____

10. Write a program that stores 23.678 in a variable, then prints it rounded to 2 decimal places, its square root rounded to 1 decimal place, and the absolute value of -23.678.

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. What does Math.round(3.49) return?

A. 0 **B.** 3

C. 3.5 **D.** 4

2. What does Math.ceil(-0.5) return?

A. -1 **B.** -0.5

C. 0 **D.** 1

3. What does Math.min(4, 4.5, -0.2) return?

A. -0.2 **B.** 0

C. 4 **D.** 4.5

4. What does console.log(Math.pow(5, 2)); print?

A. 7 **B.** 10

C. 25 **D.** 52

5. Which statement about Math.random() is TRUE?

A. It returns the same value every time **B.** It returns a decimal from 0 up to but never including 1

C. It returns whole numbers from 1 to 100 **D.** It can return exactly 1

6. What does Number("3.5 apples") return?

A. NaN **B.** 3.5

C. "3.5 apples" **D.** 0

7. What does console.log((9.876).toFixed(1)); print?

A. "9.8" **B.** 9.9 as a number

C. "10.0" **D.** "9.9"

8. Why is Math.floor preferred over Math.round for the random-range recipe?

A. round makes the end values half as likely, biasing the roll **B.** floor is faster to type

C. floor works with negative ranges only **D.** round cannot be used with Math.random

9. Math.floor(-2.5) returns -2.

☐ True ☐ False

10. Number.isNaN(Number("12abc")) is true.

☐ True ☐ False

11. Math.sqrt(0) returns 0.

☐ True ☐ False

12. A teacher wants a random student number from 1 to 30. Write the expression and explain each part in one sentence.

_____

_____

_____

13. Why should a shopping program avoid checking total === 19.99 after adding decimals? What should it do instead?

14. Write a program that simulates rolling two dice and prints both rolls and their total.

15. Store the text "97" in a variable, convert it to a number, add 3, and print 100. Then try converting "abc" and print whether it is NaN.

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. Electricity bill. A meter reads 4521.7 kWh now and 4370.2 kWh last month. Print the units used, rounded to one decimal place, and the cost at 1.25 per unit, rounded to two decimals.

Hint: Subtract first, then use toFixed for each displayed value. Order of operations matters.

Challenge 2. Time splitter. Given 5000 seconds, print hours, minutes, and seconds separately (1 hour, 23 minutes, 20 seconds).

Hint: Divide by 3600 with floor for hours; the remainder tricks from Chapter 2 work here too.

Challenge 3. Fair coin statistics. Simulate 10 coin flips where 0 is heads and 1 is tails (use Math.floor(Math.random() * 2)). Print each flip on its own line with a label.

Hint: Call the recipe ten times, storing or printing each result. Label them Flip 1, Flip 2, ...

Challenge 4. Rounding detective. Predict the output of these five lines BEFORE running, then verify:

Hint: Track each function's direction: nearest, down, up, chop, and absolute value.

Challenge 5. Range checker. Write a program that stores a test score value of 87.654, converts a raw string "87.654" with parseFloat, and prints whether the two values are within 0.001 of each other using Math.abs.

Hint: Exact equality is fragile with decimals; compare the abs difference to a tiny tolerance.

Chapter Recap

- Math.round rounds to nearest (halves up); Math.floor always down; Math.ceil always up; Math.trunc chops toward zero.
- Negative numbers split the family: floor(-4.1) is -5 but trunc(-4.1) is -4.
- Math.abs removes the sign; Math.pow(a, b) is a to the power b; Math.sqrt gives square roots.
- Math.min and Math.max compare any list of numbers.
- Binary floating point stores some decimals imprecisely: 0.1 + 0.2 is 0.30000000000000004.
- toFixed(n) rounds for DISPLAY and returns a string.
- Math.random() gives [0, 1); multiply to stretch, floor to make whole, add to shift: floor(random() * size) + min.
- Number() converts whole text; parseInt/parseFloat read leading numbers; failures give NaN.
- NaN equals nothing, even itself - detect it with Number.isNaN().

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Expecting exact decimal equality

```
if (0.1 + 0.2 === 0.3) { /* never runs */ }
```

Fix: Floating-point imprecision makes the sum slightly off. Compare with a tolerance or round first.

```
let diff = Math.abs((0.1 + 0.2) - 0.3);
console.log(diff < 0.000001);  // true
```

Mistake 2: Forgetting the +1 in a die roll

```
let die = Math.floor(Math.random() * 6);  // 0 to 5
```

Fix: The recipe needs the shift: without +1 the roll starts at 0 and never reaches 6.

```
let die = Math.floor(Math.random() * 6) + 1;
```

Mistake 3: Using round instead of floor in random ranges

```
let r = Math.round(Math.random() * 10);  // biased
```

Fix: round gives the end values half-width slots. Use floor plus shift for a fair range.

```
let r = Math.floor(Math.random() * 10) + 1;
```

Mistake 4: Trusting parseInt with leading junk

```
let age = parseInt("age: 12");  // NaN
```

Fix: parseInt reads from the START; junk first means NaN. Clean the text or use a numeric-first format.

```
let age = parseInt("12 years");  // 12
```

Mistake 5: Displaying raw floating-point totals

```
console.log("Total: " + 0.1 + 0.2);  // Total: 0.10.2
```

Fix: Parenthesize the math and round for display.

```
console.log("Total: " + (0.1 + 0.2).toFixed(2));
```

CHAPTER 5

Making Decisions

So far every program has run the same way every time, from top to bottom. Real programs are smarter than that: a game asks "is the score high enough to win?", a login page asks "do the passwords match?", a thermometer asks "is it freezing?". This chapter gives your programs the power to choose different paths based on the values they hold.

You will learn comparisons that produce true and false, the if/else family that branches your code, the logical operators that combine decisions, and the switch statement for choosing among many fixed options. Branching is where programs start to feel alive.

Lesson 5.1 - Comparisons Produce Booleans

A comparison asks a question about two values and answers with a boolean: true or false. JavaScript has six comparison operators. The star of the book is ===, which asks "are these exactly equal?". Its partner !== asks "are these different?". Then come the four inequality operators you know from mathematics: >, <, >=, and <=.

Asking questions with comparisons

```
console.log(5 === 5);
console.log(5 !== 4);
console.log(10 > 3);
console.log(2 >= 2);
console.log("cat" === "dog");
```

Why === and not =? The single = assigns a value, while === compares values without changing anything. This book uses === everywhere and recommends you do the same. JavaScript also has a loose == comparison that converts types before comparing - so "5" == 5 is true - but that surprising conversion causes subtle bugs. The strict === never converts: "5" === 5 is false because text is not a number.

Strict vs loose (know, then avoid ==)

```
console.log("5" === 5);   // false: different types
console.log("5" == 5);    // true: loose converts!
// Rule for this book: always use === and !==
```

Comparisons chain naturally with variables: score > 100, age <= 12, name === "Maya". The result is an ordinary boolean that can be stored in a variable and printed. You can even compare strings: JavaScript compares them alphabetically, so "apple" < "banana" is true - though capital letters sort before lowercase ones, a detail to save for later.

TIP

Read === out loud as "is exactly equal to". Saying the words while coding sounds silly and works brilliantly - it stops the = versus === mistake before it happens.

Lesson Check 5.1

Choose the best answer. Answers are in the Answer Key.

1. What does the expression 7 > 3 evaluate to?

A. 7 **B.** true

C. "yes" **D.** 3

2. Which operator asks "are these two values exactly equal"?

A. = **B.** ==

C. === **D.** =>

3. What does "5" === 5 evaluate to?

A. true **B.** false

C. 5 **D.** An error

4. What does 4 <= 4 evaluate to?

A. 4 **B.** undefined

C. true, because <= allows equality **D.** false, because 4 is not less than 4

Lesson 5.2 - if, else, and else if

The if statement runs a block of code only when its condition is true. The condition goes in parentheses, and the code to run goes inside curly braces. If the condition is false, the block is skipped entirely and the program continues after it. This is the first time your programs can do different things on different runs.

The if statement

```
let temperature = 38;
if (temperature > 35) {
    console.log("It is very hot today!");
}
console.log("Done checking.");
```

The else clause adds a second path: when the condition is false, the else block runs instead. Exactly one of the two blocks always runs, never both and never neither. Together they cover every possibility, like a fork in a road where the program must choose one way.

if and else together

```
let age = 11;
if (age >= 12) {
    console.log("You can join the club.");
} else {
    console.log("Come back next year!");
}
```

For more than two paths, chain else if blocks between if and else. JavaScript checks each condition from top to bottom and runs the FIRST block whose condition is true, skipping the rest. The final else catches everything no condition matched. Order matters: put specific checks before general ones, or the general condition will swallow

the cases you meant to catch later.

A chain of branches

```
let score = 78;
if (score >= 90) {
    console.log("Grade: A");
} else if (score >= 75) {
    console.log("Grade: B");
} else if (score >= 60) {
    console.log("Grade: C");
} else {
    console.log("Try again");
}
```

WARNING

The condition parentheses need no semicolon, and the curly braces matter: if (x > 5); with a stray semicolon does nothing, and forgetting the braces makes only the next single line conditional - a classic source of confusing bugs. Always use braces, even for one line.

Lesson Check 5.2

Choose the best answer. Answers are in the Answer Key.

1. When does an else block run?

A. When the if condition is false

B. When the if condition is true

C. Always, right after the if block

D. Only when the if block printed something

2. In an if / else if / else chain, how many blocks run?

A. All whose conditions are true

B. Always the last one

C. At least two

D. Exactly one - the first whose condition is true

3. What does this print?

```
let n = 15;
if (n > 10) {
console.log("big");
} else {
console.log("small");
}
```

A. big

B. small

C. big then small

D. Nothing

4. Why should the most specific conditions come first in an else-if chain?

A. Because JavaScript requires alphabetical order

B. It makes the program run faster

C. Because a general condition earlier would swallow the cases meant for later branches

D. It looks nicer

Lesson 5.3 - Logical Operators: &&, ||, !

Decisions often depend on several facts at once: "you can ride if you are at least 10 AND taller than 120 cm". JavaScript has three logical operators that combine booleans. The AND operator && is true only when BOTH sides are true. The OR operator || is true when AT LEAST one side is true. The NOT operator ! flips a boolean: !true is false and !false is true.

The three logical operators

```
let age = 11;
let height = 125;
console.log(age >= 10 && height > 120);
console.log(age >= 15 || height > 120);
console.log(!(age >= 10));
```

These operators short-circuit: JavaScript stops evaluating as soon as the answer is certain. With &&, if the left side is false, the whole thing must be false, so the right side is never checked. With ||, if the left side is true, the right side is skipped. Short-circuit evaluation is a performance feature AND a safety tool: you can write x !== 0 && 10 / x > 2 and trust that the division never runs on zero.

Short-circuit evaluation in action

```
let x = 0;
console.log(x !== 0 && 10 / x > 2);
// safe: the division never runs because x !== 0 is
// false and && stops there
```

Keep two ideas distinct, because they look similar but do different jobs. Short-circuit evaluation is about SKIPPING work once an answer is known - an evaluation rule inside a single expression. First-match branching is about choosing one PATH - the if / else if chain from Lesson 5.2 that runs the first true block and skips the rest. One decides whether parts of an expression run; the other decides which block of code runs. You will use both constantly, sometimes in the same line.

Both ideas in one program

```
let hour = 14;
// First-match branching: exactly one block runs
if (hour < 12) {
    console.log("Morning");
} else if (hour < 18) {
    console.log("Afternoon");
} else {
    console.log("Evening");
}
// Short-circuit: right side runs only if needed
let isOpen = hour >= 9 && hour <= 17;
```

NOTE

Experienced programmers sometimes use the VALUES that || and && return, not just their booleans - a trick for default values. It is elegant but confusing for beginners, so this book uses them only for true/false logic. The ?? operator waits for you in later studies.

Lesson Check 5.3

Choose the best answer. Answers are in the Answer Key.

1. What does true && false evaluate to?

A. true
B. false
C. undefined
D. Both values

2. What does !(5 > 2) evaluate to?

A. true
B. 5
C. An error
D. false

3. What does short-circuit evaluation mean for the expression false && checkSomething()?

A. checkSomething() runs twice
B. Both sides always run
C. The expression is an error
D. checkSomething() never runs, because the answer is already false

4. Which sentence best distinguishes short-circuiting from first-match branching?

A. They are two names for the same idea
B. Short-circuiting skips parts of an expression; first-match branching picks one code path in an if / else if chain
C. Branching skips parts of an expression; short-circuiting picks a code path
D. Short-circuiting only works inside switch

Lesson 5.4 - switch: Choosing Among Many Options

When one value must be compared against a list of fixed possibilities - a menu choice, a day number, a command word - the switch statement is tidier than a long else-if chain. switch (value) jumps to the case whose label matches, using strict equality (===), and runs the statements under that label.

A simple switch

```
let dayNumber = 6;
switch (dayNumber) {
    case 5:
        console.log("Friday");
        break;
    case 6:
        console.log("Saturday");
        break;
    case 7:
        console.log("Sunday");
        break;
    default:
        console.log("A weekday");
}
```

The break statement is essential. Without it, execution falls through into the next case and keeps running its code too - which is almost never what you want. The default label plays the role of the final else: it runs when nothing matched. Forgetting break produces the most classic switch bug in programming, so much so that some teams ban

switch entirely.

The fall-through surprise (no break!)

```
let menu = 2;
switch (menu) {
    case 1:
        console.log("Pizza");
    case 2:
        console.log("Pasta");   // runs, then falls
    case 3:
        console.log("Salad");   // through here too!
        break;
}
// Prints Pasta AND Salad
```

Fall-through is not always a mistake: grouping two case labels with an intentionally empty body is a neat way to share code. But as a beginner, add a break at the end of every case, including the last, and treat grouped cases as a deliberate, commented exception.

Grouping cases on purpose

```
let fruit = "apple";
switch (fruit) {
    case "apple":
    case "pear":
        console.log("Pome fruit");
        break;
    case "banana":
        console.log("Tropical fruit");
        break;
    default:
        console.log("Unknown fruit");
}
```

TIP

Choosing between switch and else if: switch shines for ONE value compared against MANY fixed labels; else if handles ranges and different variables in each check. If your conditions are like score >= 90, switch is the wrong tool - it cannot test ranges.

Lesson Check 5.4

Choose the best answer. Answers are in the Answer Key.

1. What does a switch case match using?

A. Strict === comparison
B. Loose == comparison
C. Greater-than checks
D. Alphabetical order

2. What happens if you forget break at the end of a case?

A. The switch restarts from the top
B. The default case runs
C. Execution falls through into the next case
D. An error stops the program

3. What is the role of default in a switch?

A. It runs first, before any case
B. It replaces the need for break
C. It is required in every switch
D. It runs when no case label matched

4. For which task is switch the BEST tool?

A. Checking whether score is at least 90
B. Comparing a menu number against choices 1, 2, 3, and 4
C. Testing two different variables at once
D. Repeating code ten times

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. Which expression correctly tests whether age is between 13 and 19 (inclusive)?

A. age >= 13 || age <= 19
B. age === 13 && age === 19
C. age >= 13 && age <= 19
D. 13 <= age <= 19

2. What does this program print?

```
let x = 9;
if (x % 2 === 0) {
console.log("even");
} else {
console.log("odd");
}
```

A. an error
B. even
C. odd
D. nothing

3. What does console.log(false || 3 > 2); print?

A. false
B. 3
C. undefined
D. true

4. A program must print a badge color for a level number from 1 to 5, with a fallback for anything else. Which structure fits best?

A. A chain of five separate if statements
B. A switch with five cases and a default
C. One if with the || operator
D. Math.max on the level

5. What does the expression !true || false evaluate to?

A. false
B. true
C. true then false
D. An error

6. The condition if (score = 100) correctly checks whether score equals 100.

☐ True ☐ False

7. What does this program print when password = "open123"?

```
if (password === "open123") {
  console.log("Welcome");
} else {
  console.log("Denied");
}
```

Output: _____

8. The weather app prints the wrong message for any temperature. Find the bug.

```
let temp = 31;
if (temp > 30) {
  console.log("Cold day");
} else {
  console.log("Hot day");
}
```

Bug: _____

9. Explain in your own words the difference between =, ==, and ===, and say which one this book uses for comparisons and why.

_____

_____

_____

10. Write a program that stores a number in a variable and prints "positive", "negative", or "zero" using an if / else if / else chain.

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. What does 10 !== 10 evaluate to?

A. 10 **B.** NaN

C. false **D.** true

2. What does console.log("b" > "a"); print?

A. An error **B.** "b"

C. true **D.** false

3. In the grade chain (>=90 A, >=75 B, >=60 C), what does score = 75 print?

A. Grade: A
B. Grade: B
C. Grade: C
D. Nothing

4. What does true && !false evaluate to?

A. false
B. undefined
C. An error
D. true

5. What does the expression 5 > 3 && 2 > 7 evaluate to?

A. false
B. true
C. 5
D. 2

6. Which value of dayNumber prints "Weekend" in this switch?

```
switch (dayNumber) {
case 6:
case 7:
console.log("Weekend"); break;
default: console.log("Weekday");
}
```

A. 6 or 7
B. Only 6
C. Only 7
D. Any number

7. What does this print?

```
let n = 0;
if (n) {
console.log("truthy");
} else {
console.log("falsy");
}
```

A. 0
B. An error
C. falsy
D. truthy

8. Which statement about switch fall-through is TRUE?

A. Fall-through always causes a SyntaxError
B. Fall-through skips the default case automatically
C. Fall-through happens only with string cases
D. Fall-through can be used on purpose to share code between labels

9. The else-if chain runs every block whose condition is true.

☐ True ☐ False

10. In JavaScript, the number 0 and the empty string "" are both falsy.

☐ True ☐ False

11. switch can test ranges such as score >= 90 in its case labels.

☐ True ☐ False

12. A ride allows riders who are at least 12 years old OR at least 150 cm tall. Write the boolean expression for "can ride" with variables age and height.

13. Why does this book say "always use braces" with if statements? Describe the bug that braces prevent.

14. Write a program that stores a password and prints "strong" if its length is at least 8 AND it includes the character "-", otherwise "weak".

15. Write a switch that stores the text "tue" and prints the full day name for "sat", "sun", and "tue", with default "unknown day".

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. Leap year checker. A year is a leap year if it is divisible by 4, EXCEPT centuries, which must be divisible by 400. Write a program that prints whether 2024, 1900, and 2000 are leap years.

Hint: Combine two conditions: (year % 4 === 0 AND year % 100 !== 0) OR year % 400 === 0.

Challenge 2. BMI advisor. BMI = weight / (height * height) with height in meters. For weight 70 and height 1.75, print the BMI rounded to 1 decimal and a message: below 18.5 "underweight", up to 24.9 "healthy", up to 29.9 "overweight", otherwise "obese".

Hint: Compute once, chain else if from the smallest upward, and use toFixed for the display.

Challenge 3. Short-circuit guard. Without using an if statement, write a single expression that prints true only when x is not zero AND 100 divided by x is greater than 4. Test it with x = 20 and x = 0.

Hint: && short-circuits: putting the zero check on the left protects the division.

Challenge 4. Menu machine. Build a switch for a canteen: codes "A", "B", "C" map to Sandwich 3.5, Soup 2.0, Juice 1.5. Store a code and a quantity, and print the total cost like "Total: 7". Unknown codes print "Not on the menu".

Hint: Each case sets a price variable; compute quantity * price after the switch.

Challenge 5. Range router. Without switch, write an if / else if chain that classifies a number: below 0 "freezing", 0-15 "cold", 16-25 "mild", 26-35 "hot", above 35 "extreme". Test it on -3, 20, and 40.

Hint: Order the checks from low to high so each branch captures its band exactly.

Chapter Recap

- Comparisons (===, !==, >, <, >=, <=) produce booleans; the strict === never converts types.
- Use === and !== everywhere; avoid loose == and its surprises like "5" == 5.
- if runs a block on true; else provides the false path; else if chains pick the FIRST true branch.
- && needs both sides true, || needs one, ! flips a boolean.
- Short-circuit evaluation skips the right side of && or || when the left side already decides - distinct from first-match branching, which picks one code path.
- switch matches one value against fixed labels with ===; break prevents fall-through; default handles no-match.
- Grouped empty cases can share a body on purpose.
- The values 0 and "" are falsy when used as conditions.

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Assigning instead of comparing

```
if (score = 100) {
  console.log("Perfect");
}
```

Fix: One = stores 100 into score and the condition becomes always-true. Comparisons need ===.

```
if (score === 100) {
  console.log("Perfect");
}
```

Mistake 2: Testing a range with chained math notation

```
if (13 <= age <= 19) { /* misleading */ }
```

Fix: JavaScript reads this as (13 <= age) <= 19 - nonsense. Join two comparisons with &&.

```
if (age >= 13 && age <= 19) {
  console.log("teen");
}
```

Mistake 3: Forgetting break in a switch

```
case 1:
  console.log("One");   // falls through!
case 2:
  console.log("Two");
  break;
```

Fix: Without break, case 1 also prints "Two". End every case with break unless grouping on purpose.

```
switch (menu) {
  case 1:
    console.log("One");
    break;
  case 2:
    console.log("Two");
    break;
}
```

Mistake 4: Using || when both rules are required

```
if (age >= 13 || hasTicket) {
  console.log("Enter");
}
```

Fix: OR lets either rule qualify. When BOTH must hold, use &&.

```
if (age >= 13 && hasTicket) {
  console.log("Enter");
}
```

Mistake 5: Putting the general condition first

```
if (score >= 60) {
  console.log("C");
} else if (score >= 90) {
  console.log("A");  // unreachable
}
```

Fix: First-match branching stops at the broad check. Order branches from most specific to most general.

```
if (score >= 90) {
  console.log("A");
} else if (score >= 60) {
  console.log("C");
}
```

CHAPTER 6

Loops

Computers never get bored. Ask one to print "Hello" a thousand times and it happily complies - because of loops. A loop repeats a block of code as many times as you ask, and it is the difference between writing a thousand statements and writing five.

In this chapter you will meet the while loop, which repeats as long as a condition holds, and the for loop, which is the champion of counted repetition. You will learn to break out of a loop early, skip single turns with continue, and nest loops inside each other for grids and patterns. You will also meet their most famous bug: the infinite loop - and how to escape it.

Lesson 6.1 - The while Loop

A while loop repeats a block of code for as long as its condition stays true. First JavaScript checks the condition; if it is true, the body runs once and control returns to the top for another check. The moment the condition is false, the loop ends and the program moves on. Something inside the body must eventually make the condition false - that something is called the update.

Counting down with while

```
let energy = 3;
while (energy > 0) {
    console.log("Energy: " + energy);
    energy = energy - 1;
}
console.log("Out of energy!");
```

Walk through the example: energy starts at 3, the condition 3 > 0 is true, so the body prints and lowers energy to 2, then 1, then 0. When energy reaches 0 the condition 0 > 0 is false and the loop ends. Every while loop needs the same three parts: something the condition tests (the counter), the condition itself, and the update that moves the counter toward the exit.

WARNING

If the update is missing, the condition never changes and the loop runs forever - the famous infinite loop. energy = energy - 1; is the line that saves the program above. If a browser console freezes, a runaway loop is usually the reason, so double-check the update line first.

TIP

Before running a while loop, ask: "Which line moves this loop toward its end?" If you cannot point to it, the loop is probably infinite. Point first, run second.

Lesson Check 6.1

Choose the best answer. Answers are in the Answer Key.

1. What are the three essential parts of every while loop?

A. Start value, condition, update
B. Header, footer, margin
C. A function, an array, a string
D. if, else, break

2. What happens if the update line is missing from a while loop?

A. JavaScript inserts the update for you
B. The loop never ends - an infinite loop
C. The loop runs once
D. The condition becomes false automatically

3. When is the while condition checked?

A. Only once at the start
B. Never - while loops run blind
C. Before every round, and once before the first
D. After every round only

4. What does this loop print?

```
let n = 3;
while (n > 1) {
console.log(n);
n = n - 1;
}
```

A. 3 and 2
B. 3, 2, and 1
C. Only 3
D. Nothing

Lesson 6.2 - The for Loop

Most loops count, so JavaScript packs the three loop parts into one tidy line: the for loop. Inside its parentheses live three clauses separated by semicolons: the start (runs once before anything), the condition (checked before every round), and the update (runs after every round). The body braces follow.

Counting up with for

```
for (let i = 1; i <= 5; i = i + 1) {
    console.log("Round " + i);
}
console.log("Match over!");
```

Read the header like a sentence: "let i start at 1; keep going while i is at most 5; after each round add 1 to i". The name i is tradition - short for index - and it exists only inside the loop. The update i = i + 1 is so common that JavaScript has a shortcut: i++ adds one, and i-- subtracts one. You will see both styles in the wild; this book uses i++ in for loops from here on.

The i++ shortcut and flexible steps

```
for (let i = 0; i < 3; i++) {
    console.log(i);
}
for (let step = 10; step >= 0; step -= 2) {
    console.log(step);
}
```

Choosing the condition boundary is where off-by-one bugs hide. i <= 5 runs five rounds when starting at 1, but i < 5 starting at 0 also runs five rounds - programmers count 0, 1, 2, 3, 4. A reliable habit: decide how many rounds you want, then pick the start and boundary that produce exactly that count. Test with a tiny number first.

NOTE

while and for can always replace each other: a for loop is just a while loop with its parts moved into the header. Use for when the number of rounds is known in advance, and while when the loop must watch a condition it cannot predict - like waiting for a player to type the right answer.

Lesson Check 6.2

Choose the best answer. Answers are in the Answer Key.

1. How many times does for (let i = 0; i < 4; i++) run its body?

A. 3 **B.** 5

C. Infinite **D.** 4

2. In a for header, what does the middle clause do?

A. Names the loop variable **B.** Is checked before every round

C. Runs once before the loop **D.** Runs after every round

3. What does i++ mean?

A. Add 1 to i **B.** Add 2 to i

C. Multiply i by itself **D.** Compare i to 1

4. What does this loop print?
for (let k = 2; k <= 6; k + = 2) { console.log(k); } (spaces removed)

A. 6 **B.** 2 2 2 2 forever

C. 2 4 6 **D.** 2 3 4 5 6

Lesson 6.3 - break and continue

Sometimes a loop should stop early. The break statement leaves the loop immediately - no more rounds, no matter what the condition says. It is how programs "search until found": once the answer appears, checking more rounds is wasted work.

Stopping at the target

```
for (let i = 1; i <= 10; i++) {
    if (i === 7) {
        console.log("Found it!");
        break;
    }
    console.log("Checking " + i);
}
```

The continue statement is break's gentler cousin: it skips only the REST of the current round and jumps back to the top for the next round. Use it to filter - process the interesting cases and hop over the rest without leaving the loop.

Skipping with continue

```
for (let i = 1; i <= 5; i++) {
    if (i % 2 === 0) {
        continue;  // skip even numbers
    }
    console.log("Odd round " + i);
}
```

Choosing the right loop is now a three-way decision. Counting a known number of rounds: for. Watching a condition you cannot predict: while. Searching for something with a guaranteed exit: a for or while loop with break inside. Notice that break and continue belong to all of them - they are loop tools, not for-loop tools.

TIP

A classic pattern: an endless while (true) loop that relies on break for its exit. It reads beautifully - "keep going until something inside says stop" - but every such loop MUST contain a break, or it never ends.

Lesson Check 6.3

Choose the best answer. Answers are in the Answer Key.

1. What does break do inside a loop?

A. Skips to the next round

B. Restarts the loop from round one

C. Pauses the loop for one second

D. Ends the loop immediately

2. What does continue do inside a loop?

A. Repeats the current round forever

B. Skips the rest of the current round only

C. Ends the loop

D. Restarts the computer

3. What does this print?

```
for (let i = 1; i <= 5; i++) {
if (i === 3) { break; }
console.log(i);
}
```

A. 1 2

B. 1 2 3

C. 1 2 4 5

D. 3

4. An endless while (true) loop is safe to use only if...

A. The body is short

B. The computer is fast

C. It uses for-style counters

D. It contains a break that will eventually run

Lesson 6.4 - Nested Loops and Patterns

A loop inside another loop is called a nested loop, and it is the standard shape for grids, tables, and patterns. The inner loop runs completely for every single round of the outer loop. If the outer loop runs 3 times and the inner runs 4 times, the inner body executes 3 times 4 - twelve rounds in total.

A 3 by 4 grid

```
for (let row = 1; row <= 3; row++) {
    let line = "";
    for (let col = 1; col <= 4; col++) {
        line = line + "*";
    }
    console.log("Row " + row + ": " + line);
}
```

Trace the flow carefully: row 1 starts, the inner loop builds a line of four stars, the line prints, then row 2 starts and rebuilds the inner loop from scratch. The inner loop's variable col is created and destroyed on every outer round. A fresh string variable line is rebuilt each round too - forgetting to reset it is the classic nested-loop bug that makes rows grow longer and longer.

A right-triangle pattern

```
for (let row = 1; row <= 5; row++) {
    let line = "";
    for (let col = 1; col <= row; col++) {
        line = line + "*";
    }
    console.log(line);
}
```

Nested loops multiply work quickly: 10 by 10 is 100 rounds, but 1000 by 1000 is a million. That is still fast for a computer, yet the habit of estimating rounds serves you well: when a program suddenly crawls, look for nesting first. On the fun side, nesting is how multiplication tables, game boards, and pixel images get built - one loop for rows, one for columns.

NOTE

You can nest while inside for or any other mix, but each loop needs its OWN counter variable. Sharing i between an outer and inner loop scrambles both counts and produces delightfully wrong output.

Lesson Check 6.4

Choose the best answer. Answers are in the Answer Key.

1. An outer loop runs 4 times; its inner loop runs 5 times each round. How many inner-body executions?

A. 9 **B.** 20

C. 45 **D.** 54

2. In the triangle pattern lesson, why is line = "" reset inside the outer loop?

A. Because JavaScript requires it after console.log **B.** It is not reset - it keeps growing

C. Because each row needs a fresh, empty line to build on **D.** To make the program faster

3. What is wrong with sharing variable i between an outer and an inner for loop?

A. The inner loop cannot see i **B.** It causes a SyntaxError

C. Both loops scramble the same counter and the counts go wrong **D.** Nothing - it saves memory

4. What does the 3x4 grid example print on Row 2?

A. Row 2: ****** **B.** Row 2: ****

C. Row 2: * **D.** Row 2: **

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. Which loop is the best fit for printing every number from 1 to 100?

A. A for loop with counter i from 1 to 100 **B.** A while loop with a random condition

C. Ten nested if statements **D.** A switch with 100 cases

2. How many rounds does for (let i = 3; i <= 9; i += 3) run?

A. 2 **B.** 3

C. 4 **D.** 9

3. What does this loop print?

```
let total = 0;
for (let i = 1; i <= 4; i++) {
total = total + i;
}
console.log(total);
```

A. 4 **B.** 6

C. 10 **D.** 24

4. A loop must keep asking until the user types "stop". Which loop fits best?

A. for with 10 rounds
B. break without a loop
C. switch on the input length
D. while checking the input

5. What does this nested loop print in total?

```
for (let r = 1; r <= 2; r++) {
for (let c = 1; c <= 3; c++) {
console.log(r + "-" + c);
}
}
```

A. 5 lines
B. 2 lines
C. 3 lines
D. 6 lines like 1-1, 1-2, ... 2-3

6. The continue statement ends the whole loop immediately.

☐ True ☐ False

7. What does this program print? List every line.

```
for (let i = 2; i <= 8; i += 2) {
    console.log(i);
}
console.log("Done");
```

Output: _____

8. This countdown should print 3, 2, 1 but prints 3 forever. Find the bug.

```
let n = 3;
while (n > 0) {
    console.log(n);
}
```

Bug: _____

9. Explain the difference between break and continue using a searching-a-list example: when would each be the right tool?

_____

_____

_____

10. Write a for loop that prints the 3-times table: "3 x 1 = 3" up to "3 x 5 = 15".

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. Which line correctly starts a loop that runs exactly 8 times?

A. for (let i = 1; i <= 8; i++)

B. for (let i = 0; i <= 8; i++)

C. for (let i = 1; i < 8; i++)

D. for (let i = 8; i >= 1; i++)

2. What does this print?

```
let x = 10;
while (x > 5) {
console.log(x);
x = x - 3;
}
```

A. 10 8 6

B. Nothing

C. 10 7

D. 10 7 4

3. What is the value of total after this loop?

```
let total = 0;
for (let i = 2; i <= 10; i += 2) {
total = total + i;
}
```

A. 10

B. 20

C. 30

D. 45

4. What does this loop print?

```
for (let i = 1; i <= 6; i++) {
if (i % 2 === 0) { continue; }
console.log(i);
}
```

A. 1 3 5 6

B. 1 3 5

C. 1 2 3 4 5 6

D. 2 4 6

5. Which statement about for and while is TRUE?

A. for loops can count but while loops cannot

B. while loops always run at least once

C. while loops cannot use break

D. Any for loop can be rewritten as a while loop and vice versa

6. How many stars does this print in total?

```
for (let r = 1; r <= 4; r++) {
let line = "";
for (let c = 1; c <= r; c++) {
line = line + "*";
}
console.log(line);
}
```

A. 4 **B.** 6

C. 10 **D.** 16

7. What happens when a condition is false at the very start of a while loop?

A. JavaScript converts false to true **B.** The body never runs at all

C. The loop crashes **D.** The body runs once anyway

8. In the grid example, why does the inner loop rebuild line = "" every outer round?

A. So each row starts empty instead of inheriting the last row's stars **B.** To comply with a language rule

C. To make the stars different colors **D.** Because console.log resets variables

9. The variable declared in a for loop header can be used after the loop ends.

☐ True ☐ False

10. break works inside while loops as well as for loops.

☐ True ☐ False

11. A nested loop with an outer count of 3 and inner count of 7 runs its inner body 21 times.

☐ True ☐ False

12. Your program must print "Liftoff!" after counting down from 5 to 1. Explain why a for loop with a negative step (i--) fits, and write the header line.

13. A friend's loop prints one row too many. The header is for (let i = 1; i <= 10; i++) but only 9 rows were wanted. Explain the off-by-one fix.

14. Write a loop that prints all multiples of 5 from 5 to 50 (5, 10, ... 50), one per line.

_____

_____

_____

15. Write nested loops that print a 3x3 square of # characters - three rows, three symbols each.

_____

_____

_____

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. FizzBuzz junior. Print numbers 1 to 15, but replace multiples of 3 with "Fizz", multiples of 5 with "Buzz", and multiples of both with "FizzBuzz".

Hint: Test the BOTH case first - a number like 15 matches three conditions, and order decides the output.

Challenge 2. Sum of digits. For the number 4729, print the sum of its digits (4 + 7 + 2 + 9 = 22) using a loop and arithmetic only - no strings.

Hint: n % 10 pops the last digit; Math.floor(n / 10) removes it. Repeat until nothing is left.

Challenge 3. Prime or not. A prime number is divisible only by 1 and itself. Write a program that decides whether 29 is prime by testing divisors from 2 to Math.sqrt(29) and using break.

Hint: The moment any divisor divides evenly, you can stop and declare "not prime". Reaching the end means prime.

Challenge 4. Pattern mixer. Print this exact pattern using nested loops:

```
1
22
333
4444
55555
```

Hint: Row r prints the number r exactly r times - build the line in the inner loop like the triangle example.

Challenge 5. The flip. Write a loop that prints 10 down to 1 and then "Liftoff!", but WITHOUT using i-- or i = i - 1: change something else in the header instead.

Hint: Count UP from a cleverly chosen start and do arithmetic inside the body - or invert the output math.

Chapter Recap

- while repeats as long as its condition is true; it needs a counter, the condition, and an update.
- A missing update causes an infinite loop - check the update line first when a program hangs.
- for packs start, condition, and update into one header; i++ adds one, i-- subtracts one.
- Off-by-one bugs hide in boundaries: i <= 5 vs i < 5 differ by one round.
- break exits the whole loop; continue skips to the next round.
- Use for for counted rounds, while for unpredictable conditions.
- Nested loops multiply: 3 outer x 4 inner = 12 inner runs.
- Rebuild per-row variables (like line) inside the outer loop to avoid carrying data across rows.

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Forgetting the update - the infinite loop

```
let n = 5;
while (n > 0) {
  console.log(n);
}
```

Fix: The condition never changes, so the loop never ends. Add the update inside the body.

```
let n = 5;
while (n > 0) {
  console.log(n);
  n = n - 1;
}
```

Mistake 2: Off-by-one boundary

```
for (let i = 1; i < 5; i++) {
  console.log(i);  // stops at 4
}
```

Fix: i < 5 gives 1-4. For five rounds ending at 5, use i <= 5 - or start at 0 with i < 5.

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Mistake 3: Resetting the accumulator in the wrong place

```
for (let i = 1; i <= 3; i++) {
  let total = 0;
  total = total + i;
}
```

Fix: Declaring total inside the loop wipes it every round. Declare it before the loop.

```
let total = 0;
for (let i = 1; i <= 3; i++) {
  total = total + i;
}
```

Mistake 4: Using break when you meant continue

```
if (i % 2 === 0) { break; }  // stops the loop!
```

Fix: break ends everything; continue skips one round. Filtering odd/even wants continue.

```
for (let i = 1; i <= 3; i++) {
  if (i % 2 === 0) { continue; }
  console.log(i);
}
```

Mistake 5: Sharing i between nested loops

```
for (let i = 1; i <= 3; i++) {
  for (let i = 1; i <= 2; i++) { /* chaos */ }
}
```

Fix: Each loop needs its own counter: r outside, c inside. Shadowing corrupts both counts.

```
for (let r = 1; r <= 3; r++) {
  for (let c = 1; c <= 2; c++) {
    console.log(r + "-" + c);
  }
}
```

Mini-Project 2: The Number Guessing Game

Every programmer builds a guessing game at some point - it is the perfect storm of randomness, loops, and decisions. The computer picks a secret number between 1 and 100, the player guesses, and the program answers "higher" or "lower" until the secret is found. Then it announces how many attempts it took.

This project combines Chapters 4-6: Math.random for the secret, while for the repeated guessing, if / else if for the hints, and break for the escape. It also introduces one new friend - the prompt() function, which asks the player a question and hands back what they typed as a string.

The complete program

```
// Number Guessing Game
// Runs in the BROWSER console: prompt() is a
// browser function.
let secret = Math.floor(Math.random() * 100) + 1;
let attempts = 0;
let guess = 0;

while (guess !== secret) {
    let raw = prompt("Guess my number (1-100):");
    guess = Number(raw);
    attempts = attempts + 1;

    if (Number.isNaN(guess)) {
        console.log("That is not a number - try again.");
        continue;
    }
    if (guess > secret) {
        console.log("Lower!");
    } else if (guess < secret) {
        console.log("Higher!");
    } else {
        console.log("Correct! You needed " +
            attempts + " attempts.");
        break;
    }
}
```

Line-by-line explanation

Lines 5. The secret is born: Math.random() gives [0, 1), times 100 stretches to [0, 100), floor makes whole values 0-99, and +1 shifts to the fair range 1-100.

Lines 6-7. attempts counts rounds; guess starts at 0 - a value that can never equal the secret, so the loop is guaranteed to run at least once.

Lines 9. The while condition is the whole game: keep going until the guess equals the secret.

Lines 10-12. prompt() opens a dialog and returns TEXT. Number(raw) converts it - the convert-check-calculate pattern from Chapter 4. attempts grows every round, valid or not.

Lines 14-17. If the conversion failed, NaN flows into the comparison and the player gets a friendly warning. continue skips the hint logic and starts the next round.

Lines 18-25. The three-way hint: too high, too low, or exactly right. The winning branch prints the score and breaks out of the loop - the only exit, since the condition itself would never become false without the break.

How It Works

The loop is a conversation: ask, convert, compare, reply. Each round tightens the player's knowledge, and the loop ends only through the break inside the winning branch. This "loop forever until a goal is reached" style is called a game loop, and it powers everything from quiz apps to 3D games.

Notice the defensive design. A player can type anything - "banana", an empty answer, 99999. The program survives all of it: NaN triggers the continue warning, and impossible numbers simply receive higher/lower hints forever until corrected. Robust programs expect silly input.

The attempts counter shows a classic accumulator: declared before the loop, incremented inside it, reported after the win. Watch how its position in the code tells its whole life story.

Things to Try

- Limit the player to 7 attempts: add a check at the top of the loop that breaks with a "You lost!" message when attempts reaches the limit.
- Change the range to 1-1000 and see how the average number of attempts grows.
- After a win, print a rating: 5 or fewer attempts is "Genius", 10 or fewer is "Sharp", otherwise "Persistent!".
- Count how many guesses were too high versus too low, and print both counts at the end.
- Make the game refuse guesses outside 1-100 with a special message (a fourth branch).

CHAPTER 7

Functions

Every skill you have learned so far - printing, calculating, deciding, looping - becomes more powerful when it is packaged under a name you choose. That package is a function: a reusable block of code that performs one job whenever you call it. Functions turn long programs into short, readable ones, and they are the last big tool before your programs become truly organized.

In this chapter you will define functions, feed them inputs, collect their outputs, learn where their variables live, and meet two professional extras: documentation comments that describe what a function does, and try / catch blocks that let a program recover from trouble instead of crashing.

Lesson 7.1 - Defining and Calling Functions

Defining a function is like teaching a skill and giving it a name. The definition starts with the keyword function, then your chosen name, then parentheses, then curly braces holding the body. Defining does NOT run the code - it only teaches it. To make the skill happen, you call the function by writing its name with parentheses.

Define once, call many times

```
function cheer() {
    console.log("You can do it!");
}

cheer();
cheer();
console.log("Between calls");
cheer();
```

The example defines cheer() once but calls it three times, so its message prints three times, interleaved with the between-calls line exactly in call order. The empty parentheses are required in both places: they mark the name as a function call rather than a plain word. Forgetting them is the classic silent bug - writing cheer without () refers to the function itself but never runs it.

Calling requires the parentheses

```
function wave() {
    console.log("Hello there!");
}
wave();     // runs: prints the greeting
// wave;    // does nothing useful: no call
```

Good function names follow the same camelCase rule as variables, and the best names are verbs or verb phrases: printReport, rollDice, greetPlayer. A glance at a verb name tells the reader that something happens here. Grouping related statements into a well-named function is the simplest form of cleanup any program can get.

TIP

When you copy-paste the same three lines twice in a program, stop and wrap them in a function instead. Duplicated code is duplicated bugs waiting to happen.

Lesson Check 7.1

Choose the best answer. Answers are in the Answer Key.

1. What does defining a function do?

A. Deletes old variables

B. Prints the function name

C. Teaches JavaScript a named skill without running it yet

D. Runs its code immediately

2. Which line correctly CALLS a function named cheer?

A. cheer();

B. cheer;

C. function cheer;

D. call cheer

3. What is the best name for a function that prints a report?

A. the_function

B. ReportPrint() in CAPS

C. printReport

D. data2

4. A program defines cheer() but never calls it. What is visible on screen?

A. The function body

B. Nothing

C. The word cheer

D. An error

Lesson 7.2 - Parameters, Arguments, and return

Functions become far more useful when they can receive information. A parameter is a named slot in the function definition; an argument is the actual value you pass in during a call. The definition function greet(name) declares one parameter; the call greet("Maya") supplies one argument. Inside the function, the parameter behaves like a variable holding the passed value.

One parameter, many arguments

```
function greet(name) {
    console.log("Hello, " + name + "!");
}
greet("Maya");
greet("Omar");
greet("Ada");
```

A function may declare several parameters separated by commas, and then each call supplies arguments in the same order - first to first, second to second. Order mix-ups are a classic bug: rollDice(6, 20) and rollDice(20, 6) can mean entirely different things, so check argument order whenever results look strange.

Two parameters, order matters

```
function repeatWord(word, times) {
    for (let i = 0; i < times; i++) {
        console.log(word);
    }
}
repeatWord("Echo", 3);
```

The return statement sends a value BACK to whoever called the function, and it ends the function immediately. Returning is different from printing: console.log shows a value to a human, while return hands the value to the rest of the program to use in calculations, comparisons, or further calls. A function that computes should usually return; one that displays should usually print.

return hands a value back

```
function double(n) {
    return n * 2;
}
let result = double(21);
console.log(result);
console.log(double(7) + 1);
```

WARNING

Code after a return statement in the same block never runs - the function has already handed back its value. If a function seems to ignore its last lines, look for a return above them.

Lesson Check 7.2

Choose the best answer. Answers are in the Answer Key.

1. In function greet(name) { ... }, the word name is a...

A. global variable
B. return value
C. comment
D. parameter

2. What does return do?

A. Sends a value back to the caller and ends the function
B. Prints a value for the user
C. Loops the function again
D. Deletes the function

3. What does console.log(double(7) + 1); print if double returns n * 2?

A. 8
B. An error
C. 15
D. 14

4. For repeatWord(word, times), what happens with the call repeatWord("Hi", 2);?

A. word is 2 and times is "Hi"
B. word is "Hi" and times is 2
C. An error - too few arguments
D. The function must return something

Lesson 7.3 - Scope and Arrow Functions

Scope is the rule about where a variable can be seen. A variable declared inside a function (with let or const) is local: it exists only inside that function and vanishes when the function ends. A variable declared outside all functions is global: every function can see it. Local scope keeps functions independent - two functions can each use a variable called total without any risk of collision.

Local vs global scope

```
let greeting = "Hi";   // global

function demo() {
    let inside = "local only";
    console.log(greeting);   // global works
    console.log(inside);     // local works
}
demo();
console.log(greeting);       // still fine
// console.log(inside);      // error: gone!
```

The rule of thumb in real projects: make variables as local as possible. Global variables are shared by everything, so any function can accidentally change them and cause chaos somewhere far away. Parameters and return values are the safe roads for moving information into and out of functions, and this book sticks to them.

Modern JavaScript offers a compact way to write short functions: the arrow function. Instead of the function keyword it uses a equals-greater-than arrow, and when the body is a single return, both the braces and the return word disappear. const double = (n) => n * 2; defines exactly the same doubling function as Chapter 7.2 - just shorter.

Arrow functions, step by step

```
const double = (n) => {
    return n * 2;
};
const triple = (n) => n * 3;   // short form
console.log(double(8));
console.log(triple(8));
```

NOTE

Arrow functions and function declarations do the same job for our purposes; you will meet both styles in real code. This book writes plain function declarations in lessons - they are the clearest to read - and uses arrow functions for tiny helpers.

Lesson Check 7.3

Choose the best answer. Answers are in the Answer Key.

1. A variable declared with let INSIDE a function is...

A. global
B. visible to the whole file
C. permanent
D. local to that function

2. Can a function READ a global variable?

A. No, never
B. Only if passed as a parameter
C. Only in arrow functions
D. Yes, globals are visible inside functions

3. Why do programmers prefer locals over globals?

A. Globals are illegal in modern JavaScript

B. Locals use fewer characters

C. Locals prevent distant functions from accidentally interfering with each other

D. Locals run faster

4. Which line defines an arrow function that returns n plus 1?

A. function => inc(n)

B. const inc = (n) => n + 1;

C. arrow inc(n) { return n + 1; }

D. const inc = n => n + 1;

Lesson 7.4 - Documenting and Defending: JSDoc and try/catch

A good function explains itself. JavaScript has a documentation convention called JSDoc: a special multi-line comment starting with /** placed directly above a function. Inside, a short description plus @param and @return lines document every input and the output. Editors display these comments as tooltips, and teammates read them instead of reverse-engineering your code.

A documented function

```
/**
 * Builds a full name from two parts.
 * @param {string} first - the given name
 * @param {string} last - the family name
 * @return {string} first and last joined
 */
function fullName(first, last) {
    return first + " " + last;
}
console.log(fullName("Amina", "Haddad"));
```

Even documented functions meet trouble: dividing by zero the user typed, reading a file that is missing, parsing text that is not JSON. The try / catch statement defends against crashes. The risky code goes in the try block; if anything throws an error object, execution jumps straight to the catch block, which receives the error and can print a friendly message instead of letting the program die.

try / catch in action

```
try {
    let broken = JSON.parse("not json!");
    console.log(broken);
} catch (err) {
    console.log("Caught a problem: " +
        err.message);
}
console.log("The program continues.");
```

Two finishing touches complete the picture. A finally block after the catch normally runs whether the try succeeded or the catch handled an error - useful for cleanup like closing files, though it is not an absolute guarantee (a killed process skips it). And your own code can raise errors on purpose with throw new Error("message") when a caller passes something impossible - include a helpful message when you do, because that message is what the catcher will read.

throw, then catch it yourself

```
function divide(a, b) {
    if (b === 0) {
        throw new Error("Cannot divide by zero");
    }
    return a / b;
}
try {
    console.log(divide(10, 2));
    console.log(divide(10, 0));
} catch (err) {
    console.log("Problem: " + err.message);
}
```

NOTE

SyntaxErrors - a missing brace, an unclosed quote - are a special case: they are caught before your program even starts running, so try / catch inside the program cannot rescue them. try / catch is for runtime trouble, not for typos.

Lesson Check 7.4

Choose the best answer. Answers are in the Answer Key.

1. What does a JSDoc comment start with?

A. /**
B. //
C. /*
D. ###

2. What happens when an error is thrown inside a try block?

A. The try block restarts
B. The error is printed twice
C. Execution jumps to the catch block
D. The program crashes immediately

3. What is the finally block for?

A. Making the program faster
B. Repeating the try block
C. Declaring the end of the file
D. Running cleanup code that normally executes whether or not an error occurred

4. Why does try / catch NOT rescue a SyntaxError like a missing brace?

A. SyntaxErrors are too large to catch
B. Because the program cannot start running until the syntax is valid
C. Because braces are optional
D. catch ignores syntax problems by design

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. Which program correctly defines AND uses a function?

A. function hello() { console.log("Hi"); } followed by hello();

B. function hello() { console.log("Hi"); } (nothing else)

C. hello(); written alone with no definition

D. console.log(hello);

2. What does this program print?

```
function add(a, b) {
return a + b;
}
console.log(add(2, 3) * 2);
```

A. 25

B. add

C. 10

D. 5

3. A function contains: console.log("A"); return 5; console.log("B"); What happens when it runs?

A. A and B print, returns 5

B. Only B prints

C. Nothing happens

D. Only A prints; the function returns 5

4. Which design keeps functions independent and safe?

A. Pass data in via parameters and out via return

B. Share one big global variable for everything

C. Use console.log for all data transfer

D. Avoid naming functions

5. What does this print?

```
const shout = (w) =>
w.toUpperCase() + "!";
console.log(shout("go"));
```

(function declarations are also fine)

A. go!

B. GO!

C. GO

D. go

6. A function defined as function f(x) { return x; } can be called before its definition appears in the file.

☐ True ☐ False

7. What does this program print?

```
function mystery(n) {
  if (n > 3) {
    return "big";
  }
  return "small";
}
console.log(mystery(5));
console.log(mystery(2));
```

Output: _____

8. The programmer expected 20 but sees 10 printed and nothing else. Find the bug.

```
function double(n) {
  return n * 2;
  console.log(n);
}
double(10);
```

Bug: _____

9. Explain the difference between a parameter and an argument, using function rollDice(sides) and the call rollDice(6) as your example.

_____

_____

_____

10. Write a documented function (JSDoc comment included) named areaOfRect that takes width and height and RETURNS the area. Call it with 4 and 3 and print the result.

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. How many times does the body run?

```
function f() {
console.log("once");
}
f();
f();
f();
```

A. Three times
B. Zero
C. Once
D. Forever

2. What does this print?

```
function greet(name, emoji) {
console.log("Hi " + name + " " + emoji);
}
greet("Sam", "*");
```

A. Hi * Sam
B. Hi Sam *
C. Hi Sam
D. An error

3. A function needs to give its computed total back to the caller. Which statement belongs at the end?

A. console.log(total);
B. break total;
C. export total;
D. return total;

4. What does this print?

```
const half = (n) => n / 2;
console.log(half(9));
```

A. 5
B. undefined
C. 4.5
D. 4

5. Where is the variable counter (declared with let inside a function) visible?

A. Everywhere in the program
B. Only inside that function
C. Only inside loops
D. Everywhere after the function runs

6. What belongs directly above a function in a JSDoc comment?

A. A description plus @param lines for each input and @return for the output
B. A list of every variable in the program
C. The function's code, copied as text
D. Nothing - JSDoc goes inside the body

7. What does err.message contain inside a catch block?

A. The line number of the catch block
B. The program's output so far
C. The text of the error, such as "Cannot divide by zero"
D. The name of the catch variable

8. Which function is best designed?

A. function work() { /* prints AND returns AND changes a global */ }

B. function calc() { return 42; } // ignores its inputs

C. function f(a, b, c, d, e, f2) { /* does everything */ }

D. function area(w, h) { return w * h; }

9. Writing rollDice without parentheses calls the function.

☐ True ☐ False

10. Arrow functions cannot use parameters.

☐ True ☐ False

11. A throw new Error("oops") inside a try block sends control to the matching catch block.

☐ True ☐ False

12. Your teammate wrote a function with no parameters that prints the same greeting every time. Explain why greet(name) with a parameter is better, and show the two-call improvement.

_____

_____

_____

13. Why is a function that both prints its result AND returns it sometimes a bad idea? What situation would want only one of the two?

_____

_____

_____

14. Write a function celsiusToF(c) that returns the Fahrenheit value (c * 9 / 5 + 32). Call it for 0, 37, and 100, printing each result.

_____

_____

_____

15. Write a function safeDivide(a, b) that THROWS an Error when b is 0 and otherwise returns a / b. Demonstrate it inside a try / catch that prints the error message.

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. Counter factory. Write a function makeCounter() that returns a NEW function each time it is called; each returned function should print its own count starting at 1 and increasing per call. (Hint: a local variable inside makeCounter survives inside the returned function.)

Hint: The returned arrow function can READ the local variable of the function that created it - that is the whole secret.

Challenge 2. Validator. Write isStrong(password) that returns true only when the password is at least 8 characters long, contains a digit, and contains an uppercase letter. Test it on three passwords.

Hint: Loop through characters with a for loop; check each rule; track three boolean flags.

Challenge 3. Error ladder. Write divide(a, b) that throws "Cannot divide by zero" when b is 0 and throws "Inputs must be numbers" when either input is not a number (use typeof). Catch both messages with one try / catch and test three calls.

Hint: Order the checks: type first, then zero. The first throw wins.

Challenge 4. Function library. Write three one-line arrow functions - square, cube, and fourth - that compute powers of n. Then write power(n, p) that picks among them with a switch and returns the right result.

Hint: Store the arrows in variables, then let the switch return the matching one - functions are values!

Challenge 5. The traced function. Rewrite this function so it prints a line at its start, a line before returning, and still returns the correct value:
function double(n) { return n * 2; }

Hint: return ends the function instantly, so any "before returning" line must come ABOVE the return.

Chapter Recap

- A function definition teaches a named skill; a call with parentheses runs it.
- Parameters are named slots in the definition; arguments are the real values supplied by position at call time.
- return sends a value back to the caller and ends the function immediately.
- Printing shows values to humans; returning hands values to the program - different jobs.
- Variables declared inside a function are local; globals are visible everywhere but risky.
- Arrow functions offer compact syntax: const double = (n) => n * 2;
- JSDoc comments (/** ... */) document purpose, @param inputs, and @return output.
- try / catch responds to runtime errors; finally normally runs cleanup; throw raises your own errors with a message.
- SyntaxErrors happen before execution and cannot be caught at runtime.

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Calling without parentheses

```
function bell() { console.log("Ding"); }
bell;
```

Fix: The name alone does nothing. Add the call operators: bell();

```
bell();
```

Mistake 2: Printing when you meant to return

```
function add(a, b) {
  console.log(a + b);
}
let sum = add(2, 3);  // sum is undefined
```

Fix: console.log shows to humans and returns nothing. Use return for data flow.

```
function add(a, b) {
  return a + b;
}
```

Mistake 3: Code after return

```
function f() {
  return 1;
  console.log("never");
}
```

Fix: return ends the function instantly; the trailing line is unreachable. Move it above the return.

```
function f() {
  console.log("runs");
  return 1;
}
```

Mistake 4: Arguments in the wrong order

```
function repeatWord(word, times) { /* ... */ }
repeatWord(3, "Hi");  // loops over the number 3?
```

Fix: Arguments match by position. Supply them in the parameter order: repeatWord("Hi", 3);

```
repeatWord("Hi", 3);
```

Mistake 5: Relying on a global from deep inside a function

```
let total = 0;
function add(n) {
  total = total + n;  // hidden dependency
}
```

Fix: Hidden global edits surprise callers. Take the data in, hand the result out.

```
function addTo(total, n) {
  return total + n;
}
```

CHAPTER 8

Arrays

So far every variable held exactly one value. But programs live on lists: the 40 students in a class, the 500 items in a shop, the 12 months in a year. The array is JavaScript's list - one variable that holds many values in order, ready to be visited one by one, sorted, searched, and transformed.

In this chapter you will create arrays, read and change items by position, add and remove elements at both ends, walk through arrays with modern loops, and use the searching and sorting tools that come built in. Arrays plus loops are the engine room of programming - almost every real feature you have ever clicked was powered by that pair.

Lesson 8.1 - Creating Arrays and Reading Items

An array is written as a comma-separated list of values inside square brackets. You store it in a variable like any other value. The items keep their order, and each one sits at a numbered position called an index - and here is the famous detail: counting starts at 0. The first item is at index 0, the second at index 1, and so on.

Creating and reading

```
let fruits = ["apple", "banana", "cherry"];
console.log(fruits[0]);
console.log(fruits[2]);
console.log(fruits.length);
```

The .length property reports how many items the array holds - always one MORE than the last index, because counting starts at 0. An array of 3 items has indexes 0, 1, and 2. This off-by-one relationship is the single most common source of beginner array bugs, so carve it into memory: length counts, indexes start at zero.

Indexes and the length rule

```
let colors = ["red", "green", "blue"];
console.log(colors.length);   // 3 items
console.log(colors[colors.length - 1]);
// the LAST item trick
```

Asking for an index that does not exist - colors[99] - does not crash; it politely returns undefined. That silent undefined often signals an index mistake, so when a calculation prints undefined, check your arithmetic. Changing an item is as easy as assigning to its index: colors[1] = "yellow" swaps the second item in place.

Changing items and the undefined trap

```
let colors = ["red", "green", "blue"];
colors[1] = "yellow";
console.log(colors);
console.log(colors[99]);   // undefined
```

TIP

Arrays happily mix value types - numbers, strings, booleans in one list - and even other arrays. Mixing types is legal, but most well-designed programs keep one kind of thing per array: all names, or all scores.

Lesson Check 8.1

Choose the best answer. Answers are in the Answer Key.

1. What is the index of the FIRST item in an array?

A. 1
B. It depends on the array
C. -1
D. 0

2. For let a = [10, 20, 30];, what does a.length return?

A. 31
B. undefined
C. 3
D. 2

3. What does a[99] return for the 3-item array above?

A. 0
B. undefined
C. An error stops the program
D. 30

4. Which statement changes the second item to "kiwi"?

A. fruits[1] = "kiwi";
B. fruits(1) = "kiwi";
C. fruits[2] = "kiwi";
D. fruits = "kiwi";

Lesson 8.2 - Adding and Removing: push, pop, shift, unshift

Arrays grow and shrink at both ends, and JavaScript gives each direction its own method. push() adds to the END, and pop() removes from the end. shift() removes from the FRONT, and unshift() adds to the front. All four change the array IN PLACE - the original array itself is modified.

The four edge methods

```
let queue = ["Aya", "Bilal"];
queue.push("Cyrus");     // add to end
console.log(queue);
let first = queue.shift();
console.log(first);
console.log(queue);
```

Now the subtle part that examiners love: these methods return values, and not the ones you might guess. push and unshift return the NEW LENGTH of the array. pop and shift return the REMOVED ITEM. So let n = list.push("x") stores a count in n, while let item = list.pop() stores the item that just left. Confusing the two is a classic quiz question - and a real bug.

What each method returns

```
let list = ["a", "b"];
let len = list.push("c");
console.log(len);        // 3, not "c"
let gone = list.pop();
console.log(gone);       // "c"
```

Sorting lives in this lesson too, with a warning attached. sort() arranges items in place and returns the same array (a reference to it, now rearranged). For strings it sorts alphabetically - but for NUMBERS it sorts text-style, so [10, 2, 33] becomes [10, 2, 33] ordered as "10", "2", "33" - wrong numerically! The cure is a comparator: list.sort((a, b) => a - b) sorts numbers ascending.

Sorting numbers needs a comparator

```
let scores = [10, 2, 33];
scores.sort();
console.log(scores);   // [10, 2, 33] - text order!
scores.sort((a, b) => a - b);
console.log(scores);   // [2, 10, 33]
```

WARNING

sort() and reverse() both MODIFY the original array. If you need the original order preserved, copy first with slice(): let copy = list.slice().sort(...). Assuming sort returns a fresh array is a famous beginner trap.

Lesson Check 8.2

Choose the best answer. Answers are in the Answer Key.

1. Which method adds an item to the END of an array?

A. shift()
B. unshift()
C. push()
D. pop()

2. What does list.push("x") RETURN?

A. The item "x"
B. The array
C. undefined
D. The new length

3. After these lines, what is in list?

```
let list = [1, 2, 3];
list.shift();
list.push(4);
```

A. [2, 3, 4]
B. [1, 2, 3, 4]
C. [2, 3]
D. [4, 2, 3]

4. Why does [10, 2, 33].sort() NOT give [2, 10, 33]?

A. sort() refuses to order numbers
B. Without a comparator, numbers are compared as TEXT, so "10" comes before "2"
C. sort() only works on strings
D. The array is too short to sort

Lesson 8.3 - Walking Through Arrays

The classic way to visit every item is the counted for loop from Chapter 6, using the index: start at 0, continue while i < list.length, step by one. Inside the body, list[i] is the current item. This pattern gives you the position AND the value, which matters when position means something - like question numbers.

Index loop with numbers in the output

```
let teams = ["Lions", "Sharks", "Eagles"];
for (let i = 0; i < teams.length; i++) {
    console.log((i + 1) + ". " + teams[i]);
}
// 1. Lions ... 3. Eagles
```

Notice the (i + 1) trick: the loop counts from 0 for the computer, but humans number lists from 1, so the display adds one. This is the array world's version of a numbering offset - the same idea you will need whenever an interface shows item numbers.

When you need only the VALUES, for...of is cleaner: for (let item of list) hands you each element in order, with no index bookkeeping at all. It cannot run off the end and it cannot miscount. When you need BOTH the value and its position, entries() pairs them up: for (let [i, item] of list.entries()) delivers index and value together - and you can still add 1 for human-friendly numbering.

for...of and entries()

```
let teams = ["Lions", "Sharks", "Eagles"];
for (let t of teams) {
    console.log(t);
}
for (let [i, t] of teams.entries()) {
    console.log((i + 1) + ") " + t);
}
```

One more companion: forEach is a method that takes a function and calls it once per item: list.forEach((item) => console.log(item)). It reads compactly and appears everywhere in professional code. For this book's purposes, remember the rule of thumb: for...of for simple visits, entries() when indexes matter, forEach when a tiny per-item action fits on one line.

NOTE

The square brackets in for (let [i, t] of ...) are called destructuring - they unpack the pair that entries() hands over. You will use the same unpacking style again with objects in Chapter 9.

Lesson Check 8.3

Choose the best answer. Answers are in the Answer Key.

1. Which loop header visits every index of list safely?

A. for (let i = 0; i <= list.length; i++)

B. for (let i = 0; i <= list.length - 2; i++)

C. for (let i = 0; i < list.length; i++)

D. for (let i = 1; i <= list.length; i++)

2. What does for (let item of list) give you each round?

A. The index only

B. The value only

C. Index and value

D. The length

3. In the teams example, why print (i + 1) instead of i?

A. i is a string

B. console.log requires it

C. To make the list longer

D. i starts at 0 for the computer but humans number lists from 1

4. What does for (let [i, t] of teams.entries()) deliver?

A. Each index and its value together

B. Only the values

C. Only the indexes

D. The array reversed

Lesson 8.4 - Searching, Joining, and Slicing

Arrays answer questions about their contents with search methods. includes(value) returns true or false: is this value anywhere in the array? indexOf(value) returns the INDEX of the first match, or -1 when the value is absent - the same -1 convention as strings in Chapter 3. Both compare strictly, so "5" and 5 are different items.

includes and indexOf

```
let pets = ["cat", "dog", "fish"];
console.log(pets.includes("dog"));
console.log(pets.indexOf("fish"));
console.log(pets.indexOf("bird"));
```

Two builders reshape arrays into new forms. join(separator) glues all items into one STRING with your chosen separator between them - "a,b,c" from ["a", "b", "c"] with a comma. And slice(start, end) copies a section into a NEW array, from index start up to but not including end, exactly like the string version. slice() with no arguments copies the whole array - the standard way to protect an array before sorting it.

join and slice

```
let letters = ["a", "b", "c", "d"];
console.log(letters.join("-"));
let part = letters.slice(1, 3);
console.log(part);
let copy = letters.slice();
copy.push("z");
console.log(letters.length);  // still 4
```

Here is a subtlety worth slowing down for: copying an array with slice() copies the LIST of positions, not deep-cloning every object inside it. For arrays of simple values - numbers, strings, booleans - slice() gives a fully independent copy, and that covers everything in this book. (Arrays of objects share their objects; that nuance waits for later studies.)

These tools combine into real recipes. A search-and-report: if (list.indexOf(x) !== -1) tells you the value exists AND where. A display formatter: list.join(", ") turns data into a readable line. A safe sort: list.slice().sort((a, b) => a - b) sorts a copy and keeps the original intact. Recipes like these are the daily bread of working programmers.

TIP

includes answers "is it there?"; indexOf answers "where is it?" - reaching for the wrong one is harmless but wasteful. Ask a true/false question with includes; ask for a position with indexOf.

Lesson Check 8.4

Choose the best answer. Answers are in the Answer Key.

1. What does ["a", "b", "c"].join("-") return?

A. "abc" **B.** ["a", "-", "b"]

C. 3 **D.** "a-b-c"

2. What does pets.indexOf("bird") return when bird is not in the array?

A. -1 **B.** 0

C. true **D.** undefined

3. What does letters.slice(1, 3) return for ["a", "b", "c", "d"]?

A. ["a", "b"] **B.** ["b", "c"]

C. ["b", "c", "d"] **D.** ["c"]

4. Why write list.slice().sort(...) instead of list.sort(...)?

A. sort cannot run on the original array **B.** slice makes the sort numeric

C. sort modifies the original; slice first protects it **D.** slice is faster

Lesson 8.5 - Transforming Arrays: map, filter, reduce, splice

Loops visit arrays one item at a time, but many tasks are so common that JavaScript ships with a ready-made tool. Three methods - map, filter, and reduce - power a huge share of the JavaScript you will ever read online. Each one takes a function, walks the array for you, and builds something new from it - and the original array is never changed.

map transforms every item

```
let prices = [5, 10, 15];
let doubled = prices.map((p) => p * 2);
console.log(doubled);
console.log(prices);   // unchanged
```

map calls your function once for every item, collects each return value into a NEW array, and hands that array back - always the same length as the original. The arrow function (p) => p * 2 is the style from Chapter 7: it takes one price and returns the doubled value. Transform, collect, return: that is the whole idea.

filter keeps the items that pass

```
let scores = [45, 82, 91, 38, 77];
let passing = scores.filter((s) => s >= 60);
console.log(passing);
console.log(passing.length);
```

filter keeps an item only when your function returns true for it - here, the scores of 60 or more. The result is a new array that can be shorter than the original, and the rejected items are simply left out, not deleted from anywhere. Together, filter and map cover an amazing amount of daily programming: keep the interesting records, then transform them for display.

reduce boils everything down to one value

```
let sales = [10, 25, 7];
let total = sales.reduce((acc, n) => acc + n, 0);
console.log(total);
```

reduce is the odd one out: it returns a single VALUE, not an array. The accumulator (acc) carries the running result from item to item, starting from the 0 you supply after the function. Here it walks through 10, 25, and 7, adding each one: 10, then 35, then 42. Sums, counts, and averages all begin with this one pattern.

One more tool completes the family. splice removes or inserts items IN PLACE, at any position - unlike slice from Lesson 8.4, which copies and touches nothing. splice(start, howMany) deletes that many items starting at the index you name and returns them as a new array; put a 0 in the second slot and a value after it, and splice inserts instead.

splice removes and inserts in place

```
let lineup = ["Aya", "Bilal", "Cyrus"];
let removed = lineup.splice(1, 1);
console.log(removed);
console.log(lineup);
lineup.splice(1, 0, "Dalia");
console.log(lineup);
```

WARNING

slice copies; splice cuts. Mixing them up can delete data you only meant to copy - and because sort, reverse, and splice all mutate in place, the safe habit is to copy with slice() first whenever you need the original kept.

This lesson closes with a small operator you will see constantly: the ternary operator. It squeezes a whole if/else choice into one line and produces a VALUE: condition ? valueIfTrue : valueIfFalse. It shines inside map, where a full if statement would not fit.

The ternary operator ? :

```
let ages = [12, 15, 17, 9];
let groups = ages.map((a) =>
    (a >= 13 ? "teen" : "kid"));
console.log(groups);
```

NOTE

The ternary operator is an expression, not a statement - it produces a value you can store, return, or nest inside another call. Use it for short either-or choices, and use a full if/else when the logic grows beyond one line, because readability always wins.

TIP

These tools compose: scores.filter((s) => s >= 60).map((s) => s * 2) filters first and transforms the result in one chained line. Chains read left to right, and each step receives the new array that the previous step produced.

Lesson Check 8.5

Choose the best answer. Answers are in the Answer Key.

1. After let big = nums.map((n) => n * 10);, what is true?

A. nums has been replaced by bigger numbers
B. big and nums are the same array
C. map returns a single number
D. big is a new array; nums is unchanged

2. Which items does filter keep?

A. Every item in the array
B. Only the first and last items
C. Only items where the function returns true
D. Only items where the function returns false

3. What does [10, 25, 7].reduce((acc, n) => acc + n, 0); return?

A. 42
B. [10, 25, 7]
C. [42]
D. 0

4. Which method REMOVES items from the original array in place?

A. map()
B. splice()
C. slice()
D. concat()

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. Which array holds exactly four items?

A. [1, [2, 3], 4]
B. [5, 10, 15]
C. ["a", "b", "c", "d"]
D. []

2. What does console.log(x[1]); print for x = ["sun", "moon", "star"]?

A. "sun"
B. "star"
C. undefined
D. "moon"

3. After let q = [1, 2]; q.unshift(0); what is q?

A. [2, 1]
B. [0]
C. [0, 1, 2]
D. [1, 2, 0]

4. Which expression gives the LAST item of arr, whatever its length?

A. arr[arr.length - 1]
B. arr[arr.length]
C. arr.last()
D. arr[-1]

5. What does this loop print?

```
let nums = [4, 8];
for (let n of nums) {
console.log(n * 2);
}
```

A. 0 and 1 **B.** 8

C. 8 and 16 **D.** 4 and 8

6. The pop() method returns the new length of the array.

☐ True ☐ False

7. What does this program print?

```
let tools = ["pen", "ink", "pad"];
tools.push("clip");
console.log(tools.length);
console.log(tools[3]);
```

Output: _____

8. The programmer wanted the last color but sees undefined. Find the bug.

```
let colors = ["red", "green", "blue"];
console.log(colors[colors.length]);
```

Bug: _____

9. Explain the relationship between an array's length and its valid indexes, and why colors[colors.length - 1] is the standard last-item recipe.

_____

_____

_____

10. Create an array of your three favorite subjects, print the first one, replace the last one with "Art", and print the whole array.

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. What does [7, 8, 9].length evaluate to?

A. 2 **B.** 3

C. 8 **D.** 9

2. After let s = ["a"]; s.push("b"); s.push("c"); what is s?

A. ["b", "c"] **B.** ["a", "b", "c"]

C. ["c", "b", "a"] **D.** ["a", "c"]

3. What does let last = ["x", "y", "z"].pop(); store in last?

A. "x" **B.** 3

C. undefined **D.** "z"

4. Which expression tests whether "cat" is anywhere in pets?

A. Both A and B work **B.** pets.indexOf("cat") >= 0

C. pets.includes("cat") **D.** cat in pets

5. What does [3, 1, 2].sort((a, b) => a - b) return?

A. undefined **B.** [1, 2, 3]

C. [3, 2, 1] **D.** [3, 1, 2]

6. What does ["a", "b", "c", "d"].slice(2) return?

A. ["b", "c"] **B.** ["d"]

C. ["c", "d"] **D.** ["a", "b"]

7. Which loop prints every item with its position, like "0: pen"?

A. for (let p of pens) **B.** for (let i of pens.length)

C. pens.forEach(pen) **D.** for (let [i, p] of pens.entries())

8. What does ["red", "green"].join(" & ") produce?

A. "red & green" **B.** "redgreen"

C. ["red", " & ", "green"] **D.** "red&&green"

9. The includes method returns the index of a found item.

☐ True ☐ False

10. An array can hold numbers and strings at the same time.

☐ True ☐ False

11. sort() leaves the original array unchanged.

☐ True ☐ False

12. You need to show a numbered shopping list (1., 2., 3.) from an array. Which loop style would you choose and why?

_____

_____

_____

13. A teammate writes list[list.length] to read the last item "for safety". What actually happens, and what is the correct recipe?

_____

_____

_____

14. Start with an empty array, push the numbers 5, 10, 15, then print the array, its length, and the second item.

_____

_____

_____

15. Given let letters = ["d", "a", "c", "b"]; print them sorted alphabetically WITHOUT changing the original array, then print the original.

_____

_____

_____

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. Second largest. Without sorting, find the second largest number in [12, 45, 2, 41, 31, 8] using one loop and two tracker variables.

Hint: Track best and secondBest; when a value beats best, the old best is demoted to second.

Challenge 2. Palindromic array. Check whether [1, 2, 3, 2, 1] reads the same forwards and backwards, using a loop that compares item i with item length - 1 - i.

Hint: You only need to compare the first half against the mirrored second half.

Challenge 3. Rotate right. Move every item of [1, 2, 3, 4, 5] one step to the right, so the last becomes first: [5, 1, 2, 3, 4]. Print before and after.

Hint: pop the last item, then unshift it to the front - the edge methods do all the work.

Challenge 4. Merge and dedupe. Combine [1, 2, 3] and [3, 4, 5] into one array WITHOUT duplicates, keeping first-seen order: [1, 2, 3, 4, 5].

Hint: Build a result array; push each candidate only if result.includes(candidate) is false.

Challenge 5. The sandwich test. Given ["bread", "cheese", "bread"], print whether the FIRST and LAST items are equal using one comparison - then do the same for ["bread", "cheese", "ham"] without changing your code.

Hint: The first item is arr[0]; the last is arr[arr.length - 1]. One expression works for any length.

Chapter Recap

- Arrays are ordered lists in square brackets; indexes start at 0 and length is always one more than the last index.
- Reading a missing index returns undefined, not an error.
- push adds to the end and pop removes from it; unshift adds to the front and shift removes from it - all in place.
- push/unshift return the new LENGTH; pop/shift return the removed ITEM.
- sort() orders in place; numbers need a comparator (a, b) => a - b; copy first with slice() to preserve the original.
- Counted for loops give index + value; for...of gives values; entries() pairs them; forEach calls a function per item.
- includes answers true/false; indexOf returns the first index or -1.
- join(separator) builds a string; slice(start, end) copies a section; slice() copies everything.
- map transforms every item into a new array; filter keeps the items that pass; reduce boils the array down to one value.
- splice removes or inserts in place and returns what it removed; the ternary operator ? : chooses between two values in one line.

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Using length as an index

```
let last = list[list.length];  // undefined!
```

Fix: The last index is length - 1; length itself is one past the end.

```
let last = list[list.length - 1];
```

Mistake 2: Sorting numbers without a comparator

```
[10, 2, 33].sort();  // [10, 2, 33]
```

Fix: Default sort compares text. Pass (a, b) => a - b for numeric order.

```
[10, 2, 33].sort((a, b) => a - b);  // [2, 10, 33]
```

Mistake 3: Assuming pop returns the array

```
let shorter = list.pop();  // shorter is the ITEM
```

Fix: pop returns the removed item; the array changed in place. Keep using the original variable.

```
list.pop();  // list is now shorter
```

Mistake 4: Destroying the original with sort

```
let ranked = scores.sort();  // scores also changed!
```

Fix: sort works in place. Copy first: scores.slice().sort().

```
let ranked = scores.slice().sort((a, b) => a - b);
```

Mistake 5: Off-by-one loop bounds

```
for (let i = 0; i <= list.length; i++) {  // reads undefined
```

Fix: The last valid index is length - 1, so use the strict < in the boundary.

```
for (let i = 0; i < list.length; i++) {
  console.log(list[i]);
}
```

CHAPTER 9

Objects

Arrays collect items in order, but many real things are not simple lists. A student has a name, an age, and a grade. A car has a brand, a color, and a speed. An object bundles related facts about ONE thing into a single value, giving each fact a name - and it is the data structure that powers JavaScript everywhere, from web pages to entire databases.

In this chapter you will build objects, read and change their properties two different ways, combine objects with arrays into realistic records, give objects their own functions, and meet JSON - the text format that lets objects travel into files and across the internet.

Lesson 9.1 - Properties: Dot and Bracket Access

An object stores key-value pairs called properties. The key is the property's name; the value is anything you can store in a variable. Objects are written with curly braces, keys followed by colons, pairs separated by commas. Read the example as: "a student whose name is Maya, whose age is 12, and whose grade is B".

Creating an object

```
let student = {
    name: "Maya",
    age: 12,
    grade: "B"
};
console.log(student.name);
console.log(student.age);
```

You reach a property with a dot: student.name gives "Maya". Assigning to it changes the value, and assigning to a NEW key adds a property on the spot - objects grow whenever you need them to. The bracket style student["name"] does the same job with a string; it exists mainly for keys chosen at runtime, such as student[question], where the key comes from a variable.

Changing, adding, and brackets

```
let student = { name: "Maya", age: 12 };
student.age = 13;            // update
student.city = "Cairo";      // add
console.log(student);
let key = "name";
console.log(student[key]);   // bracket style
```

Choosing between dot and bracket is a style decision with a rule: use the dot when you know the key's name while writing the code - it is shorter and clearer - and use brackets when the key lives in a variable or contains spaces or unusual characters. Reading a property that does not exist returns undefined, the same polite miss as arrays.

WARNING

student[name] without quotes means "use the VALUE of the variable name as the key" - not the key "name". The missing quotes are a classic bug: if name happens to hold "Maya", you read student.Maya, which does not exist and gives undefined.

Lesson Check 9.1

Choose the best answer. Answers are in the Answer Key.

1. Which syntax creates an object?

A. let x = { name: "Maya" };
B. let x = (name: "Maya");
C. let x = [name: "Maya"];
D. let x = <name: "Maya">;

2. How do you read the age property of student?

A. student->age
B. student(age)
C. age.student
D. student.age

3. What does student.city return when city was never added to the object?

A. null
B. undefined
C. An error
D. ""

4. When is bracket access REQUIRED instead of the dot?

A. When the value is a number
B. When the object has many properties
C. When the key name comes from a variable at runtime
D. Never - dots always work

Lesson 9.2 - Arrays of Objects: Real Records

The strongest data pattern in beginner programming is an array of objects: a list where every item is one structured record. A class is a list of students; a shop is a list of products; a music app is a list of songs. Each object has the same shape - the same keys - which makes the list predictable to loop over.

A tiny database

```
let class_ = [
    { name: "Maya", score: 92 },
    { name: "Omar", score: 85 },
    { name: "Hana", score: 97 }
];
console.log(class_[1].name);
console.log(class_[2].score);
```

Read class_[1].name from the inside out: class_[1] picks the second record, and .name reaches into it. The combination unlocks powerful loops: visit each record with for...of, then use the dot to pull out exactly the field you need. This loop-plus-dot pattern is how reports, leaderboards, and feeds are built.

Looping over records

```
let class_ = [
    { name: "Maya", score: 92 },
    { name: "Omar", score: 85 },
    { name: "Hana", score: 97 }
];
for (let s of class_) {
    console.log(s.name + " scored " + s.score);
}
```

The for...in loop offers a different walk: it visits the KEYS of one object, one at a time. Inside the loop, object[key] with brackets reads each value - the bracket style is required here because the key is a variable. Use for...in to inspect or summarize an object whose keys you did not predict.

for...in visits keys

```
let student = { name: "Maya", age: 12,
    grade: "B" };
for (let key in student) {
    console.log(key + " = " + student[key]);
}
```

NOTE

The variable name class_ ends with an underscore because class is a reserved word in JavaScript and cannot be used as a variable name. The underscore keeps the meaning while dodging the keyword.

Lesson Check 9.2

Choose the best answer. Answers are in the Answer Key.

1. What does roster[0].name mean for an array of student objects?

A. The name key of the whole array
B. An array of all names
C. A syntax error
D. The name property of the FIRST record

2. What does for...in visit on an object?

A. Its methods only
B. Its length
C. Its keys
D. Its values

3. Inside for (let key in student), how do you read each value?

A. student[key]
B. student.key
C. student(key)
D. key.student

4. Why did the lesson name its array class_ instead of class?

A. Underscores are faster
B. class is a reserved word and cannot be a variable name
C. To make the array private
D. It is a spelling mistake

Lesson 9.3 - Objects with Methods

A function stored as an object property is called a method - and objects full of data plus methods are the closest thing in this book to a living thing: they hold facts AND know how to do things. Defining one looks like any property, except the value is a function. Calling it uses the dot: student.introduce().

A method in the wild

```
let student = {
    name: "Maya",
    score: 92,
    introduce: function () {
        console.log("I am " + student.name);
    }
};
student.introduce();
```

Inside a method, the keyword this refers to the object the method was called on - so this.name is "the name of whoever is speaking". Using this instead of the object's variable name keeps the method correct even if the object is copied to another variable later. Methods that compute from the object's own data, like a getAverage method reading this.scores, are the cleanest design in this book.

this inside methods

```
let player = {
    name: "Zaid",
    points: 40,
    describe: function () {
        return this.name + " has " +
            this.points + " points";
    }
};
console.log(player.describe());
```

A shorter method syntax exists and is worth knowing: you may drop the colon and the word function, writing describe() { ... } directly. Both forms are common in real code. Notice that arrow functions are NOT ideal for methods that use this - the arrow's this does not point at the object. Save arrows for standalone helpers.

Shorthand method syntax

```
let counter = {
    count: 0,
    bump() {
        this.count = this.count + 1;
    }
};
counter.bump();
counter.bump();
console.log(counter.count);  // 2
```

Lesson Check 9.3

Choose the best answer. Answers are in the Answer Key.

1. What is a method?

A. A loop over keys
B. A JSON rule
C. A function stored as an object property
D. Any variable in an object

2. Inside a regular method, what does this refer to?

A. The object the method was called on
B. The global scope
C. The function itself
D. The previous method

3. Why prefer this.name over student.name inside a method?

A. It is shorter to type
B. It keeps working even if the object is copied to another variable
C. this is required by JSON
D. student does not exist inside functions

4. Which shorthand defines a method correctly?

A. bump: function => {}
B. function bump() { ... } inside the object
C. () bump {}
D. bump() { ... }

Lesson 9.4 - JSON: Objects as Text

Objects live in memory and vanish when the program ends. To store them in a file - or send them across the internet - they must become TEXT. JSON (JavaScript Object Notation) is that text format, and it looks remarkably like JavaScript code: keys in quotes, values after colons, arrays and objects nesting freely.

stringify: object to text

```
let student = { name: "Maya", age: 12 };
let text = JSON.stringify(student);
console.log(text);
console.log(typeof text);
```

JSON.stringify(obj) converts any object or array into one JSON string. With two extra arguments - null and 2 - it formats the text with two-space indentation, which makes saved files friendly for humans to read. The reverse trip uses JSON.parse(text): give it valid JSON and it rebuilds a live object, properties and all.

parse: text back to object

```
let text = '{"name": "Maya", "age": 12}';
let student = JSON.parse(text);
console.log(student.name);
console.log(student.age + 1);
```

One rule trips everyone once: JSON requires DOUBLE quotes around keys and strings - single quotes are illegal JSON, and trailing commas after the last property are forbidden. If JSON.parse throws an error, the text was not valid JSON; wrap the parse in try / catch when the text comes from outside your program. This exact round trip - stringify out, parse back - powers the file storage in the final project.

The full round trip

```
let roster = [
    { name: "Maya", score: 92 },
    { name: "Omar", score: 85 }
];
let text = JSON.stringify(roster, null, 2);
let copy = JSON.parse(text);
console.log(copy[0].name);
console.log(copy.length);
```

TIP

JSON has become the universal language of data online: weather apps, game servers, and school portals all ship JSON. Learning it here is learning one of the most transferable skills in all of programming.

Lesson Check 9.4

Choose the best answer. Answers are in the Answer Key.

1. What does JSON.stringify do?

A. Turns JSON text into an object
B. Turns an object into a JSON text string
C. Deletes object properties
D. Saves a file to disk

2. What does JSON.parse('{"a": 1}') return?

A. The number 1
B. An error
C. An object with a = 1
D. The text "{"a": 1}"

3. Which is VALID JSON?

A. '{"name": "Maya"}'
B. '{name: "Maya"}'
C. '{'name': 'Maya'}'
D. '{"name": "Maya",}'

4. What do the extra arguments in JSON.stringify(obj, null, 2) do?

A. Encrypt the output
B. Limit the output to 2 properties
C. Parse twice for safety
D. Format the text with 2-space indentation

Lesson 9.5 - Looping Over Objects: keys, values, entries

Arrays answer the question "what is at index 2?", but objects answer "what is stored under the key name?" Often you want the opposite: to walk over EVERY property without naming keys one by one. JavaScript puts three tools on the global Object for exactly that: Object.keys, Object.values, and Object.entries.

Object.keys and Object.values

```
let user = { name: "Aya", age: 12, city: "Cairo" };
console.log(Object.keys(user));
console.log(Object.values(user));
```

Object.keys(obj) returns an array of the property NAMES, and Object.values(obj) returns their values. Both results are real arrays, so every tool from Chapter 8 works on them immediately: .length counts the properties, for...of walks them, and map can transform them.

Counting and visiting properties

```
let user = { name: "Aya", age: 12 };
console.log(Object.keys(user).length);
for (let key of Object.keys(user)) {
    console.log(key + " = " + user[key]);
}
```

The star of the family is Object.entries, which returns an array of [key, value] PAIRS - one little two-item array per property. Paired with the destructuring you met in Lesson 8.3, the loop becomes remarkably clean: for (let [key, value] of Object.entries(obj)) names both halves of each pair at once.

Object.entries with destructuring

```
let scores = { Aya: 91, Bilal: 78, Cyrus: 84 };
for (let [name, score] of
    Object.entries(scores)) {
    console.log(name + " scored " + score);
}
```

Destructuring also works on a single object in one line: let { title, pages } = book; creates a variable called title holding book.title, and a variable called pages holding book.pages. The variable names must match the property names - that is the whole trick. It is the tidy way to pull several settings out of a configuration object at once.

Object destructuring in one line

```
let book = { title: "JS Foundations", pages: 200 };
let { title, pages } = book;
console.log(title);
console.log(pages);
```

One last modern convenience: optional chaining, written as a question mark followed by a dot. Reading user.pet.name crashes with an error when user.pet does not exist, because undefined has no properties to offer. With ?., JavaScript politely stops instead: user.pet?.name returns undefined rather than throwing, whenever the part before the dot is missing.

Optional chaining ? .

```
let user = { name: "Aya" };
console.log(user.pet?.name);
console.log(user.name?.length);
```

NOTE

The question mark in ?. short-circuits: when the value before it is undefined or null, JavaScript skips the rest of the chain and returns undefined. Think of it as the polite dot - the same access, but no crash when something is missing.

TIP

These tools appear in almost every real program that handles data: keys to count or check properties, entries to build reports and tables, and ?. to read uncertain data safely. You now know the complete beginner toolkit for objects AND arrays.

Lesson Check 9.5

Choose the best answer. Answers are in the Answer Key.

1. What does Object.keys({ a: 1, b: 2 }) return?

A. ["a", "b"] **B.** [1, 2]

C. ["a: 1", "b: 2"] **D.** 2

2. What does Object.entries({ Aya: 91 }) return?

A. ["Aya"] **B.** [91]

C. [["Aya", 91]] **D.** ["Aya", 91]

3. After let { pages } = book;, what does the variable pages hold?

A. undefined **B.** The value of book.pages

C. The string "pages" **D.** The whole book object

4. What does user.pet?.name return when user.pet is undefined?

A. An error stops the program **B.** null

C. "" **D.** undefined

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. Which object literal is written correctly?

A. let car = ["brand: Fiat", "year: 2020"]; **B.** let car = (brand: "Fiat");

C. let car = { brand: "Fiat", year: 2020 }; **D.** let car = { brand = "Fiat", year = 2020 };

2. For let book = { title: "Dune", pages: 412 };, what does book.pages return?

A. "pages" **B.** "Dune"

C. undefined **D.** 412

3. What does this print?

```
let menu = { drink: "tea", size: "L" };
for (let k in menu) {
console.log(k);
}
```

A. tea then L **B.** drink then size

C. drink = tea, size = L **D.** An error

4. A variable key holds "score". Which line reads that property of record?

A. record.key **B.** record[key]

C. record("score") **D.** key.record

5. What does JSON.stringify([1, 2, 3]) return?

A. "1,2,3" **B.** 6

C. "[1, 2, 3]" **D.** [1, 2, 3]

6. Adding a brand-new property to an existing object requires special permission from JavaScript.

☐ True ☐ False

7. What does this program print?

```
let pet = { kind: "cat", age: 3 };
console.log(pet.kind);
console.log(pet["age"] + 1);
```

Output: _____

8. The programmer expected "Maya" but sees undefined. Find the bug.

```
let student = { name: "Maya" };
let key = "name";
console.log(student.key);
```

Bug: _____

9. Explain the difference between student.name and student[name], including what happens when name holds "Maya".

_____

_____

_____

10. Create an object describing your favorite game (title, hours, fun: true). Print one property with dot access and add a new property platform with a value.

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. What does this print?

```
let o = { a: 1, b: 2 };
o.c = 3;
console.log(o.b + o.c);
```

A. 3 **B.** 5

C. 23 **D.** undefined

2. Which statement about object keys is TRUE?

A. Keys are usually names, read with dot or bracket access **B.** Keys must be numbers

C. Keys cannot be added after creation **D.** Keys must match their values

3. What does this loop print?

```
let conf = { user: "adm", port: 80 };
for (let k in conf) {
console.log(k + ": " + conf[k]);
}
```

A. adm and 80 **B.** user and port

C. An error **D.** user: adm and port: 80

4. For roster = [{n: "A", s: 9}, {n: "B", s: 7}];, what does roster[1].n return?

A. "B" **B.** "A"

C. 9 **D.** undefined

5. What does this print?

```
let hero = {
name: "Kira",
show() { return this.name + "!"; }
};
console.log(hero.show());
```

A. hero! **B.** undefined!

C. Kira! **D.** name!

6. Which JSON text is valid?

A. "{'k': 1}" **B.** '{k: 1}'

C. '{"k": 1,}' **D.** '{"k": 1}'

7. After let t = JSON.stringify(o); what is typeof t?

A. "object" **B.** "string"

C. "json" **D.** "undefined"

8. Why wrap JSON.parse in try / catch for outside data?

A. Parsing is slow

B. JSON files are always broken

C. try / catch makes parsing faster

D. Invalid JSON throws an error that would crash the program

9. Objects can store arrays as property values.

☐ True ☐ False

10. for...in can also walk the indexes of an array.

☐ True ☐ False

11. JSON.stringify({a: undefined}) includes the a property in the output.

☐ True ☐ False

12. A teammate asks why this.score is used inside a method instead of writing the object's name directly. Give the one-sentence reason from the lesson.

13. Describe the JSON round trip used to save and load data, naming both functions in the right order.

14. Build an object movie with title, year, and a ratings array of three numbers. Print the title and the average rating using a loop.

15. Write the JSON string for an object with keys city (Cairo) and population (10000000), parse it back, and print the population with a thousands separator using toLocaleString.

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. Record builder. Write a function makeStudent(name, scores) that returns an object with the name, the scores array, and a method average() that computes the mean of this.scores. Demonstrate it on two students.

Hint: The method can loop over this.scores - this makes the function reusable for any student.

Challenge 2. Inventory report. Given products = [{name: "Pen", price: 2, stock: 100}, {name: "Book", price: 9, stock: 40}, {name: "Bag", price: 15, stock: 12}]; print each product as "Pen x100 - 2" and then the TOTAL value of the inventory (price times stock, summed).

Hint: One loop for the lines, an accumulator for the total value.

Challenge 3. Key collector. Write a function keysOf(obj) that returns an ARRAY of the object's keys using for...in and push. Test it on { a: 1, b: 2, c: 3 } and print the result joined with ", ".

Hint: for...in hands you keys; collect them like any list.

Challenge 4. JSON doctor. The text '{"name": "Ada", "age": 36,}' fails JSON.parse. Identify the JSON rule it breaks, fix the text, then parse it and print the age.

Hint: Look at the very end of the object - one character does not belong in JSON.

Challenge 5. Depth charge. Build a school object containing a name and a students ARRAY of two objects (each with name and grade). Print the school name, then each student's name and grade using nested access.

Hint: Objects inside arrays inside objects - navigate one level at a time: school.students[0].name.

Chapter Recap

- Objects store key-value pairs in braces; keys name their values.
- Dot access (obj.key) for known names; bracket access obj["key"] or obj[keyVar] for dynamic keys.
- Missing properties read as undefined; assigning to a new key adds the property.
- Arrays of objects are the standard record-collection pattern.
- for...of walks arrays; for...in walks an object's keys (read values with obj[key]).
- Methods are function properties; this refers to the object the method was called on.
- Shorthand methods drop the colon and function keyword.
- JSON.stringify turns data into text (null, 2 = pretty); JSON.parse rebuilds it; JSON demands double quotes and no trailing commas.
- Object.keys, Object.values, and Object.entries turn an object into arrays; destructuring { title } = book pulls properties into variables by name.
- Optional chaining (?.) reads missing data safely: it returns undefined instead of crashing.

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Using a variable key with a dot

```
let key = "name";
console.log(student.key);
```

Fix: The dot reads a literal "key" property. Brackets use the variable's value.

```
console.log(student[key]);
```

Mistake 2: Building an object with = inside braces

```
let p = { name = "Ali" };
```

Fix: Literals use key: value with colons - = is for assignments.

```
let p = { name: "Ali" };
```

Mistake 3: Single quotes in JSON text

```
JSON.parse("{'a': 1}");  // throws
```

Fix: JSON requires double quotes around keys and strings. Fix the text before parsing.

```
JSON.parse('{"a": 1}');
```

Mistake 4: Expecting for...in to give values

```
for (let k in obj) {
  console.log(k);  // keys only
```

Fix: for...in visits keys. Read values with obj[k].

```
for (let k in obj) {
  console.log(obj[k]);
}
```

Mistake 5: Forgetting this inside methods

```
let p = {
  name: "Ada",
  hi() { return name + "!"; }
};
```

Fix: Bare name looks for a variable that does not exist. this.name reaches the object's own property.

```
let p = {
  name: "Ada",
  hi() {
    return this.name + "!";
  }
};
```

Mini-Project 3: The Student Gradebook

Time to combine the heavy machinery: arrays, objects, functions, and loops all working together. The gradebook stores students as objects inside an array, computes each student's average with a function, assigns letter grades with the Chapter 5 branching chain, and prints a formatted report with a loop.

This is the shape of thousands of real programs: a list of records, a few calculations, a report at the end. Master this pattern here and the final project will feel familiar.

The complete program

```
// Student Gradebook
let students = [
    { name: "Layla", scores: [88, 92, 79] },
    { name: "Omar", scores: [95, 87, 91] },
    { name: "Hana", scores: [72, 68, 80] }
];

function average(scores) {
    let total = 0;
    for (let i = 0; i < scores.length; i++) {
        total = total + scores[i];
    }
    return total / scores.length;
}

function letterFor(avg) {
    if (avg >= 90) {
        return "A";
    } else if (avg >= 75) {
        return "B";
    } else if (avg >= 60) {
        return "C";
    }
    return "Needs work";
}

for (let s of students) {
    let avg = average(s.scores);
    console.log(s.name + ": " + avg.toFixed(1) +
        "  (" + letterFor(avg) + ")");
}
```

Line-by-line explanation

Lines 2-6. The roster: an array of objects. Each object bundles a name string with a scores array - structured data, exactly like a tiny database row.

Lines 8-14. average() walks the scores array with a counting for loop, accumulates the total, and returns the mean. The loop knows the length from .length, so it adapts to any number of scores.

Lines 16-27. letterFor() is pure Chapter 5: the if / else if chain tests from highest to lowest and the first match wins. The final return handles everything below 60 without an else.

Lines 29-33. The report loop uses for...of to visit each student object, calls both functions, and prints one tidy line per student. The data structure does the organizing; the loop does the walking.

How It Works

The design splits responsibilities cleanly: storage (the array), computation (average), classification (letterFor), and presentation (the report loop). Each piece could change without breaking the others - swap the grade boundaries, or add students, and nothing else moves. This separation is the heart of software design.

Functions earn their keep here: average is called once per student but written once in total. If the average formula needed fixing - say, dropping the lowest score - you would edit one function, and all students benefit instantly.

for...of shines because the code never touches indexes: it asks for each student, not student number 0, 1, 2. The loop cannot run off the end of the array, and the intent reads like English.

Things to Try

- Add yourself to the roster with your real scores and check that the report includes you automatically.
- Write a second function, bestScore(student), that returns the highest single score using Math.max in a loop, and print it per student.
- Count how many students earned a B or better, and print the count after the report.
- Give the class an "easy day": loop through students and add 5 to every score (careful: modify the arrays, then reprint the report).
- Print a class average: the average of the three student averages.

CHAPTER 10

Files, Modules, and Real Programs

Everything you have built so far lives in memory and vanishes when the program ends. Real programs remember: documents save, games store progress, apps keep settings. This final chapter gives your programs a memory (files) and organization (modules) - the two features that turn scripts into software.

You will learn how modules share code between files, how to read and write text files with Node.js, how to append instead of overwriting, why "utf-8" appears in every file command, and how to combine it all into one organized program - ready for the final project that follows this chapter.

Lesson 10.1 - Modules: Sharing Code Between Files

As programs grow, programmers split them across several files, each one a module with a single clear job. JavaScript connects modules with two keywords: export marks what a file GIVES to others, and import collects what another file OFFERS. Exporting is like labeling boxes in a warehouse; importing is carrying the labeled boxes to your own room.

Two files, one program

```
// math-tools.js
function double(n) {
    return n * 2;
}
export { double };
```

The importing file

```
// main.js
import { double } from "./math-tools.js";
console.log(double(21));
```

The import line names the file path exactly, starting with ./ which means "in the same folder as me". A module may export several things in one statement: export { double, triple, square }; and the importer chooses only what it needs. This choose-what-you-take style keeps dependencies visible: one glance at the top of a file tells you exactly which powers it borrows.

Node.js - the JavaScript runner from Chapter 1 - ships with ready-made modules of its own, and you import them the same way. The file system module is called node:fs, and it carries every file tool in this chapter. Built-in modules need no ./ prefix: import { readFileSync } from "node:fs"; pulls the reading tool straight out of the box.

TIP

Name module files after their job: math-tools.js, save-load.js, student-data.js. Six months from now, the import lines should read like a table of contents for your program.

NOTE

For import and export to work in Node, your folder needs a package.json file containing {"type": "module"} - or your files can use the .mjs extension. One line of setup unlocks the whole system.

Lesson Check 10.1

Choose the best answer. Answers are in the Answer Key.

1. Which keyword marks what a module GIVES to other files?

A. import **B.** export

C. include **D.** share

2. What does "./math-tools.js" mean in an import?

A. A folder on the internet **B.** A special shortcut

C. A file in the same folder as the importing file **D.** A built-in Node module

3. How is the built-in file system module imported?

A. from "node:fs" **B.** from "files"

C. from "./fs.js" **D.** from "system"

4. Why does the lesson suggest naming modules after their job?

A. Node requires it **B.** Shorter names are illegal

C. It makes import lines read like a table of contents **D.** It makes imports run faster

Lesson 10.2 - Reading Files with readFileSync

The node:fs module (fs for file system) holds the tools, and readFileSync is the reader. Give it a file name and the encoding "utf-8", and it hands back the entire file contents as one string. The encoding tells Node how to translate the file's raw bytes into characters - "utf-8" is the world standard that every modern text file uses, and including it means your text arrives as proper letters instead of mysterious codes.

Reading a text file

```
import { readFileSync } from "node:fs";

let text = readFileSync("story.txt", "utf-8");
console.log(text);
console.log("Characters: " + text.length);
```

Where does Node look for story.txt? In the CURRENT WORKING DIRECTORY - the folder your terminal was pointing at when you ran the program, not necessarily the folder of the file itself. If you run node main.js from your home folder while main.js lives in projects/, Node searches your home folder for story.txt. The built-in process.cwd() prints the working directory, which is the first thing to check when a file is mysteriously "not found".

Where am I? Where are my files?

```
import { readFileSync } from "node:fs";

console.log("Looking in: " + process.cwd());
let text = readFileSync("data.txt", "utf-8");
console.log(text.slice(0, 50));
```

Files on disk can disappoint: the name may be misspelled, the file may not exist, or another program may be holding it locked. Any of these throws an error and - without protection - crashes the program. Wrapping the read in try / catch turns a crash into a message, exactly as Chapter 7 promised.

Defensive reading

```
import { readFileSync } from "node:fs";

try {
    let text = readFileSync("data.txt",
        "utf-8");
    console.log("Got it: " + text.length);
} catch (err) {
    console.log("Could not read file: " +
        err.message);
}
```

Lesson Check 10.2

Choose the best answer. Answers are in the Answer Key.

1. What does readFileSync return?

A. An array of lines
B. A number of characters
C. A file object
D. The whole file as one string

2. Why pass "utf-8" to every file read and write?

A. It tells Node how to translate bytes into characters
B. It compresses the file
C. It is required by law in some countries
D. It makes the file write faster

3. Where does Node look for "story.txt" when you run a program?

A. Always the folder containing the .js file
B. The current working directory of the terminal
C. The internet
D. Wherever the file was created

4. What does process.cwd() tell you?

A. The program's line count
B. How much memory is free
C. The file encoding
D. The current working directory

Lesson 10.3 - Writing and Appending Files

Writing is the mirror image of reading. writeFileSync (name, text, "utf-8") puts text INTO a file, completely replacing whatever was there - and if the file does not exist, Node creates it first. This replace-everything behavior makes writeFileSync perfect for saving a fresh, complete version of your data: the whole story, the full list, the updated score table.

Creating and overwriting

```
import { writeFileSync } from "node:fs";

writeFileSync("shopping.txt",
    "milk\neggs\nflour\n", "utf-8");
console.log("Saved.");
```

But replacing is not always what you want. A diary, a log, a high-score table: these GROW. appendFileSync adds text to the END of an existing file - and like its sibling, it creates the file if needed. Each call glues its text after whatever came before, which is exactly how logs work: one entry at a time, oldest first. (The same effect is available by passing the flag "a" to writeFileSync, but the dedicated appendFileSync reads more clearly.)

Appending one entry at a time

```
import { appendFileSync, readFileSync }
    from "node:fs";

appendFileSync("diary.txt",
    "Day 1: learned loops\n", "utf-8");
appendFileSync("diary.txt",
    "Day 2: learned functions\n", "utf-8");
console.log(readFileSync("diary.txt",
    "utf-8"));
```

Notice the \n at the end of each appended entry: without it, entries would glue into one endless line. Ending every entry with a newline is the simple convention that keeps log files readable - one line per event. Combine append for adding and read for reviewing, and you have the complete notebook that programs like chat apps and diaries are built on.

WARNING

writeFileSync is fearless: it overwrites without asking. Accidentally pointing it at the wrong file name can erase hours of saved data in one call. Double-check the file name, and when data must accumulate, prefer appendFileSync.

Lesson Check 10.3

Choose the best answer. Answers are in the Answer Key.

1. What does writeFileSync do to an existing file?

A. Refuses to change it
B. Renames it
C. Replaces its entire contents
D. Adds to its end

2. Which tool adds text to the END of a file?

A. writeFileSync
B. appendFileSync
C. readFileSync
D. pushFileSync

3. What happens if you append to a file that does not exist yet?

A. Node creates the file first, then appends
B. An error crashes the program
C. Nothing happens at all
D. The text is lost silently

4. Why end each appended log entry with \n?

   A. It encrypts the entry

   B. It makes writes faster

   C. So entries stay on separate lines in the file

   D. It is required by appendFileSync

Lesson 10.4 - Organizing a Real Program

Everything now assembles into a working shape. A well-organized program keeps its modules by job: one file for data definitions, one for calculations, one for file storage, and a main file that wires them together. The main file's imports read like a list of ingredients, and each job stays findable. This layout - small modules, clear exports, one conductor - scales from ten-line exercises to ten-thousand-line applications.

A project layout

```
// study-log/
//   logData.js   - data helpers
//   stats.js     - calculations
//   main.js      - the conductor

// main.js
import { sessions } from "./logData.js";
import { totalMinutes } from "./stats.js";
console.log(totalMinutes(sessions));
```

When data crosses the file boundary, it travels as text - and Chapter 9 built the translator. JSON.stringify turns your arrays and objects into a string that writeFileSync can store; JSON.parse turns what readFileSync brings back into living data again. This save-JSON, read-JSON round trip is the simplest possible database, and it is genuinely used by small tools everywhere.

The data round trip

```
import { writeFileSync, readFileSync }
    from "node:fs";

let scores = [88, 92, 79];
writeFileSync("scores.json",
    JSON.stringify(scores), "utf-8");
let loaded = JSON.parse(
    readFileSync("scores.json", "utf-8"));
console.log(loaded[1]);
```

Real programs also expect trouble at every file border: the folder may not exist, the disk may be full, the JSON may be damaged by an interrupted save. The professional pattern wraps each file operation in try / catch, reports a calm message through err.message, and - where possible - falls back to a sane default, such as starting with an empty list when the save file is missing. finally can tidy up after either path, and it normally runs whether the try succeeded or the catch handled the error.

Load with a safe fallback

```
import { readFileSync } from "node:fs";

function loadScores() {
    try {
        let text = readFileSync("scores.json",
            "utf-8");
        return JSON.parse(text);
    } catch (err) {
        console.log("Starting fresh: " +
            err.message);
        return [];
    }
}
let scores = loadScores();
```

NOTE

There is a difference between the two error families you have now met: syntax errors stop the program before it starts (no catching possible), while runtime errors - missing files, bad JSON - happen during execution and can be caught. Design your file code assuming the file might be absent; the catch block is your safety net.

Lesson Check 10.4

Choose the best answer. Answers are in the Answer Key.

1. Which pair converts data for file storage and back?

A. writeFileSync / readFileSync
B. export / import
C. push / pop
D. JSON.stringify / JSON.parse

2. A well-organized project splits files by...

A. Alphabetical order
B. Job: data, calculations, storage, conductor
C. File size
D. Creation date

3. What is a safe fallback when loading a missing save file?

A. Return an empty list and continue
B. Crash with a full error dump
C. Delete the program
D. Ask the internet

4. Which error family CANNOT be caught with try / catch?

A. A thrown Error with a message
B. A SyntaxError from a missing brace
C. A missing file error
D. Invalid JSON during parse

Practice Questions

Work on paper first, then try each one in the console. Answers are in the Answer Key.

1. Which line correctly imports the double function from a local module?

A. const double = require math-tools;
B. include double from "math-tools";
C. import { double } from "./math-tools.js";
D. import math-tools.double;

2. A file story.txt contains "Hello\nWorld". After reading with "utf-8", what is text.length?

A. 2
B. 5
C. 11
D. 12

3. Which sequence adds "new entry" to the END of log.txt without erasing it?

A. writeFileSync("log.txt", "new entry", "utf-8");
B. readFileSync("log.txt", "new entry");
C. export "new entry" to "log.txt";
D. appendFileSync("log.txt", "new entry", "utf-8");

4. Your program says "ENOENT: no such file" for "data.txt" even though the file exists next to your .js file. What is the FIRST thing to check?

A. The file's font
B. Your import statements
C. process.cwd() - the terminal's working directory
D. Reinstall Node.js

5. What does this print?

```
import { readFileSync } from
"node:fs";
try {
console.log(readFileSync("missing.txt",
"utf-8"));
} catch (err) {
console.log("safe");
}
```

A. An uncaught error
B. undefined
C. safe
D. The file contents

6. writeFileSync creates the file automatically if it does not exist.

☐ True ☐ False

7. What does this program print? (assume diary.txt did not exist before)

```
import { appendFileSync, readFileSync } from
  "node:fs";
appendFileSync("diary.txt", "Day 1\n",
  "utf-8");
appendFileSync("diary.txt", "Day 2\n",
  "utf-8");
console.log(readFileSync("diary.txt",
  "utf-8"));
```

Output: _____

8. Every run of this saver erases the previous scores. The programmer wanted to ADD each new score. Find the bug.

```
import { writeFileSync } from "node:fs";
writeFileSync("scores.txt", "95\n", "utf-8");
```

Bug: _____

9. Explain the JSON round trip for saving a scores array: which two functions, in which order, and why the second one is needed on load.

_____

_____

_____

10. Write a program that reads names.txt (utf-8), prints how many characters it has, and catches a missing-file error with a friendly message.

_____

_____

_____

Mixed Quiz

Fifteen questions covering the whole chapter. Answers are in the Answer Key.

1. Which statement about modules is TRUE?

A. export works only with functions

B. A module can export several things at once

C. Import must grab everything a module offers

D. Built-in modules need the ./ prefix

2. What does "utf-8" control?

A. The file's size limit

B. The sort order of lines

C. The file's permissions

D. How bytes become characters

3. After these calls, what does log.txt contain?
writeFileSync("log.txt", "A\n", "utf-8");
appendFileSync("log.txt", "B\n", "utf-8");

A. A then B, on two lines
B. B only
C. A only
D. An error

4. Which read is SAFEST for a file that may not exist?

A. readFileSync(name, "utf-8") with no protection
B. readFileSync inside try / catch with a fallback
C. Reading it twice to be sure
D. Using writeFileSync first to create it

5. What does the ./ in "./logData.js" signal?

A. A hidden file
B. A TypeScript file
C. A relative path in the same folder
D. A built-in module

6. What does JSON.parse(readFileSync("d.json", "utf-8")) give you?

A. The data rebuilt as live objects/arrays
B. The raw file text
C. The number of properties
D. A promise

7. Why is finally useful around file operations?

A. It retries the file operation
B. It deletes the file
C. It makes errors disappear
D. It normally runs cleanup whether the try worked or the catch handled an error

8. Which program structure matches the lesson's recommendation?

A. Small modules by job plus a main file that wires them together
B. One giant file with everything
C. Every function in its own folder
D. No main file at all

9. readFileSync("note.txt") without "utf-8" returns a Buffer of raw bytes instead of readable text.

☐ True ☐ False

10. Two appendFileSync calls to the same file will place the second text before the first.

☐ True ☐ False

11. A missing file causes a runtime error that try / catch can handle.

☐ True ☐ False

12. Your friend's log program prints each new entry on the same line as the last. What one character sequence is missing from each append, and why does it matter?

_____

_____

_____

13. Explain the difference between writeFileSync and appendFileSync, and name one job that suits each.

_____

_____

_____

14. Write a program that saves the array [3, 6, 9] to numbers.json using JSON, then reads it back and prints the second item.

_____

_____

_____

15. Write loadOrEmpty(name) that returns the parsed contents of a JSON file, or an empty array with a warning when the file is missing.

_____

_____

_____

Challenges

These are hard thinking problems. Read each hint before you start, and do not worry if your first attempt does not work. Full solutions are in the Answer Key.

Challenge 1. Line counter. Write a program that reads notes.txt and prints how many LINES it contains. (Hint: split the text on "\n" and count the pieces; remember a trailing newline can create one empty last piece - filter or account for it.)

Hint: text.split("\n") turns the file into an array of lines; length does the rest.

Challenge 2. Safe diary. Build a two-function program: addEntry(text) appends a timestamped line to diary.txt, and showEntries() prints the file with entries numbered. Demonstrate both.

Hint: new Date().toLocaleString() makes a readable timestamp; entries() numbers the lines.

Challenge 3. High-score file. scores.json holds a JSON array of numbers. Write a program that reads it (empty array if missing), adds a new score of 97, saves it back, and prints the best score with Math.max and spread or a loop.

Hint: This is the full round trip: read, parse, modify, stringify, write.

Challenge 4. Module maker. Move your totalMinutes function into its own file stats.js with an export, and write main.js to import it and print the total for [30, 45, 25]. Show both files.

Hint: One file exports, one imports - keep the paths relative with ./

Challenge 5. The corrupted save. data.json contains "not json!" and your program must survive: catch the parse error, print the message, and continue with an empty object. Show the try / catch around JSON.parse specifically.

Hint: Reading may succeed while PARSING fails - each step can need its own guard.

Chapter Recap

- export labels what a module offers; import { x } from "./file.js" brings it in; built-ins come from "node:fs".
- readFileSync(name, "utf-8") returns the whole file as a string; without the encoding you get raw bytes.
- Relative file names resolve from process.cwd() - the terminal's working directory, not the script folder.
- writeFileSync replaces a file completely; appendFileSync adds to the end; both create missing files.
- End appended log entries with \n to keep one entry per line.
- JSON.stringify saves data as text; JSON.parse revives it - the simplest database.
- Wrap file operations in try / catch; err.message carries the explanation; finally normally runs cleanup.
- Split projects into modules by job: data, computation, storage, and one conductor file.
- Syntax errors strike before runtime; file and parse errors are runtime errors you can catch.

Common Mistakes

Almost every beginner makes these mistakes. Learn to spot them now and you will debug faster later.

Mistake 1: Using writeFileSync for a growing log

```
writeFileSync("log.txt", entry, "utf-8");  // wipes!
```

Fix: Every call replaces the file. Appending needs appendFileSync.

```
appendFileSync("log.txt", entry + "\n", "utf-8");
```

Mistake 2: Forgetting "utf-8" on reads

```
let text = readFileSync("a.txt");  // raw Buffer!
```

Fix: Without an encoding you get bytes, not text. Pass "utf-8" on every text read.

```
let text = readFileSync("a.txt", "utf-8");
```

Mistake 3: Blaming the script folder for missing files

```
// node run from Desktop, file sits in projects/
readFileSync("data.txt", "utf-8");  // ENOENT
```

Fix: Relative names resolve from process.cwd(). Run from the right folder or use a path relative to the working directory.

```
console.log("Looking in " + process.cwd());
```

Mistake 4: Parsing before catching

```
let data = JSON.parse(
  readFileSync("d.json",
  "utf-8"));  // crashes on bad JSON
```

Fix: Outside text can be invalid JSON. Wrap the parse in try / catch with a fallback.

```
try {
  data = JSON.parse(text);
} catch (err) {
  data = {};
}
```

Mistake 5: Importing built-ins with ./

```
import { readFileSync } from "./node:fs";
```

Fix: The ./ prefix is for YOUR files. Built-ins use the node: prefix with no slash.

```
import { readFileSync } from "node:fs";
```

The Final Project: Study Log Manager

Everything in this book converges here. The Study Log Manager is a real program with real files: it records study sessions into a log, saves the log to a file on disk, reads it back, appends new entries, and prints a summary report. It uses variables, strings, numbers, decisions, loops, functions, arrays, objects, modules, and file handling - the complete toolkit from Chapters 1 to 10.

The project runs under Node.js and is built in six steps. Each step adds one capability, and every step runs on its own, so you can test as you go - the way professional developers actually work. Create a folder called study-log and save each file inside it.

Step 1: The session data (logData.js)

Modules start with what they GIVE to others. This file exports a starter array of session objects and a helper that builds new ones. Every session records the subject, the minutes studied, and a short note.

```
// logData.js - the data module
let sessions = [
    { subject: "Math", minutes: 45, note: "algebra practice" },
    { subject: "English", minutes: 30, note: "reading" },
    { subject: "Science", minutes: 60, note: "lab report" }
];

function makeSession(subject, minutes, note) {
    return { subject: subject, minutes: minutes,
        note: note };
}

export { sessions, makeSession };
```

Step 2: Reading it back (readLog.js)

Before writing anything, prove that importing works. This tiny program imports the data module and prints one line per session using for...of. Run it with: node readLog.js

```
// readLog.js - test the import
import { sessions } from "./logData.js";

for (let s of sessions) {
    console.log(s.subject + " - " + s.minutes +
        " min (" + s.note + ")");
}
```

Step 3: Statistics (stats.js)

A second module, exporting pure computation: the total minutes, the best session, and a formatted summary string. Notice that a module can export several things at once.

```
// stats.js - computation module
function totalMinutes(list) {
    let total = 0;
    for (let s of list) {
        total = total + s.minutes;
    }
    return total;
}

function bestSession(list) {
    let best = list[0];
    for (let s of list) {
        if (s.minutes > best.minutes) {
            best = s;
        }
    }
    return best;
}

function summary(list) {
    return "Sessions: " + list.length +
        ", Total: " + totalMinutes(list) + " min";
}

export { totalMinutes, bestSession, summary };
```

Step 4: Saving to a file (saveLog.js)

Now the program gains a memory. The node:fs module (short for file system) writes the log to log.txt using writeFileSync, always with "utf-8" so every computer reads the file the same way. JSON.stringify turns the array into text first - the Chapter 9 trick.

```
// saveLog.js - write the log to disk
import { writeFileSync } from "node:fs";
import { sessions } from "./logData.js";

let text = JSON.stringify(sessions, null, 2);
writeFileSync("log.txt", text, "utf-8");
console.log("Saved " + sessions.length +
    " sessions to log.txt");
console.log("Saved in folder: " + process.cwd());
```

Step 5: Appending and reading (addLog.js)

Real logs grow. appendFileSync adds text to the END of an existing file - creating it first if needed - while readFileSync brings the data back. JSON.parse converts the file text into a living array again.

```
// addLog.js - append a session, then read all
import { appendFileSync, readFileSync } from
    "node:fs";
import { makeSession } from "./logData.js";

let fresh = makeSession("Art", 25, "sketching");
appendFileSync("log.txt",
    JSON.stringify(fresh) + "\n", "utf-8");
console.log("Appended one session.");

let text = readFileSync("log.txt", "utf-8");
console.log("--- log.txt now contains ---");
console.log(text);
```

Step 6: Putting it together (main.js)

The conductor imports both modules, wraps risky file work in try / catch so a missing file produces a friendly message instead of a crash, and prints the final report.

```
// main.js - the Study Log Manager
import { sessions, makeSession } from
    "./logData.js";
import { totalMinutes, bestSession, summary }
    from "./stats.js";
import { readFileSync, writeFileSync } from
    "node:fs";

sessions.push(makeSession("History", 40, "timeline"));

try {
    writeFileSync("log.txt",
        JSON.stringify(sessions, null, 2), "utf-8");
    let text = readFileSync("log.txt", "utf-8");
    console.log("File OK, " + text.length +
        " characters.");
} catch (err) {
    console.log("File problem: " + err.message);
}

console.log("=== STUDY LOG REPORT ===");
console.log(summary(sessions));
let best = bestSession(sessions);
console.log("Best session: " + best.subject +
    " (" + best.minutes + " min)");
for (let s of sessions) {
    console.log("- " + s.subject + ": " +
        s.minutes + " min");
}
```

Combining the files

- Save all five files (logData.js, readLog.js, stats.js, saveLog.js, addLog.js, main.js) in ONE folder called study-log - modules find each other by relative path.
- Run each step in order from that folder: node readLog.js, node saveLog.js, node addLog.js, and finally node main.js.
- For the import syntax to work, your folder needs a file named package.json containing {"type": "module"} - or rename the files with the .mjs extension. The book recommends package.json.
- After main.js runs, open log.txt in any text editor: the sessions are stored as readable JSON, one array that grows every time you run the program.

How It Works

The architecture is three layers: data (logData), computation (stats), and coordination (main). Each layer imports only what it needs, which is exactly how large real-world programs stay manageable - thousands of small modules instead of one giant file.

The file layer uses the round trip every data app relies on: object array -> JSON.stringify -> file, then file -> JSON.parse -> object array. As long as the text obeys JSON rules, the objects come back alive with all their properties.

try / catch around the file work means a missing file or a locked folder produces one calm message instead of a stacking crash. finally could tidy up after either path - it normally runs whether the try succeeded or the catch handled an error.

Extension Challenges

- Add a deleteLast() function to logData.js that removes the final session with pop() and re-save the file.
- Track the date of each session: add a date property to makeSession using new Date().toDateString() and show it in the report.
- Write a goals feature: if totalMinutes is under 120, print "Keep going!" - otherwise print "Goal reached!".
- Sort the report by minutes using sort with a comparator before printing, so the longest session comes first.
- Count how many sessions mention reading in their note using includes, and print the count.

Glossary

Eighty-four words you now own. Terms are alphabetical; lesson references point to where each idea was first taught.

appendFileSync	The Node tool that adds text to the end of a file, creating it if needed. *(Ch 10, L3)*
Argument	The actual value you pass into a function when calling it. *(Ch 7, L2)*
Array	An ordered list of values stored in one variable, written in square brackets. *(Ch 8, L1)*
Arrow function	A compact function written with the => arrow, such as (n) => n * 2. *(Ch 7, L3)*
Assignment	Storing a value into a variable with the = operator. *(Ch 2, L1)*
Boolean	A value that is either true or false. *(Ch 2, L4)*
Bracket access	Reading object properties with square brackets and a string key. *(Ch 9, L1)*
break	A statement that exits a loop immediately. *(Ch 6, L3)*
Built-in function	A ready-made function the environment provides, such as console.log. *(Ch 1, L2)*
Callback	A function you pass to another function so it can be called at the right moment, such as the test inside filter(). *(Ch 8, L5)*
camelCase	The naming style that starts lowercase and capitalizes each new word: totalPrice. *(Ch 2, L4)*
Case-sensitive	Treating capital and small letters as different; console.log and Console.log are not the same. *(Ch 1, L2)*
catch	The block that receives and handles an error thrown inside a try block. *(Ch 7, L4)*
Code block	Statements grouped inside curly braces { }. *(Ch 5, L2)*
Comment	A note for humans inside code, ignored by the computer; starts with // or sits between /* and */. *(Ch 1, L4)*
Comparator	A function passed to sort() that decides the order, such as (a, b) => a - b. *(Ch 8, L2)*
Concatenation	Joining strings together with the + operator. *(Ch 2, L3)*
Condition	The boolean test a loop or if statement checks before acting. *(Ch 5, L2)*
console.log()	The built-in function that prints values to the screen; a function, not a keyword. *(Ch 1, L2)*
const	A keyword that creates a variable whose binding cannot be reassigned. *(Ch 2, L1)*
continue	A statement that skips the rest of the current loop round. *(Ch 6, L3)*
Current working directory	The folder a program considers "here"; relative file names resolve from it (process.cwd()). *(Ch 10, L2)*
Destructuring	Unpacking values from an array or object into variables, like the [i, item] of entries(). *(Ch 8, L3)*
else	The branch that runs when an if condition is false. *(Ch 5, L2)*

Encoding	The rule for translating file bytes into characters; "utf-8" is the universal standard. *(Ch 10, L2)*
entries()	An array method that pairs each index with its value for looping. *(Ch 8, L3)*
Error	An object describing a problem; thrown errors can be caught at runtime. *(Ch 7, L4)*
Escape sequence	A backslash pair that inserts a special character, such as \n or \t. *(Ch 3, L2)*
Execution	The moment code actually runs, statement by statement, top to bottom. *(Ch 1, L3)*
Expression	Code that produces a value, such as 2 + 3 or name.toUpperCase(). *(Ch 2, L2)*
false	One of the two boolean values. *(Ch 2, L4)*
filter()	An array method that keeps only the items whose test returns true, and returns a NEW array. *(Ch 8, L5)*
Finally	The block that normally runs after try/catch whether or not an error occurred. *(Ch 7, L4)*
Float	Informal name for a number with a decimal point; subject to tiny binary rounding errors. *(Ch 4, L2)*
for loop	The loop that packs start, condition, and update into one counted header. *(Ch 6, L2)*
for...in	The loop that visits the KEYS of an object. *(Ch 9, L2)*
for...of	The loop that visits the VALUES of an array in order. *(Ch 8, L3)*
Function	A named, reusable block of code that can take inputs and return an output. *(Ch 7, L1)*
Global variable	A variable declared outside all functions, visible everywhere. *(Ch 7, L3)*
Index	The numbered position of an item in an array or string; counting starts at 0. *(Ch 8, L1)*
indexOf()	A search that returns the first position of a match, or -1 when absent. *(Ch 3, L3)*
Infinite loop	A loop whose condition never becomes false; usually a missing update line. *(Ch 6, L1)*
Infinity	A special numeric value greater than every other number; used here as a starting "best" tracker. *(Ch 8, L4)*
join()	An array method that glues items into one string with a separator. *(Ch 8, L4)*
JSDoc	The /** */ comment convention that documents a function's purpose, parameters, and return. *(Ch 7, L4)*
JSON	JavaScript Object Notation - the text format for storing and sending objects and arrays. *(Ch 9, L4)*
let	The keyword that creates a variable whose value can be reassigned. *(Ch 2, L1)*
Local variable	A variable declared inside a function, visible only there. *(Ch 7, L3)*
Loop	A structure that repeats a block of code: while, for, for...of. *(Ch 6, L1)*
map()	An array method that runs a function on every item and returns a NEW array of the results. *(Ch 8, L5)*
Math object	The built-in toolbox of number functions: round, floor, ceil, abs, sqrt, random, and friends. *(Ch 4, L1)*
Method	A function stored as an object property, called with a dot. *(Ch 9, L3)*

Module	A file that exports code for other files to import. *(Ch 10, L1)*
NaN	"Not a Number" - the value produced by failed numeric conversions; detect it with Number.isNaN. *(Ch 4, L4)*
Node.js	The program that runs JavaScript outside the browser, including file operations. *(Ch 1, L2)*
Object	A value that stores named key-value pairs in curly braces. *(Ch 9, L1)*
Optional chaining	The ?. operator that returns undefined instead of crashing when a property in the chain is missing. *(Ch 9, L5)*
Parameter	The named slot in a function definition that receives an argument. *(Ch 7, L2)*
parseFloat()	A converter that reads a decimal number from the start of a string. *(Ch 4, L4)*
parseInt()	A converter that reads a whole number from the start of a string. *(Ch 4, L4)*
pop()	An array method that removes and returns the LAST item. *(Ch 8, L2)*
push()	An array method that adds to the end and returns the new length. *(Ch 8, L2)*
readFileSync	The Node tool that returns a file's contents as a string. *(Ch 10, L2)*
reduce()	An array method that combines every item into one total value using an accumulator. *(Ch 8, L5)*
return	The statement that hands a value back from a function and ends it. *(Ch 7, L2)*
Scope	The region of code where a variable is visible; locals live inside functions. *(Ch 7, L3)*
Short-circuit evaluation	Skipping the right side of && or || when the left side already decides the result. *(Ch 5, L3)*
Slice	To copy a section: slice(start, end) works on both strings and arrays. *(Ch 3, L4)*
Statement	A single complete instruction, normally ending with a semicolon. *(Ch 1, L3)*
String	Text wrapped in quotes; immutable - methods return new strings. *(Ch 2, L3)*
Stringify	To convert data into text with JSON.stringify. *(Ch 9, L4)*
switch	A statement that matches one value against fixed case labels. *(Ch 5, L4)*
Syntax	The strict spelling and punctuation rules of a programming language. *(Ch 1, L1)*
SyntaxError	An error caught before the program runs, such as a missing brace; cannot be caught at runtime. *(Ch 7, L4)*
Template literal	A backtick string that evaluates ${...} placeholders and can span lines. *(Ch 3, L1)*
Ternary operator	The one-line conditional written a ? b : c that chooses between two values. *(Ch 8, L5)*
this	Inside a method, the object the method was called on. *(Ch 9, L3)*
throw	A statement that raises your own error with a message. *(Ch 7, L4)*
toFixed()	A number method that rounds for display and returns a string. *(Ch 4, L2)*
try	The block that guards risky code so its errors can be caught. *(Ch 7, L4)*
Typeof	The operator that reports a value's type as a string. *(Ch 2, L4)*
Variable	A named storage place for a value. *(Ch 2, L1)*

while loop	The loop that repeats as long as its condition stays true. *(Ch 6, L1)*
writeFileSync	The Node tool that writes text to a file, replacing its contents. *(Ch 10, L3)*

Answer Key

Every question in the book is answered here. Correct choices appear in green, followed by the reason - and, for multiple choice, why each other option is wrong. Challenge solutions appear at the end in step-by-step form.

Lesson Check Answers

Chapter 1 - Hello, JavaScript

Lesson 1.1 - What Is Programming?

Q1 - Correct answer: C (A list of instructions that tells a computer what to do)
Q: *What is a computer program?*

Why: A program is exactly a list of instructions that the computer follows one at a time, in order.
Why the other options are wrong:

A (A picture stored on the internet) - A picture is a file, not a set of instructions.

B (A machine that stores files) - A machine is hardware; a program is software that runs on hardware.

D (A type of computer memory) - Memory stores data; it is not a program itself.

Q2 - Correct answer: B (It runs in web browsers and also outside them, for example with Node.js)
Q: *Which statement about JavaScript is true?*

Why: JavaScript runs in every major browser and, thanks to Node.js, on servers and computers outside browsers too.
Why the other options are wrong:

A (It is the same language as Java) - JavaScript and Java are two unrelated languages with similar names.

C (It only runs inside the game industry) - JavaScript is used far beyond games, for example on almost every website.

D (It was invented before the internet existed) - JavaScript was created in 1995, long after the early internet.

Q3 - Correct answer: D (syntax)
Q: *The strict spelling and punctuation rules of a programming language are called...*

Why: Syntax is the set of rules that says how valid code must be written so the computer can read it.
Why the other options are wrong:

A (grammar school) - "Grammar school" is a type of school, not a programming term.

B (alphabet) - The alphabet is only the set of letters, not the full rule system.

C (keyboards) - A keyboard is a physical input device.

Q4 - Correct answer: A (It stops and reports a problem)
Q: *What does the computer do if an instruction is unclear?*

Why: Computers do not guess. When code cannot be understood, execution stops and an error is reported.
Why the other options are wrong:

B (It guesses what you probably meant) - Computers never silently guess your intention.

C (It skips the instruction and continues) - Unclear instructions are not skipped; they stop the program.

D (It asks another computer what to do) - No other computer is consulted; the error appears on your own machine.

Lesson 1.2 - Running Your First Code

Q1 - Correct answer: B (The developer console)

Q: *Which tool lets you type JavaScript and see results immediately inside a browser?*

Why: The developer console (usually opened with F12) accepts JavaScript and shows results at once.

Why the other options are wrong:

A (A calculator app) - A calculator app is a separate program, not a JavaScript console.

C (The address bar) - The address bar takes web addresses, not code.

D (The bookmarks menu) - Bookmarks store links to pages, not code.

Q2 - Correct answer: C (a built-in function you can call)

Q: *console.log() is best described as...*

Why: console.log() is a ready-made function provided by the environment; you call it with a message in parentheses.

Why the other options are wrong:

A (a comment) - Comments are ignored notes; console.log() does work.

B (a web browser) - A web browser is a program that hosts JavaScript, not console.log itself.

D (a keyword of the language grammar) - It is not a grammar keyword like let or if - it is a function.

Q3 - Correct answer: D (5)

Q: *What does console.log(2 + 3); print?*

Why: JavaScript first calculates 2 + 3, then prints the result, which is 5.

Why the other options are wrong:

A (2 + 3) - It does not print the literal text "2 + 3"; the math is evaluated first.

B (23) - "23" would be gluing characters together, which is not what + does with numbers.

C (Nothing, it is an error) - Adding two numbers is perfectly valid, so no error occurs.

Q4 - Correct answer: A (JavaScript is case-sensitive and the name is spelled wrong)

Q: *You type Console.log("hi"); and get an error. Why?*

Why: Names must match exactly: it is console.log with a lowercase c. JavaScript treats capital and small letters as different.

Why the other options are wrong:

B (Semicolons are not allowed there) - A semicolon after a statement is fine and normal.

C (The message is too short) - Message length does not matter; the capital C is the problem.

D (console.log needs two inputs) - console.log works happily with one input, as all our examples show.

Lesson 1.3 - Statements Run in Order

Q1 - Correct answer: B (A, because statements normally run from top to bottom)

Q: *A program has three console.log statements in this order: "A", then "B", then "C". What prints first?*

Why: The normal flow of a JavaScript program is top to bottom, so the first statement in the file runs first.

Why the other options are wrong:

A (All three at exactly the same moment) - Statements run one after another, not simultaneously.

C (C, because the console shows the newest line on top) - Console display order does not change execution order.

D (The order is random each run) - Execution order is not random; it is predictable.

Q2 - Correct answer: A (A semicolon (;))

Q: *Which symbol normally ends a JavaScript statement?*

Why: Statements in this book end with a semicolon, which clearly marks where one instruction stops.

Why the other options are wrong:

B (A period (.)) - A period is used inside decimal numbers like 2.5.

C (A colon (:)) - A colon appears in other places, such as objects in Chapter 9, not at statement ends.

D (An exclamation mark (!)) - An exclamation mark is used in comparisons like !=.

Q3 - Correct answer: C (Nothing at all)

Q: *An error happens on line 3 of a 5-line program. What has run by then?*

Why: Careful reading: statements before the error (lines 1 and 2) already ran; the program stopped at line 3, so lines 4 and 5 never ran.

Why the other options are wrong:

A (Lines 1 and 2 only) - Lines 1 and 2 ran before the error appeared.

B (Lines 1, 2, 4, and 5) - The computer cannot jump over a broken line and keep going.

D (All five lines) - Lines 4 and 5 are after the error, so they never ran.

Q4 - Correct answer: D (Because the computer runs top to bottom and has not reached that part)

Q: *Why can a statement normally not use something the computer has not seen yet?*

Why: Execution flows downward, so on the way down a statement can only rely on what earlier statements already created. (Later techniques can bend this rule, but the flow itself never disappears.)

Why the other options are wrong:

A (Because later lines are always comments) - Later lines are usually real code, not comments.

B (Because the console hides future lines) - The console does not hide anything from the program.

C (Because JavaScript downloads them slowly) - Nothing is downloaded; the rule comes from execution order.

Lesson 1.4 - Comments and Clean Code

Q1 - Correct answer: D (It ignores it completely)

Q: *What does the computer do with a comment?*

Why: Comments are notes for humans; the computer skips them entirely when running the program.

Why the other options are wrong:

A (It prints it on the screen) - Printing is done by console.log(), not comments.

B (It treats it as an error) - Comments are valid code, so they cause no error.

C (It translates it into another language) - No translation happens; comments are simply skipped.

Q2 - Correct answer: A (// note here)

Q: *Which line correctly starts a single-line comment?*

Why: In JavaScript, two forward slashes start a comment that runs to the end of the line.

Why the other options are wrong:

B (note here)** - Double stars belong to multi-line comment endings (*/), not starts.

C (## note here) - Hash symbols start comments in some other languages, not JavaScript.

D (<!-- note here -->) - Angle-bracket dashes are HTML comments, not JavaScript.

Q3 - Correct answer: C (Explaining a longer idea or switching off a block of code while testing)

Q: *What is a good use of a multi-line comment?*

Why: A /* ... */ comment can hold a long explanation, or temporarily disable several lines during testing.

Why the other options are wrong:

A (Making the program run faster) - Comments are ignored, so they cannot change speed.

B (Storing a password safely) - Comments are not security; anyone who can read the code can read the comment.

D (Printing several messages at once) - Only console.log prints; comments never output anything.

Q4 - Correct answer: B (The program prints "Hi" only, because the comment is ignored)

Q: *Read carefully. A student writes this program: // print the greeting console.log("Hi"); Which of the following is TRUE?*

Why: Comments are skipped by the computer, so only the console.log statement does anything: it prints Hi.

Why the other options are wrong:

A (The comment replaces the console.log line, so nothing prints) - Comments do not switch off the lines below them; only their own text is ignored.

C (The program prints "// print the greeting" and then "Hi") - Comments never print; only console.log produces output.

D (The program shows an error because of the comment) - Comments are valid JavaScript, so they never cause errors.

Chapter 2 - Variables and Values

Lesson 2.1 - Labels for Values: let and const

Q1 - Correct answer: A (city let = "Cairo";)

Q: *Which line correctly creates a variable named city with the value "Cairo"?*

Why: The pattern is: keyword, then name, then =, then the value - let city = "Cairo";

Why the other options are wrong:

B (let city = "Cairo";) - Keywords come first; the name cannot go before let.

C (const = "Cairo";) - const needs a name after it before the = sign.

D (let "city" = Cairo;) - The name must not be in quotes, and the string value needs quotes.

Q2 - Correct answer: D (An error)

Q: *What does this program print? let lives = 3; lives = 2; console.log(lives);*

Why: lives starts as 3, then the second assignment replaces it with 2. The print shows the current value: 2.

Why the other options are wrong:

A (32) - Assigning a new value to a let variable is perfectly legal.

B (3) - 3 was replaced before the console.log line ran.

C (2) - Concatenation never happens here; both statements assign plain numbers.

Q3 - Correct answer: D (When the value should never be reassigned)

Q: *When should you choose const instead of let?*

Why: const creates a binding that cannot be reassigned, which is ideal for fixed values.

Why the other options are wrong:

A (When the value is a number) - let handles numbers just as well as const.

B (When the variable is used many times) - How often a variable is read does not matter.

C (When the program is long) - Program length is irrelevant; only reassignment matters.

Q4 - Correct answer: C (JavaScript reports a TypeError and stops)

Q: *What happens if a program assigns a new value to a const?*

Why: Reassigning a const is forbidden; JavaScript raises a TypeError to protect the fixed value.

Why the other options are wrong:

A (The const becomes a let automatically) - The keyword never transforms into another one.

B (The old value is stored twice) - Nothing is duplicated; the assignment is simply refused.

D (The value quietly changes) - Protection is the whole point of const; no quiet change occurs.

Lesson 2.2 - Numbers and Arithmetic

Q1 - Correct answer: A (2)

Q: *What is the value of the expression 17 % 5?*

Why: % gives the remainder after division: 17 divided by 5 is 3 with 2 left over, so 17 % 5 is 2.

Why the other options are wrong:

B (3) - 3 is the quotient, not the remainder.

C (3.4) - 3.4 is the exact decimal result of 17 / 5.

D (12) - 12 is 17 minus 5, which is a subtraction, not a remainder.

Q2 - Correct answer: B (**)

Q: *Which operator raises a number to a power?*

Why: Two asterisks mean exponent: 2 ** 5 is 2 to the power of 5, which is 32.

Why the other options are wrong:

A (%) - % gives the remainder of a division.

C (^) - The caret ^ is a bitwise operator in JavaScript, not power.

D (*) - A single asterisk multiplies: 2 * 5 is 10.

Q3 - Correct answer: B (14)

Q: *What does console.log(2 + 3 * 4); print?*

Why: Multiplication runs before addition: 3 * 4 = 12, then 2 + 12 = 14.

Why the other options are wrong:

A (10) - 10 adds before multiplying - precedence forbids that.

C (20) - 20 would need parentheses around 2 + 3.

D (24) - 24 multiplies everything: that is not the rule.

Q4 - Correct answer: B (16)

Q: *What is the value of 2 ** 3 * 2?*

Why: ** runs before *: 2 ** 3 is 8, then 8 * 2 is 16.

Why the other options are wrong:

A (12) - 12 treats ** like multiplication of 2*3*2 - wrong operator meaning.

C (64) - 64 computes 2 ** 6 or (2*3)*2... neither matches the expression.

D (128) - 128 would be 2 ** 7, not this expression.

Lesson 2.3 - Strings and Concatenation

Q1 - Correct answer: D ("sunflower")

Q: *What does "sun" + "flower" evaluate to?*

Why: With two strings, + concatenates: the pieces join with no space unless you include one.

Why the other options are wrong:

A (An error, because two strings cannot be added) - String concatenation with + is completely legal.

B ("sun flower") - A space appears only if you write "sun " + "flower".

C ("flower sun") - The left piece stays on the left; order is preserved.

Q2 - Correct answer: C ("345")

Q: *What does "3" + 4 + 5 evaluate to?*

Why: Evaluation is left to right: "3" + 4 gives the string "34", then "34" + 5 gives "345".

Why the other options are wrong:

A ("12") - "12" would require numeric addition, which never happens once a string appears.

B (12) - 12 is wrong for the same reason - the leading string makes everything text.

D (An error) - Mixing strings and numbers with + is allowed.

Q3 - Correct answer: A ("75")

Q: *What does 3 + 4 + "5" evaluate to?*

Why: Left to right: 3 + 4 is the number 7, then 7 + "5" concatenates to "75".

Why the other options are wrong:

B ("345") - "345" ignores that the first two values are numbers.

C (12) - 12 forgets that the final + joins with a string.

D ("7" + "5" as an error) - There is no error; number plus string is legal concatenation.

Q4 - Correct answer: B (Page 3)

Q: *What does console.log("Page " + 3); print?*

Why: The number 3 is converted to text and joined after the space: Page 3.

Why the other options are wrong:

A (NaN) - NaN means "not a number" and appears in broken numeric conversions, not here.

C (Page3) - Careful reading: option B "Page3" drops the space that is really inside the string, so it is wrong - the space is part of "Page ".

D ("Page 3" including quotes) - Quotes are syntax; they are never printed.

Lesson 2.4 - Booleans, typeof, and Good Names

Q1 - Correct answer: C ("number")

Q: *What does the expression typeof 42 evaluate to?*

Why: typeof reports the type as a string; JavaScript has one number type, so the answer is "number".

Why the other options are wrong:

A ("digit") - "digit" is not a type name either.

B (42) - typeof returns a type description, never the value itself.

D ("integer") - "integer" is not a JavaScript type name.

Q2 - Correct answer: D (2ndPlayer)

Q: *Which variable name is illegal in JavaScript?*

Why: Names may not start with a digit; 2ndPlayer breaks that rule.

Why the other options are wrong:

A (totalPrice) - totalPrice is perfect camelCase.

B (user2) - Digits are fine as long as they are not first.

C (_hidden) - A leading underscore is legal.

Q3 - Correct answer: A (undefined)

Q: *What is the value stored in a variable that was never created?*

Why: Reading a name that was never declared gives undefined (or an error in strict contexts) - a strong hint of a spelling mistake.

Why the other options are wrong:

B (0) - 0 is a real number that must be assigned on purpose.

C (null) - null is a deliberate "no value" that a programmer assigns.

D (An empty string) - "" is an empty string, also assigned on purpose.

Q4 - Correct answer: A (It reports "object", a famous historical quirk)

Q: *Which statement about typeof null is true?*

Why: For historical reasons, typeof null returns the string "object" - a documented JavaScript quirk.

Why the other options are wrong:

B (It reports "null") - "null" would be tidy, but that is not what the language does.

C (It reports "undefined") - "undefined" is the type of undeclared variables, not null.

D (It causes a SyntaxError) - typeof is a normal operator; it never causes a syntax error.

Chapter 3 - Working with Strings

Lesson 3.1 - Template Literals

Q1 - Correct answer: B (With backticks: `text ${expr}`)

Q: *How do you write a template literal?*

Why: Template literals are wrapped in backticks; only inside them do ${...} placeholders work.

Why the other options are wrong:

A (With asterisks: *text ${expr}*) - Asterisks are not string syntax in JavaScript.

C (With double quotes: "text ${expr}") - Double quotes make a normal string; ${} stays literal text there.

D (With single quotes: 'text ${expr}') - Single quotes are also normal strings.

Q2 - Correct answer: A ("2 + 2 = 4")

Q: *What does `2 + 2 = ${2 + 2}` evaluate to?*

Why: The placeholder is evaluated: 2 + 2 becomes 4, which is inserted into the text.

Why the other options are wrong:

B (An error) - Inside backticks, ${...} is evaluated, so no error occurs.

C ("4") - The whole literal is one string, not just the 4.

D ("2 + 2 = ${2 + 2}") - Placeholders never print their own code.

Q3 - Correct answer: A (They can span multiple lines naturally)

Q: *Which feature do template literals have that quoted strings lack?*

Why: A backtick string keeps the line breaks you type; a quoted string errors if it breaks across lines.

Why the other options are wrong:

B (They run faster) - Speed is not the difference that matters here.

C (They can contain variables without placeholders) - Variables only enter through ${...} placeholders.

D (They ignore spelling mistakes) - No string type fixes spelling.

Q4 - Correct answer: D ("Sum: 7")

Q: *What does `Sum: ${3 + 4}` evaluate to?*

Why: The expression 3 + 4 is computed first, then joined after the text "Sum: ".

Why the other options are wrong:

A ("Sum: 3 + 4") - Placeholders show results, not source code.

B ("Sum: ${3 + 4}") - The ${...} syntax disappears in the final string.

C (7) - The result is the whole sentence, not just the number.

Lesson 3.2 - Escape Sequences

Q1 - Correct answer: A (A on one line and B on the next)

Q: *What does console.log("A\nB"); print?*

Why: \n is the escape sequence for a new line, so the output splits into two lines.

Why the other options are wrong:

B (The single line: A\nB) - The backslash-n is an escape, not literal text.

C (An error) - Escapes are valid syntax - no error here.

D (AB on one line) - The whole point of \n is to break the line.

Q2 - Correct answer: B (\\)

Q: *How do you put a real backslash inside a string?*

Why: One backslash starts an escape sequence, so you write two to get a single backslash character.

Why the other options are wrong:

A (\) - A lone backslash escapes the closing quote and breaks the string.

C (//) - Double slashes start comments, not backslashes.

D (~~) - Tildes are ordinary characters with no special meaning.

Q3 - Correct answer: C (Prints Name: Maya with a tab gap)

Q: *What does "Name:\tMaya" do when printed?*

Why: \t inserts a tab character, aligning the text that follows it.

Why the other options are wrong:

A (Prints the letter t after the colon) - The t is consumed as part of the escape.

B (Removes the colon) - The colon stays exactly where it is.

D (Starts a new line before Maya) - A new line would need \n, not \t.

Q4 - Correct answer: D (console.log("She said \"hi\"");)

Q: *Which line prints: She said "hi" (with double quotes around hi)?*

Why: Inner double quotes must be escaped as \" so they do not end the string early.

Why the other options are wrong:

A (console.log("She said "hi"");) - The inner quotes here end the string early - a SyntaxError.

B (console.log('She said \'hi\');) - This prints hi with single quotes, not double.

C (console.log("She said \hi\");) - A bare \h is not a valid escape, and \" would be needed for the quotes.

Lesson 3.3 - String Methods: Measure and Change Case

Q1 - Correct answer: C (9)

Q: *What is "open door".length?*

Why: Count every character including the space: o-p-e-n (4) + space (1) + d-o-o-r (4) = 9.

Why the other options are wrong:

A (7) - 7 misses two characters.

B (8) - 8 forgets the space or one letter.

D (10) - 10 overcounts.

Q2 - Correct answer: B ("MIXED CASE")

Q: *What does "Mixed Case".toUpperCase() return?*

Why: toUpperCase returns a copy with every letter capitalized.

Why the other options are wrong:

A ("mixed case") - That is the result of toLowerCase (plus losing the capital M).

C ("Mixed Case") - Correct: every letter becomes uppercase.

D (undefined) - The original is unchanged, but a new uppercase string IS returned.

Q3 - Correct answer: A ("red apple")

Q: *What does " red apple ".trim() return?*

Why: trim removes whitespace from both ends only; the inner space remains.

Why the other options are wrong:

A ("redapple") - Inner spaces are never removed by trim.

C (" red apple") - The leading spaces are exactly what trim removes.

D ("red apple ") - The trailing space is removed too.

Q4 - Correct answer: D (2)

Q: *What does "banana".indexOf("na") return?*

Why: Positions count from 0: b=0, a=1, n=2. The first "na" starts at position 2.

Why the other options are wrong:

A (-1) - -1 means the text was not found at all.

B (0) - 0 is where "ba" starts.

C (1) - 1 is where "an" starts.

Lesson 3.4 - Slicing, Replacing, and Immutability

Q1 - Correct answer: C ("cd")

Q: *What does "abcdef".slice(2, 4) return?*

Why: slice copies from position 2 up to but not including position 4: characters c and d.

Why the other options are wrong:

A ("cde") - "cde" would be slice(2, 5).

B ("bc") - "bc" is slice(1, 3).

D ("bcd") - "bcd" would run from 1 to 4.

Q2 - Correct answer: D (true)

Q: *What does "hello world".includes("lo w") return?*

Why: includes checks for the exact text anywhere; "lo w" appears spanning "hello world".

Why the other options are wrong:

A (false) - The text really is present - check the middle: hel-lo- -w-orld.

B ("lo w") - includes returns a boolean, not a string.

C (3) - It never returns a position number.

Q3 - Correct answer: D ("cot")

Q: *What does "cat".replace("a", "o") return?*

Why: replace returns a new string with the first match swapped: "cot".

Why the other options are wrong:

A ("cat") - Without using the result, nothing visibly happens - but the RETURN VALUE is "cot".

B ("cot" and the original becomes "cot") - The original "cat" never changes - strings are immutable.

C (undefined) - replace always returns a string.

Q4 - Correct answer: B (Existing strings never change; methods return new strings)

Q: *Strings in JavaScript are immutable. This means...*

Why: Immutability means the characters of an existing string are fixed; every "change" produces a new string.

Why the other options are wrong:

A (They cannot contain numbers) - Strings can hold any characters, including digits.

C (They cannot be used in variables) - Strings live happily inside variables.

D (They must stay under 26 characters) - There is no length limit like that.

Chapter 4 - Numbers and Math

Lesson 4.1 - The Math Object: Rounding and Powers

Q1 - Correct answer: D (8)

Q: *What does Math.round(7.5) return?*

Why: round goes to the nearest whole number, and halves round up: 7.5 becomes 8.

Why the other options are wrong:

A (0.75) - 0.75 is unrelated arithmetic.

B (7) - 7 would be rounding down.

C (7.5) - Correct: nearest whole with halves up.

Q2 - Correct answer: B (-4)

Q: *What does Math.floor(-3.2) return?*

Why: floor rounds DOWN toward negative infinity, so -3.2 goes to -4.

Why the other options are wrong:

A (3) - floor always returns a whole number.

C (-3.2) - floor never leaves decimals.

D (-3) - -3 is truncation, not flooring.

Q3 - Correct answer: A (Math.trunc)

Q: *Which function simply chops off the decimal part?*

Why: trunc removes the decimals, moving toward zero: trunc(4.9) is 4 and trunc(-4.9) is -4.

Why the other options are wrong:

B (Math.round) - round picks the nearest whole number.

C (Math.floor) - floor moves down, which differs from chopping for negatives.

D (Math.ceil) - ceil moves up.

Q4 - Correct answer: A (12)

Q: *What does Math.sqrt(144) return?*

Why: sqrt gives the square root: 12 times 12 is 144.

Why the other options are wrong:

B (14) - 14 is close but 14*14 is 196.

C (24) - 24 is double the answer.

D (72) - 72 divides by 12, not the operation asked.

Lesson 4.2 - Limits, Precision, and toFixed

Q1 - Correct answer: C (5)

Q: *What does Math.max(3, -8, 5) return?*

Why: max answers the largest argument: among 3, -8, and 5, the largest is 5.

Why the other options are wrong:

A (-8) - -8 is the smallest, that is min.

B (3) - 3 is neither largest nor smallest here.

D (16) - 16 adds them; max never sums.

Q2 - Correct answer: D (false)

Q: *What does console.log(0.1 + 0.2 === 0.3); print?*

Why: Binary floating point stores 0.1 and 0.2 imperfectly, so their sum is 0.30000000000000004 - not exactly 0.3.

Why the other options are wrong:

A (true) - Tempting, but the tiny storage error makes it false.

B (0.3) - The expression is a comparison; it never prints 0.3 itself.

C (NaN) - Nothing here is NaN.

Q3 - Correct answer: C ("3.14" as a string)

Q: *What does (3.14159).toFixed(2) return?*

Why: toFixed rounds to 2 decimals and returns the result as a STRING.

Why the other options are wrong:

A ("3.15") - Rounding 3.14159 to two places is 3.14, not 3.15.

B (3.1) - One decimal place would be "3.1".

D (3.14 as a number) - toFixed returns text, not a number.

Q4 - Correct answer: A (2)

Q: *What does Math.min(10, 2, 7) return?*

Why: min answers the smallest argument: 2.

Why the other options are wrong:

B (7) - 7 is in the middle.

C (10) - 10 is the largest - that is max.

D (19) - 19 would be the sum; min never adds.

Lesson 4.3 - Random Numbers for Games

Q1 - Correct answer: B (From 0 up to (but never including) 1)

Q: *What is the range of Math.random()?*

Why: Math.random() returns a decimal in [0, 1): zero is possible, one never is.

Why the other options are wrong:

A (From 1 to 10) - 1 to 10 needs multiplying and shifting.

C (Any whole number) - Whole numbers need Math.floor on top.

D (From -1 to 1) - The range never goes negative.

Q2 - Correct answer: B (A number from 0 to 5)

Q: *What does Math.floor(Math.random() * 6) return?*

Why: random()*6 lands in [0, 6); floor makes it a whole number from 0 to 5.

Why the other options are wrong:

A (Always 0) - Only 0 if random() returned exactly 0.

C (A number from 1 to 6) - 1 to 6 would need the +1 shift.

D (A number from 0 to 6) - 6 itself is never reached by random()*6.

Q3 - Correct answer: A (Math.floor(Math.random() * 6) + 1)

Q: *How do you make a fair die roll from 1 to 6?*

Why: The stretch-floor-shift pattern: six values 0-5, then +1 shifts to 1-6.

Why the other options are wrong:

B (Math.random(6)) - random() takes no range arguments.

C (Math.round(Math.random() * 6)) - round makes 0 and 6 half as likely - a biased die.

D (Math.floor(Math.random()) + 6) - floor(random()) is always 0, so this always gives 7.

Q4 - Correct answer: C (How many different values the roll can be)

Q: *In the recipe Math.floor(Math.random() * (max - min + 1)) + min, what does (max - min + 1) count?*

Why: It is the SIZE of the range: from min to max there are max - min + 1 distinct values.

Why the other options are wrong:

A (The smallest possible result) - The smallest result is min.

B (How many times to call random) - random is called exactly once.

D (The largest possible result) - The largest result is max, produced after the + min shift.

Lesson 4.4 - Converting Text to Numbers (and NaN)

Q1 - Correct answer: D (The number 42)

Q: *What does Number("42") return?*

Why: Number converts purely numeric text into a real number.

Why the other options are wrong:

A ("42" as a string) - The quotes disappear - the result is a number.

B (NaN) - NaN appears only when conversion fails.

C (undefined) - A successful conversion never gives undefined.

Q2 - Correct answer: D (18)

Q: *What does parseInt("18 years old") return?*

Why: parseInt reads the leading whole number and ignores trailing non-numeric text.

Why the other options are wrong:

A (NaN) - Conversion succeeds because digits come first.

B ("18 years old") - The result is a number, not the original text.

C (0) - Digits at the start are read, not skipped.

Q3 - Correct answer: B (NaN)

Q: *What does Number("hello") return?*

Why: Non-numeric text converts to NaN - Not a Number.

Why the other options are wrong:

A (An error stops the program) - NaN is a normal return value; the program keeps running.

C (0) - 0 is not used for failed conversions.

D ("hello") - NaN is a special value, not the original text.

Q4 - Correct answer: C (Number.isNaN(value))

Q: *Which expression reliably detects NaN?*

Why: NaN is not equal to anything, even itself, so the built-in Number.isNaN() is the reliable test.

Why the other options are wrong:

A (value == "NaN") - Comparison with the text "NaN" tests text, not the value.

B (typeof value == "nan") - typeof NaN is "number" anyway.

D (value === NaN) - === with NaN is always false, even for NaN.

Chapter 5 - Making Decisions

Lesson 5.1 - Comparisons Produce Booleans

Q1 - Correct answer: B (true)

Q: *What does the expression 7 > 3 evaluate to?*

Why: Every comparison produces a boolean, and 7 really is greater than 3.

Why the other options are wrong:

A (7) - The operands never become the result.

C ("yes") - Correct: comparisons answer true or false.

D (3) - Text like "yes" is not what comparisons return.

Q2 - Correct answer: C (===)

Q: *Which operator asks "are these two values exactly equal"?*

Why: === is the strict equality test this book uses everywhere.

Why the other options are wrong:

A (=) - A single = assigns; it does not compare.

B (==) - Loose == converts types first and is avoided.

D (=>) - => declares arrow functions in Chapter 7.

Q3 - Correct answer: B (false)

Q: *What does "5" === 5 evaluate to?*

Why: Strict equality never converts types: text "5" and the number 5 are different values.

Why the other options are wrong:

A (true) - That would be the loose == answer.

C (5) - Comparisons never return an operand.

D (An error) - It is a legal expression - no error.

Q4 - Correct answer: C (true, because <= allows equality)

Q: *What does 4 <= 4 evaluate to?*

Why: The <= operator is true when the left side is less than OR exactly equal to the right side.

Why the other options are wrong:

A (4) - The result is boolean, not a number.

B (undefined) - A valid comparison never gives undefined.

D (false, because 4 is not less than 4) - The "or equal" half makes it true.

Lesson 5.2 - if, else, and else if

Q1 - Correct answer: A (When the if condition is false)

Q: *When does an else block run?*

Why: else is the fallback path: it runs exactly when the if condition fails.

Why the other options are wrong:

B (When the if condition is true) - Then the if block runs and else is skipped.

C (Always, right after the if block) - Only one of the two ever runs.

D (Only when the if block printed something) - Printing has no effect on branching.

Q2 - Correct answer: D (Exactly one - the first whose condition is true)

Q: *In an if / else if / else chain, how many blocks run?*

Why: The chain is first-match branching: the first true condition wins and the rest are skipped.

Why the other options are wrong:

A (All whose conditions are true) - Only the first match runs, never several.

B (Always the last one) - The last block (else) runs only if nothing matched.

C (At least two) - Zero blocks run only if there is no else and no condition matched.

Q3 - Correct answer: A (big)

Q: *What does this print? let n = 15; if (n > 10) { console.log("big"); } else { console.log("small"); }*

Why: 15 > 10 is true, so the if block runs and else is skipped.

Why the other options are wrong:

B (small) - The condition is true, so else never runs.

C (big then small) - Both blocks never run together.

D (Nothing) - One of the two blocks always runs.

Q4 - Correct answer: C (Because a general condition earlier would swallow the cases meant for later branches)

Q: *Why should the most specific conditions come first in an else-if chain?*

Why: First-match branching stops at the first true condition, so an early broad check would capture cases meant for later branches.

Why the other options are wrong:

A (Because JavaScript requires alphabetical order) - No such requirement exists.

B (It makes the program run faster) - Speed is unaffected.

D (It looks nicer) - Style is a side effect, not the reason.

Lesson 5.3 - Logical Operators: &&, ||, !

Q1 - Correct answer: B (false)

Q: *What does true && false evaluate to?*

Why: && needs BOTH sides true; one false makes the whole thing false.

Why the other options are wrong:

A (true) - && is strict: any false dooms the result.

C (undefined) - No third value appears here.

D (Both values) - && returns a single boolean result.

Q2 - Correct answer: D (false)

Q: *What does !(5 > 2) evaluate to?*

Why: 5 > 2 is true, and ! flips it to false.

Why the other options are wrong:

A (true) - The flip inverts the inner truth.

B (5) - ! returns a boolean, not the operand.

C (An error) - Flipping a comparison is legal.

Q3 - Correct answer: D (checkSomething() never runs, because the answer is already false)

Q: *What does short-circuit evaluation mean for the expression false && checkSomething()?*

Why: && stops as soon as the left side is false - the right side cannot change the outcome.

Why the other options are wrong:

A (checkSomething() runs twice) - Nothing runs twice.

B (Both sides always run) - Short-circuiting exists precisely to skip.

C (The expression is an error) - It is a normal, safe expression.

Q4 - Correct answer: B (Short-circuiting skips parts of an expression; first-match branching picks one code path in an if / else if chain)

Q: *Which sentence best distinguishes short-circuiting from first-match branching?*

Why: One is an expression-evaluation rule, the other is the if / else if chain's pick-one-path behavior.

Why the other options are wrong:

A (They are two names for the same idea) - They operate at different levels.

C (Branching skips parts of an expression; short-circuiting picks a code path) - That swaps the two definitions.

D (Short-circuiting only works inside switch) - switch is unrelated to short-circuiting.

Lesson 5.4 - switch: Choosing Among Many Options

Q1 - Correct answer: A (Strict === comparison)

Q: *What does a switch case match using?*

Why: switch compares the switch value to each case label with strict equality.

Why the other options are wrong:

B (Loose == comparison) - No type conversion happens.

C (Greater-than checks) - switch cannot test ranges.

D (Alphabetical order) - Labels are tested by value, not alphabetically.

Q2 - Correct answer: C (Execution falls through into the next case)

Q: *What happens if you forget break at the end of a case?*

Why: Without break, execution continues into the following case's statements.

Why the other options are wrong:

A (The switch restarts from the top) - The switch never restarts.

B (The default case runs) - default runs only when nothing matched.

D (An error stops the program) - Fall-through is silent, not an error.

Q3 - Correct answer: D (It runs when no case label matched)

Q: *What is the role of default in a switch?*

Why: default is the catch-all, like the final else.

Why the other options are wrong:

A (It runs first, before any case) - Cases are jumped to by match, not by position.

B (It replaces the need for break) - break still ends each case block.

C (It is required in every switch) - default is optional.

Q4 - Correct answer: B (Comparing a menu number against choices 1, 2, 3, and 4)

Q: *For which task is switch the BEST tool?*

Why: switch matches one value against fixed labels - exactly the menu case.

Why the other options are wrong:

A (Checking whether score is at least 90) - Ranges need if / else if; switch cannot test "at least".

C (Testing two different variables at once) - switch matches one value; multi-variable tests need conditions.

D (Repeating code ten times) - Repetition is looping, Chapter 6.

Chapter 6 - Loops

Lesson 6.1 - The while Loop

Q1 - Correct answer: A (Start value, condition, update)

Q: *What are the three essential parts of every while loop?*

Why: The loop needs something to test (the counter), the condition itself, and an update that moves toward the exit.

Why the other options are wrong:

B (Header, footer, margin) - That is page layout, not code.

C (A function, an array, a string) - Those are Chapter 7 and 8 topics, not loop parts.

D (if, else, break) - if/else belong to branching, not the loop anatomy.

Q2 - Correct answer: B (The loop never ends - an infinite loop)

Q: *What happens if the update line is missing from a while loop?*

Why: Nothing changes the tested value, so the condition stays true forever.

Why the other options are wrong:

A (JavaScript inserts the update for you) - The language never fixes your logic.

C (The loop runs once) - The body would repeat, not run once.

D (The condition becomes false automatically) - Conditions do not change by themselves.

Q3 - Correct answer: C (Before every round, and once before the first)

Q: *When is the while condition checked?*

Why: The check happens at the top of each round, including before the very first one.

Why the other options are wrong:

A (Only once at the start) - It is re-checked every round.

B (Never - while loops run blind) - The condition is the loop's steering wheel.

D (After every round only) - The check leads, it does not follow.

Q4 - Correct answer: A (3 and 2)

Q: *What does this loop print? let n = 3; while (n > 1) { console.log(n); n = n - 1; }*

Why: Rounds run while n > 1: n is 3 (print, becomes 2), then 2 (print, becomes 1); then 1 > 1 is false.

Why the other options are wrong:

B (3, 2, and 1) - Correct: two rounds, values 3 and 2.

C (Only 3) - The loop repeats, not a single pass.

D (Nothing) - The condition is true at first, so it prints.

Lesson 6.2 - The for Loop

Q1 - Correct answer: D (4)

Q: *How many times does for (let i = 0; i < 4; i++) run its body?*

Why: i takes the values 0, 1, 2, 3 - four rounds - before 4 < 4 fails.

Why the other options are wrong:

A (3) - That would need i < 3.

B (5) - Five needs i <= 4.

C (Infinite) - The update moves i toward the boundary.

Q2 - Correct answer: B (Is checked before every round)

Q: *In a for header, what does the middle clause do?*

Why: The middle clause is the condition, tested at the top of each round.

Why the other options are wrong:

A (Names the loop variable) - The variable is declared in the start clause.

C (Runs once before the loop) - That is the first (start) clause.

D (Runs after every round) - That is the third (update) clause.

Q3 - Correct answer: A (Add 1 to i)

Q: *What does i++ mean?*

Why: i++ is the classic shorthand for i = i + 1.

Why the other options are wrong:

B (Add 2 to i) - Adding 2 would be i += 2 or i = i + 2.

C (Multiply i by itself) - Squaring is i * i.

D (Compare i to 1) - Comparison uses === or <=.

Q4 - Correct answer: C (2 4 6)

Q: *What does this loop print? for (let k = 2; k <= 6; k + = 2) { console.log(k); } (spaces removed)*

Why: k steps by 2: 2, 4, 6 - then 8 > 6 ends the loop.

Why the other options are wrong:

A (6) - The loop repeats, not one pass.

B (2 2 2 2 forever) - The update always advances k.

D (2 3 4 5 6) - Correct: three values, two apart.

Lesson 6.3 - break and continue

Q1 - Correct answer: D (Ends the loop immediately)

Q: *What does break do inside a loop?*

Why: break exits the whole loop, continuing the program after it.

Why the other options are wrong:

A (Skips to the next round) - That is continue.

B (Restarts the loop from round one) - Restarting is the opposite of leaving.

C (Pauses the loop for one second) - Pausing is not a loop keyword.

Q2 - Correct answer: B (Skips the rest of the current round only)

Q: *What does continue do inside a loop?*

Why: continue jumps back to the top and starts the next round.

Why the other options are wrong:

A (Repeats the current round forever) - The next round begins normally.

C (Ends the loop) - That is break.

D (Restarts the computer) - Computers are not restarted by keywords.

Q3 - Correct answer: A (1 2)

Q: *What does this print? for (let i = 1; i <= 5; i++) { if (i === 3) { break; } console.log(i); }*

Why: Rounds 1 and 2 print; on round 3 the break fires before the print, ending the loop.

Why the other options are wrong:

B (1 2 3) - The break happens before 3 is printed.

C (1 2 4 5) - continue would give 1 2 4 5, but this is break.

D (3) - 3 itself never prints.

Q4 - Correct answer: D (It contains a break that will eventually run)

Q: *An endless while (true) loop is safe to use only if...*

Why: The only exit from while (true) is a break - the loop must contain one that the data reaches.

Why the other options are wrong:

A (The body is short) - Body length does not end loops.

B (The computer is fast) - Speed cannot end a logic loop.

C (It uses for-style counters) - Counters do not matter to while (true).

Lesson 6.4 - Nested Loops and Patterns

Q1 - Correct answer: B (20)

Q: *An outer loop runs 4 times; its inner loop runs 5 times each round. How many inner-body executions?*

Why: Nesting multiplies: 4 rounds times 5 inner rounds = 20 executions.

Why the other options are wrong:

A (9) - 9 adds instead of multiplying.

C (45) - 45 is a triangular count, not this case.

D (54) - 54 is 4 + 5 flipped and scaled - no.

Q2 - Correct answer: C (Because each row needs a fresh, empty line to build on)

Q: *In the triangle pattern lesson, why is line = "" reset inside the outer loop?*

Why: Every row starts from an empty string; forgetting the reset makes later rows inherit earlier stars.

Why the other options are wrong:

A (Because JavaScript requires it after console.log) - No such requirement exists.

B (It is not reset - it keeps growing) - The lesson explicitly resets it each round.

D (To make the program faster) - Speed is unaffected.

Q3 - Correct answer: C (Both loops scramble the same counter and the counts go wrong)

Q: *What is wrong with sharing variable i between an outer and an inner for loop?*

Why: The inner loop's updates stomp on the outer loop's counter, corrupting both counts.

Why the other options are wrong:

A (The inner loop cannot see i) - The inner loop sees it all too well.

B (It causes a SyntaxError) - It is legal syntax with wrong results.

D (Nothing - it saves memory) - It definitely changes behavior.

Q4 - Correct answer: B (Row 2: **)**

Q: *What does the 3x4 grid example print on Row 2?*

Why: Every row builds a fresh line of exactly 4 stars.

Why the other options are wrong:

A (Row 2: ****)** - Six stars would need a 6-column grid.

C (Row 2: *) - One star is just the first column.

D (Row 2: **) - Two stars is row 2 of the triangle, not the grid.

Chapter 7 - Functions

Lesson 7.1 - Defining and Calling Functions

Q1 - Correct answer: C (Teaches JavaScript a named skill without running it yet)

Q: *What does defining a function do?*

Why: A definition only registers the code under a name; the body runs when the function is called.

Why the other options are wrong:

A (Deletes old variables) - Definitions create no deletions.

B (Prints the function name) - Nothing prints at definition time.

D (Runs its code immediately) - Only a call runs the body.

Q2 - Correct answer: A (cheer();)

Q: *Which line correctly CALLS a function named cheer?*

Why: A call is the name followed by parentheses, optionally with arguments inside.

Why the other options are wrong:

B (cheer;) - Without parentheses the name refers to the function but nothing runs.

C (function cheer;) - That syntax defines, not calls.

D (call cheer) - There is no call keyword in JavaScript.

Q3 - Correct answer: C (printReport)

Q: *What is the best name for a function that prints a report?*

Why: Verb-phrase camelCase names say what happens: printReport.

Why the other options are wrong:

A (the_function) - Legal but vague.

B (ReportPrint() in CAPS) - Shouting caps is not a JavaScript convention.

D (data2) - Cryptic names hide the purpose.

Q4 - Correct answer: B (Nothing)

Q: *A program defines cheer() but never calls it. What is visible on screen?*

Why: Definitions are silent; without a call, no code runs and nothing appears.

Why the other options are wrong:

A (The function body) - The body stays hidden until called.

C (The word cheer) - Referring to a name prints nothing.

D (An error) - An unused definition is perfectly legal.

Lesson 7.2 - Parameters, Arguments, and return

Q1 - Correct answer: D (parameter)

Q: *In function greet(name) { ... }, the word name is a...*

Why: Parameters are the named slots declared in the definition.

Why the other options are wrong:

A (global variable) - It is local to the function, not global.

B (return value) - The return value is what comes OUT.

C (comment) - It is live code, not a comment.

Q2 - Correct answer: A (Sends a value back to the caller and ends the function)

Q: *What does return do?*

Why: return hands the value to the calling code and stops the function body immediately.

Why the other options are wrong:

B (Prints a value for the user) - Printing is console.log's job.

C (Loops the function again) - Loops are loop keywords' job.

D (Deletes the function) - Functions are never deleted by return.

Q3 - Correct answer: C (15)

Q: *What does console.log(double(7) + 1); print if double returns n * 2?*

Why: double(7) is 14, then + 1 makes 15 - the returned value flows into the bigger expression.

Why the other options are wrong:

A (8) - 8 would be doubling 4.

B (An error) - Returning a number fits perfectly into arithmetic.

D (14) - 14 forgets the + 1.

Q4 - Correct answer: B (word is "Hi" and times is 2)

Q: *For repeatWord(word, times), what happens with the call repeatWord("Hi", 2);?*

Why: Arguments match parameters by position: first to first, second to second.

Why the other options are wrong:

A (word is 2 and times is "Hi") - Order swapping is the classic mistake.

C (An error - too few arguments) - Two arguments fit two parameters.

D (The function must return something) - Printing functions need no return.

Lesson 7.3 - Scope and Arrow Functions

Q1 - Correct answer: D (local to that function)

Q: *A variable declared with let INSIDE a function is...*

Why: Function scope: the variable lives and dies inside the function.

Why the other options are wrong:

A (global) - Only outer declarations are global.

B (visible to the whole file) - The whole file sees only globals.

C (permanent) - Locals vanish when the function ends.

Q2 - Correct answer: D (Yes, globals are visible inside functions)

Q: *Can a function READ a global variable?*

Why: Scope flows inward: inner code can see outer names.

Why the other options are wrong:

A (No, never) - Reading globals is normal and common.

B (Only if passed as a parameter) - Passing parameters is safer style, not a requirement for reading.

C (Only in arrow functions) - All functions share the same scope rules.

Q3 - Correct answer: C (Locals prevent distant functions from accidentally interfering with each other)

Q: *Why do programmers prefer locals over globals?*

Why: Narrow scope limits who can change a value, which prevents far-away bugs.

Why the other options are wrong:

A (Globals are illegal in modern JavaScript) - Globals are legal, just risky.

B (Locals use fewer characters) - Length is irrelevant.

D (Locals run faster) - Speed is not the deciding factor.

Q4 - Correct answer: B (const inc = (n) => n + 1;)

Q: *Which line defines an arrow function that returns n plus 1?*

Why: The arrow syntax is (parameters) => body.

Why the other options are wrong:

A (function => inc(n)) - Reversed syntax does not exist.

C (arrow inc(n) { return n + 1; }) - There is no arrow keyword.

D (const inc = n => n + 1;) - Correct too! Both A and C are valid arrow forms - A with explicit parentheses around one parameter, C without. The book shows A first.

Lesson 7.4 - Documenting and Defending: JSDoc and try/catch

Q1 - Correct answer: A (/)**

Q: *What does a JSDoc comment start with?*

Why: JSDoc uses the extra-star opener /** so tools recognize it as documentation.

Why the other options are wrong:

B (//) - Double slashes make single-line comments.

C (/*) - Plain /* is a normal multi-line comment.

D (###) - Hashes are not JavaScript comment syntax.

Q2 - Correct answer: C (Execution jumps to the catch block)

Q: *What happens when an error is thrown inside a try block?*

Why: The catch block receives the error object and gets a chance to respond.

Why the other options are wrong:

A (The try block restarts) - No automatic retry exists.

B (The error is printed twice) - One error, one catch.

D (The program crashes immediately) - That is exactly what catch prevents.

Q3 - Correct answer: D (Running cleanup code that normally executes whether or not an error occurred)

Q: *What is the finally block for?*

Why: finally normally runs after try and/or catch - ideal for cleanup like closing files.

Why the other options are wrong:

A (Making the program faster) - Speed is not its purpose.

B (Repeating the try block) - Nothing repeats automatically.

C (Declaring the end of the file) - finally is about flow, not files.

Q4 - Correct answer: B (Because the program cannot start running until the syntax is valid)

Q: *Why does try / catch NOT rescue a SyntaxError like a missing brace?*

Why: Syntax is checked before execution; a broken program has no running code to catch anything.

Why the other options are wrong:

A (SyntaxErrors are too large to catch) - Size has nothing to do with it.

C (Because braces are optional) - Braces are required for blocks.

D (catch ignores syntax problems by design) - catch handles runtime errors only.

Chapter 8 - Arrays

Lesson 8.1 - Creating Arrays and Reading Items

Q1 - Correct answer: D (0)

Q: *What is the index of the FIRST item in an array?*

Why: Array counting starts at zero: the first item sits at index 0.

Why the other options are wrong:

A (1) - Humans count from 1, computers from 0.

B (It depends on the array) - The rule never changes.

C (-1) - -1 signals "not found" in searches.

Q2 - Correct answer: C (3)

Q: *For let a = [10, 20, 30];, what does a.length return?*

Why: The array holds three items, so length is 3 - one more than the last index (2).

Why the other options are wrong:

A (31) - That confuses values with count.

B (undefined) - A real array always has a length.

D (2) - 2 is the last INDEX.

Q3 - Correct answer: B (undefined)

Q: *What does a[99] return for the 3-item array above?*

Why: Out-of-range reads politely return undefined - a silent signal of index trouble.

Why the other options are wrong:

A (0) - Zero is a real value, not a miss.

C (An error stops the program) - No crash for reading too far.

D (30) - 30 lives at index 2.

Q4 - Correct answer: A (fruits[1] = "kiwi";)

Q: *Which statement changes the second item to "kiwi"?*

Why: The second item lives at index 1, and assignment to that slot replaces it.

Why the other options are wrong:

B (fruits(1) = "kiwi";) - Parentheses call functions; indexes use brackets.

C (fruits[2] = "kiwi";) - Index 2 is the THIRD item.

D (fruits = "kiwi";) - That replaces the whole array with text.

Lesson 8.2 - Adding and Removing: push, pop, shift, unshift

Q1 - Correct answer: C (push())

Q: *Which method adds an item to the END of an array?*

Why: push grows the array at the back end.

Why the other options are wrong:

A (shift()) - shift removes from the front.

B (unshift()) - unshift adds to the front.

D (pop()) - pop removes from the end.

Q2 - Correct answer: D (The new length)

Q: *What does list.push("x") RETURN?*

Why: push and unshift return the new length; pop and shift return the removed item.

Why the other options are wrong:

A (The item "x") - That is pop's return.

B (The array) - push does return the array in some languages, but not JavaScript - here it is a count.

C (undefined) - It always returns a number.

Q3 - Correct answer: A ([2, 3, 4])

Q: *After these lines, what is in list? let list = [1, 2, 3]; list.shift(); list.push(4);*

Why: shift removes the front (1), push adds 4 at the end: [2, 3, 4].

Why the other options are wrong:

B ([1, 2, 3, 4]) - shift definitely removed the 1.

C ([2, 3]) - The push is missing from this answer.

D ([4, 2, 3]) - unshift would put it at the front; push appends.

Q4 - Correct answer: B (Without a comparator, numbers are compared as TEXT, so "10" comes before "2")

Q: *Why does [10, 2, 33].sort() NOT give [2, 10, 33]?*

Why: Default sort converts items to text and orders them alphabetically.

Why the other options are wrong:

A (sort() refuses to order numbers) - It does sort - just text-style.

C (sort() only works on strings) - It works on anything - the comparator decides the order.

D (The array is too short to sort) - Size is irrelevant.

Lesson 8.3 - Walking Through Arrays

Q1 - Correct answer: C (for (let i = 0; i < list.length; i++))

Q: *Which loop header visits every index of list safely?*

Why: Indexes run 0 to length - 1, so the strict < keeps every read in range.

Why the other options are wrong:

A (for (let i = 0; i <= list.length; i++)) - The <= reads one index too far: undefined.

B (for (let i = 0; i <= list.length - 2; i++)) - Misses the last index.

D (for (let i = 1; i <= list.length; i++)) - Starts at 1 and overruns the end.

Q2 - Correct answer: B (The value only)

Q: *What does for (let item of list) give you each round?*

Why: for...of visits values directly, no index involved.

Why the other options are wrong:

A (The index only) - That is a counted loop or entries().

C (Index and value) - Pairs come from entries().

D (The length) - length is a property, not a loop yield.

Q3 - Correct answer: D (i starts at 0 for the computer but humans number lists from 1)

Q: *In the teams example, why print (i + 1) instead of i?*

Why: The display offset converts computer counting to human counting.

Why the other options are wrong:

A (i is a string) - i is a number.

B (console.log requires it) - console.log accepts any value.

C (To make the list longer) - The offset changes labels, not the list.

Q4 - Correct answer: A (Each index and its value together)

Q: *What does for (let [i, t] of teams.entries()) deliver?*

Why: entries() pairs each index with its item; the brackets unpack the pair.

Why the other options are wrong:

B (Only the values) - That is plain for...of.

C (Only the indexes) - Half of the pair.

D (The array reversed) - Reversing is reverse().

Lesson 8.4 - Searching, Joining, and Slicing

Q1 - Correct answer: D ("a-b-c")

Q: *What does ["a", "b", "c"].join("-") return?*

Why: join glues items into one string with the separator between each pair.

Why the other options are wrong:

A ("abc") - A separator was supplied, so it must appear.

B (["a", "-", "b"]) - join returns a string, not an array.

C (3) - It returns text, not a count.

Q2 - Correct answer: A (-1)

Q: *What does pets.indexOf("bird") return when bird is not in the array?*

Why: -1 is the universal "not found" answer from indexOf.

Why the other options are wrong:

B (0) - 0 would mean bird is the first item.

C (true) - includes returns booleans, not indexOf.

D (undefined) - undefined belongs to missing indexes, not failed searches.

Q3 - Correct answer: B (["b", "c"])

Q: *What does letters.slice(1, 3) return for ["a", "b", "c", "d"]?*

Why: slice copies from index 1 up to but not including 3: items b and c.

Why the other options are wrong:

A (["a", "b"]) - That starts at 0.

C (["b", "c", "d"]) - The end is exclusive, so d stays out.

D (["c"]) - That would be slice(2, 3).

Q4 - Correct answer: C (sort modifies the original; slice first protects it)

Q: *Why write list.slice().sort(...) instead of list.sort(...)?*

Why: sort sorts IN PLACE; slicing first leaves the original untouched.

Why the other options are wrong:

A (sort cannot run on the original array) - It can - that is the danger.

B (slice makes the sort numeric) - The comparator makes it numeric.

D (slice is faster) - Speed is not the point.

Lesson 8.5 - Transforming Arrays: map, filter, reduce, splice

Q1 - Correct answer: D (big is a new array; nums is unchanged)

Q: *After let big = nums.map((n) => n * 10);, what is true?*

Why: map builds a NEW array from the returned values and leaves the original array untouched.

Why the other options are wrong:

A (nums has been replaced by bigger numbers) - Correct: a new array, original safe.

B (big and nums are the same array) - They are two separate arrays.

C (map returns a single number) - That is reduce - map returns an array.

Q2 - Correct answer: C (Only items where the function returns true)

Q: *Which items does filter keep?*

Why: filter calls your function once per item and keeps exactly the items for which it returned true.

Why the other options are wrong:

A (Every item in the array) - That would make selecting pointless.

B (Only the first and last items) - Position never matters to filter.

D (Only items where the function returns false) - It is the opposite of that.

Q3 - Correct answer: A (42)

Q: *What does [10, 25, 7].reduce((acc, n) => acc + n, 0); return?*

Why: reduce adds every item onto the accumulator starting at 0 and finishes with one number: 42.

Why the other options are wrong:

B ([10, 25, 7]) - reduce does not return the list.

C ([42]) - No array comes out of reduce here.

D (0) - 0 is only the starting accumulator.

Q4 - Correct answer: B (splice())

Q: *Which method REMOVES items from the original array in place?*

Why: splice cuts items out of the original array and returns what it removed.

Why the other options are wrong:

A (map()) - map returns a new transformed array.

C (slice()) - slice copies a section and changes nothing.

D (concat()) - concat builds a new combined array.

Chapter 9 - Objects

Lesson 9.1 - Properties: Dot and Bracket Access

Q1 - Correct answer: A (let x = { name: "Maya" };)

Q: *Which syntax creates an object?*

Why: Objects use curly braces with key: value pairs.

Why the other options are wrong:

B (let x = (name: "Maya");) - Parentheses are for grouping and calls.

C (let x = [name: "Maya"];) - Square brackets make arrays.

D (let x = <name: "Maya">;) - Angle brackets are not JavaScript syntax.

Q2 - Correct answer: D (student.age)

Q: *How do you read the age property of student?*

Why: Dot access: objectName.propertyName.

Why the other options are wrong:

A (student->age) - Arrow syntax belongs to other languages.

B (student(age)) - Parentheses call functions.

C (age.student) - Property and object are in reverse.

Q3 - Correct answer: B (undefined)

Q: *What does student.city return when city was never added to the object?*

Why: Missing properties read as undefined, just like missing array indexes.

Why the other options are wrong:

A (null) - null must be assigned on purpose.

C (An error) - Reading a missing key is silent, not fatal.

D ("") - An empty string is a real value that was never stored.

Q4 - Correct answer: C (When the key name comes from a variable at runtime)

Q: *When is bracket access REQUIRED instead of the dot?*

Why: student[key] uses the variable's VALUE as the key; the dot only works with names written literally.

Why the other options are wrong:

A (When the value is a number) - Value types do not matter.

B (When the object has many properties) - Size is irrelevant.

D (Never - dots always work) - Brackets exist for a reason.

Lesson 9.2 - Arrays of Objects: Real Records

Q1 - Correct answer: D (The name property of the FIRST record)

Q: *What does roster[0].name mean for an array of student objects?*

Why: roster[0] picks the first object; .name reaches inside it.

Why the other options are wrong:

A (The name key of the whole array) - Arrays have no name property here.

B (An array of all names) - Collecting all names needs a loop.

C (A syntax error) - Index-then-dot is perfectly legal.

Q2 - Correct answer: C (Its keys)

Q: *What does for...in visit on an object?*

Why: for...in hands you each key name; read values with object[key].

Why the other options are wrong:

A (Its methods only) - Methods are properties too, if present.

B (Its length) - Objects have no length.

D (Its values) - Values need object[key] inside the loop.

Q3 - Correct answer: A (student[key])

Q: *Inside for (let key in student), how do you read each value?*

Why: The key is a variable, so bracket access is required.

Why the other options are wrong:

B (student.key) - The dot would look for a property literally named "key".

C (student(key)) - Parentheses call functions.

D (key.student) - Objects do not own the loop variable.

Q4 - Correct answer: B (class is a reserved word and cannot be a variable name)

Q: *Why did the lesson name its array class_ instead of class?*

Why: Reserved words cannot be variable names; the underscore dodges the clash.

Why the other options are wrong:

A (Underscores are faster) - Naming does not affect speed.

C (To make the array private) - Privacy needs different tools.

D (It is a spelling mistake) - The underscore is deliberate.

Lesson 9.3 - Objects with Methods

Q1 - Correct answer: C (A function stored as an object property)

Q: *What is a method?*

Why: Methods give objects their own behavior.

Why the other options are wrong:

A (A loop over keys) - That is for...in.

B (A JSON rule) - JSON formats text, not methods.

D (Any variable in an object) - Plain properties hold data.

Q2 - Correct answer: A (The object the method was called on)

Q: *Inside a regular method, what does this refer to?*

Why: this points at the owner of the call: player.describe() gives this = player.

Why the other options are wrong:

B (The global scope) - Regular methods are not global.

C (The function itself) - Functions are values, not this.

D (The previous method) - There is no chain here.

Q3 - Correct answer: B (It keeps working even if the object is copied to another variable)

Q: *Why prefer this.name over student.name inside a method?*

Why: this travels with the call, so the method stays correct for any object that uses it.

Why the other options are wrong:

A (It is shorter to type) - Length is a side benefit, not the reason.

C (this is required by JSON) - JSON never appears here.

D (student does not exist inside functions) - It does, but the global reference is fragile.

Q4 - Correct answer: D (bump() { ... })

Q: *Which shorthand defines a method correctly?*

Why: The shorthand drops the colon and the function keyword: bump() { ... }.

Why the other options are wrong:

A (bump: function => {}) - Mixed syntax does not exist.

B (function bump() { ... } inside the object) - That defines a plain function, not a property.

C (() bump {}) - Reversed arrows are not valid.

Lesson 9.4 - JSON: Objects as Text

Q1 - Correct answer: B (Turns an object into a JSON text string)

Q: *What does JSON.stringify do?*

Why: stringify serializes: live data becomes text.

Why the other options are wrong:

A (Turns JSON text into an object) - That is JSON.parse.

C (Deletes object properties) - It changes nothing in memory.

D (Saves a file to disk) - File writing needs fs - Chapter 10.

Q2 - Correct answer: C (An object with a = 1)

Q: *What does JSON.parse('{"a": 1}') return?*

Why: parse rebuilds a live object from valid JSON text.

Why the other options are wrong:

A (The number 1) - The whole object comes back, not one value.

B (An error) - The text is valid JSON, so no error.

D (The text "{"a": 1}") - The text was consumed and converted.

Q3 - Correct answer: A ('{"name": "Maya"}')

Q: *Which is VALID JSON?*

Why: JSON demands double-quoted keys and strings, with no trailing comma.

Why the other options are wrong:

B ('{name: "Maya"}') - Unquoted keys are invalid JSON.

C ('{'name': 'Maya'}') - Single quotes are illegal in JSON.

D ('{"name": "Maya",}') - The trailing comma breaks the rule.

Q4 - Correct answer: D (Format the text with 2-space indentation)

Q: *What do the extra arguments in JSON.stringify(obj, null, 2) do?*

Why: The third argument is the indent size for human-readable output.

Why the other options are wrong:

A (Encrypt the output) - No encryption happens.

B (Limit the output to 2 properties) - All properties are included.

C (Parse twice for safety) - Nothing is parsed here.

Lesson 9.5 - Looping Over Objects: keys, values, entries

Q1 - Correct answer: A (["a", "b"])

Q: *What does Object.keys({ a: 1, b: 2 }) return?*

Why: Object.keys returns an array of the property NAMES.

Why the other options are wrong:

B ([1, 2]) - [1, 2] is what Object.values returns.

C (["a: 1", "b: 2"]) - That mixes keys with values.

D (2) - 2 is the count: Object.keys(...).length.

Q2 - Correct answer: C ([["Aya", 91]])

Q: *What does Object.entries({ Aya: 91 }) return?*

Why: entries wraps each key-value pair as its own two-item array, so the result is an array of arrays.

Why the other options are wrong:

A (["Aya"]) - That is only the keys.

B ([91]) - That is only the values.

D (["Aya", 91]) - That is one flat array of two items.

Q3 - Correct answer: B (The value of book.pages)

Q: *After let { pages } = book;, what does the variable pages hold?*

Why: Object destructuring creates a variable named after the property and fills it with that property value.

Why the other options are wrong:

A (undefined) - It is defined whenever the property exists.

C (The string "pages") - It holds what was stored in book.pages.

D (The whole book object) - It is a plain variable, not the object.

Q4 - Correct answer: D (undefined)

Q: *What does user.pet?.name return when user.pet is undefined?*

Why: Optional chaining short-circuits and returns undefined instead of throwing an error.

Why the other options are wrong:

A (An error stops the program) - Avoiding that error is the whole point.

B (null) - null would have to be stored somewhere.

C ("") - An empty string is a real value, not a miss.

Chapter 10 - Files, Modules, and Real Programs

Lesson 10.1 - Modules: Sharing Code Between Files

Q1 - Correct answer: B (export)

Q: *Which keyword marks what a module GIVES to other files?*

Why: export labels the boxes a module offers; import carries them away.

Why the other options are wrong:

A (import) - import takes FROM other modules.

C (include) - include is not a JavaScript keyword.

D (share) - share is not either.

Q2 - Correct answer: C (A file in the same folder as the importing file)

Q: *What does "./math-tools.js" mean in an import?*

Why: ./ means "this folder" - a relative path to your own module.

Why the other options are wrong:

A (A folder on the internet) - Paths stay local unless written as URLs.

B (A special shortcut) - It is the standard relative path form.

D (A built-in Node module) - Built-ins use node: prefixes, no ./

Q3 - Correct answer: A (from "node:fs")

Q: *How is the built-in file system module imported?*

Why: Built-in modules use the node: prefix; fs is the file system module.

Why the other options are wrong:

B (from "files") - No module named just "files".

C (from "./fs.js") - That would be your own file.

D (from "system") - "system" is not a Node module.

Q4 - Correct answer: C (It makes import lines read like a table of contents)

Q: *Why does the lesson suggest naming modules after their job?*

Why: Clear file names make the program self-documenting at the top of every file.

Why the other options are wrong:

A (Node requires it) - Any legal file name works.

B (Shorter names are illegal) - Long descriptive names are preferred.

D (It makes imports run faster) - Naming has no speed effect.

Lesson 10.2 - Reading Files with readFileSync

Q1 - Correct answer: D (The whole file as one string)

Q: *What does readFileSync return?*

Why: With "utf-8" encoding, the full text arrives as a single string.

Why the other options are wrong:

A (An array of lines) - Splitting into lines is slice/split work.

B (A number of characters) - text.length gives the count afterward.

C (A file object) - No wrapper object - just the text.

Q2 - Correct answer: A (It tells Node how to translate bytes into characters)

Q: *Why pass "utf-8" to every file read and write?*

Why: The encoding maps raw bytes to readable letters; utf-8 is the universal standard.

Why the other options are wrong:

B (It compresses the file) - No compression happens.

C (It is required by law in some countries) - It is a technical standard, not a law.

D (It makes the file write faster) - Encoding is about correctness, not speed.

Q3 - Correct answer: B (The current working directory of the terminal)

Q: *Where does Node look for "story.txt" when you run a program?*

Why: Relative names resolve against process.cwd() - the folder the terminal was in at launch.

Why the other options are wrong:

A (Always the folder containing the .js file) - Not necessarily the script folder!

C (The internet) - Files are local.

D (Wherever the file was created) - Creation location does not matter.

Q4 - Correct answer: D (The current working directory)

Q: *What does process.cwd() tell you?*

Why: cwd stands for current working directory - the first thing to inspect when files go missing.

Why the other options are wrong:

A (The program's line count) - Unrelated to lines.

B (How much memory is free) - Memory tools live elsewhere.

C (The file encoding) - Encoding is what YOU pass in.

Lesson 10.3 - Writing and Appending Files

Q1 - Correct answer: C (Replaces its entire contents)

Q: *What does writeFileSync do to an existing file?*

Why: writeFileSync overwrites completely - fresh content every time.

Why the other options are wrong:

A (Refuses to change it) - It writes boldly.

B (Renames it) - Renaming is renameSync's job.

D (Adds to its end) - That is appendFileSync.

Q2 - Correct answer: B (appendFileSync)

Q: *Which tool adds text to the END of a file?*

Why: appendFileSync glues text after the existing content, creating the file if needed.

Why the other options are wrong:

A (writeFileSync) - That replaces everything.

C (readFileSync) - That reads.

D (pushFileSync) - No such method exists.

Q3 - Correct answer: A (Node creates the file first, then appends)

Q: *What happens if you append to a file that does not exist yet?*

Why: Both write and append create missing files automatically.

Why the other options are wrong:

B (An error crashes the program) - Creation is automatic - no crash.

C (Nothing happens at all) - The text really lands in a new file.

D (The text is lost silently) - Nothing is lost.

Q4 - Correct answer: C (So entries stay on separate lines in the file)

Q: *Why end each appended log entry with \n?*

Why: Without the newline, entries glue into one endless line.

Why the other options are wrong:

A (It encrypts the entry) - No encryption involved.

B (It makes writes faster) - Speed is unaffected.

D (It is required by appendFileSync) - The parameter is optional.

Lesson 10.4 - Organizing a Real Program

Q1 - Correct answer: D (JSON.stringify / JSON.parse)

Q: *Which pair converts data for file storage and back?*

Why: stringify turns data into text for the file; parse rebuilds data from the text.

Why the other options are wrong:

A (writeFileSync / readFileSync) - Those move raw text, not structured data.

B (export / import) - Those share code, not data.

C (push / pop) - Those edit arrays.

Q2 - Correct answer: B (Job: data, calculations, storage, conductor)

Q: *A well-organized project splits files by...*

Why: Modules by responsibility stay findable and testable.

Why the other options are wrong:

A (Alphabetical order) - Order of letters helps nobody.

C (File size) - Size is not a design principle.

D (Creation date) - Dates do not reflect roles.

Q3 - Correct answer: A (Return an empty list and continue)

Q: *What is a safe fallback when loading a missing save file?*

Why: A calm message plus a sane default (empty list) keeps the program usable.

Why the other options are wrong:

B (Crash with a full error dump) - Crashing is what catch prevents.

C (Delete the program) - Never self-destruct.

D (Ask the internet) - Local handling is the norm.

Q4 - Correct answer: B (A SyntaxError from a missing brace)

Q: *Which error family CANNOT be caught with try / catch?*

Why: Syntax errors stop the program before it runs - no runtime code exists to catch them.

Why the other options are wrong:

A (A thrown Error with a message) - Thrown errors are exactly what catch takes.

C (A missing file error) - Runtime file errors are catchable.

D (Invalid JSON during parse) - Parse errors are catchable.

Practice Answers

Chapter 1 - Hello, JavaScript

P1.1 - Correct answer: D (Inside a refrigerator that was never programmed)

Q: *Which of these is NOT a place where JavaScript normally runs?*

Why: JavaScript runs in browsers, on servers via Node.js, and in many app tools - but only if someone put a JavaScript engine inside the device. An ordinary fridge has none.

Why the other options are wrong:

A (Inside a web browser) - Browsers have had JavaScript engines since 1995.

B (On a server with Node.js) - Node.js brings JavaScript to servers and computers.

C (In mobile apps built with JavaScript tools) - Many mobile apps are built with JavaScript-based tools.

P1.2 - Correct answer: C (It prints the text 7, without the quotation marks)

Q: *What does the statement console.log("7"); print on the screen?*

Why: The quotes only mark where the text begins and ends; they are not printed. The console shows the characters 7 and nothing more.

Why the other options are wrong:

A (It prints nothing, because numbers are invisible) - A single 7 is printed; it is not doubled or multiplied.

B (It prints an error message) - Numbers are perfectly printable - console.log prints them all the time.

D (It prints the number 14) - The quotes are syntax, not content, so no quotation marks appear.

P1.3 - Correct answer: B (console.log("Noon"); then console.log("Morning"); then console.log("Night");)

Q: *Which order of statements prints "Morning", "Noon", "Night"?*

Why: Statements run from top to bottom, so the first statement must print Morning, the second Noon, and the third Night.

Why the other options are wrong:

A (console.log("Morning"); then console.log("Noon"); then console.log("Night");) - That order prints Night first.

C (console.log("Morning"); then console.log("Night"); then console.log("Noon");) - That order prints Noon first.

D (console.log("Night"); then console.log("Morning"); then console.log("Noon");) - That order prints Morning, then Night, then Noon.

P1.4 - Correct answer: A (Comments are ignored, so they make no difference to speed)

Q: *A friend claims comments make programs run faster. What is the truth?*

Why: The computer skips comments entirely, so they neither speed up nor slow down a program.

Why the other options are wrong:

B (Comments always make programs much faster) - Comments are skipped, so they cannot improve speed.

C (Comments make programs run much slower) - Being skipped, they cannot add any delay.

D (Only long comments make programs slower) - Comments have zero effect on speed, long or short.

P1.5 - Correct answer: A (console.log("One"); and then console.log("Two"); on the next line)

Q: *Which program prints two separate lines of output?*

Why: Two console.log statements, one per line, print two separate lines. A single call with "One Two" inside prints just one line.

Why the other options are wrong:

B (console.log("One Two");) - That prints the single line: One Two.

C (console.log(One); and then console.log(Two);) - This is the correct answer: two calls, two lines.

D (console.log "One"; and then console.log "Two";) - Without quotes, One and Two are treated as unknown names, so this crashes.

P1.6 False - console.log is a built-in function that you call. Keywords such as let and if are part of the grammar; console.log is provided by the environment as a function.

P1.7 Expected output: Top then Bottom, each on its own line. The middle statement is a comment, so the computer ignores it completely.

P1.8 Bug: Line 1 misspells the function name: consol.log is not defined. Fix it to console.log with the correct spelling.

P1.9 Sample answer: Sample answer: The computer runs statements from top to bottom, so a statement can only use results from statements above it. For example, if line 2 prints a message and line 1 defines it, swapping them breaks the program.

P1.10 One possible solution:

```
console.log("Sara");
console.log("Green");
console.log(7);
```

Chapter 2 - Variables and Values

P2.1 - Correct answer: D (const MAX_SCORE = 100;)

Q: *A program keeps a player's maximum allowed score fixed at 100. Which declaration fits best?*

Why: A fixed value is exactly what const is for; the name is written plainly, without quotes.

Why the other options are wrong:

A (let MAX_SCORE = 100;) - let would allow accidental changes later.

B (MAX_SCORE = 100;) - Without let or const the name becomes a global variable - a bad habit, not a declaration.

C (const "MAX_SCORE" = 100;) - Names never take quotes.

P2.2 - Correct answer: B (1)

Q: *What does console.log(10 % 3); print?*

Why: 10 divided by 3 is 3 with 1 left over, so the remainder is 1.

Why the other options are wrong:

A (0) - 0 would mean 3 divides 10 exactly, which it does not.

C (3) - 3 is the quotient 10 / 3 rounded down, not the remainder.

D (3.33) - 3.33 is the decimal quotient of 10 / 3.

P2.3 - Correct answer: B ((5 + 4) * 10 / 2)

Q: *Which expression equals 45?*

Why: Parentheses first: 5 + 4 = 9, then 9 * 10 = 90, then 90 / 2 = 45.

Why the other options are wrong:

A (6 + 4 * 10) - 6 + 4 * 10 is 6 + 40 = 46 (multiplication first).

C (5 * 4 + 10) - 5 * 4 + 10 is 20 + 10 = 30.

D (54 - 10) - 54 - 10 is 44.

P2.4 - Correct answer: C ("a12")
Q: *What does console.log("a" + 1 + 2); print?*

Why: Left to right: "a" + 1 gives "a1", then "a1" + 2 gives "a12".
Why the other options are wrong:

A (3) - 3 forgets the leading string.

B (An error) - Mixing with + is legal; no error occurs.

D ("a3") - "a3" would need numeric addition after the "a", impossible.

P2.5 - Correct answer: D (remainingLives)
Q: *Which name follows camelCase and clearly describes a value?*

Why: camelCase starts lowercase and capitalizes each new word: remainingLives.
Why the other options are wrong:

A (TP) - TP is legal but far too cryptic.

B (remaininglives) - All lowercase words run together and become hard to read.

C (Remaining Lives) - Spaces are not allowed inside names.

P2.6 False - Variable names cannot start with a digit, so 3cats causes a SyntaxError. A legal alternative is cats3.

P2.7 Expected output: 7 then 1. First x becomes 5 + 2 = 7; then 7 % 3 leaves a remainder of 1.

P2.8 Bug: The leading string makes + concatenate: total becomes "12" as TEXT here, but if the expectation was numeric 3 vs 12 - the bug is quoting the 1. To add numbers, write let total = 1 + 2;

P2.9 Sample answer: Sample answer: let creates a variable whose value can be reassigned, such as a game score that increases. const creates one that cannot be reassigned, such as the number of days in a week.

P2.10 One possible solution:

```
let apples = 3;
let bananas = 4;
console.log(apples + bananas);
console.log("I have " + (apples + bananas) + " fruits");
```

Chapter 3 - Working with Strings

P3.1 - Correct answer: C (console.log("One\nTwo\nThree");)
Q: *Which line prints three separate lines of output?*

Why: Each \n starts a new line, so three lines appear.
Why the other options are wrong:

A (console.log("One\tTwo\tThree");) - Tabs indent on the SAME line.

B (console.log("One" "Two" "Three");) - Adjacent strings without + are a syntax error.

D (console.log("One Two Three");) - One line with spaces is printed here.

P3.2 - Correct answer: A ("Box: 15")
Q: *What does `Box: ${5 * 3}` evaluate to?*

Why: The placeholder evaluates 5 * 3 and inserts 15.
Why the other options are wrong:

B ("Box: ${5 * 3}") - Backtick placeholders always evaluate.

C ("Box: 5 * 3") - The math happens; the code text does not survive.

D (15) - The literal is the whole sentence, not just the number.

P3.3 - Correct answer: D ("ers")
Q: *What does "Conversation".slice(4, 7) return?*

Why: Count: C=0, o=1, n=2, v=3, e=4, r=5, s=6. Positions 4 to 6 are e, r, s - so the slice is "ers".
Why the other options are wrong:

A ("ver") - "ver" starts at position 3 (slice(3, 6)).

B ("vers") - "vers" would need slice(4, 8).

C ("sa") - "sa" skips ahead to positions 6-7.

P3.4 - Correct answer: C (" Paris ".trim())

Q: *A variable holds " Paris ". Which expression gives exactly "Paris"?*

Why: trim strips whitespace from both ends, giving "Paris".

Why the other options are wrong:

A (" Paris ".length) - length returns a number (9), not text.

B (" Paris ".replace(" ", "")) - replace(" ", "") removes only the FIRST space and would give " Paris ".

D (" Paris ".toUpperCase()) - Uppercasing changes letters, not spaces.

P3.5 - Correct answer: C (1)

Q: *What does "moon".indexOf("o") return?*

Why: Positions: m=0, o=1, o=2, n=3. indexOf reports the FIRST match: 1.

Why the other options are wrong:

A (-1) - -1 would mean no o exists.

B (0) - 0 is where "m" sits.

D (2) - 2 is the second o; indexOf stops at the first.

P3.6 False - Strings are immutable: toUpperCase returns a new string. The original stays "abc" unless you store the result back.

P3.7 Expected output: 4, then b, then "ook". The length is 4; position 0 is b; slice(1) copies from position 1 to the end.

P3.8 Bug: toUpperCase returns a NEW string that was thrown away. Fix: shout = shout.toUpperCase(); before printing.

P3.9 Sample answer: Sample answer: Immutable means an existing string's characters can never be changed. Methods build and return new strings. Assigning the result stores the new string into the variable; without assignment the new string is created and immediately lost.

P3.10 One possible solution:

```
let name = "Lina";
let age = 13;
console.log(`I am ${name} and I am ${age} years old`);
```

Chapter 4 - Numbers and Math

P4.1 - Correct answer: A (Math.floor(17 / 6))

Q: *A shop packs 17 muffins into boxes of 6. Which expression gives the number of FULL boxes?*

Why: "FULL boxes" means whole completed boxes: floor(17 / 6) is 2.

Why the other options are wrong:

B (17 % 6) - 17 % 6 is 5 - that counts leftover muffins, not boxes.

C (Math.ceil(17 / 6)) - ceil gives 3, counting the half-filled box too.

D (Math.round(17 / 6)) - round gives 3, counting a box that is not actually full.

P4.2 - Correct answer: C (12)

Q: *What does Math.abs(-12) return?*

Why: abs returns the distance from zero: |-12| = 12.

Why the other options are wrong:

A (-12) - abs never returns a negative value.

B (0) - Only abs(0) is 0.

D (NaN) - Converting -12 is a normal job for abs.

P4.3 - Correct answer: D (3)

Q: *What does Math.ceil(2.01) return?*

Why: ceil rounds UP to the next whole number: any decimal, however small, pushes 2.01 to 3.

Why the other options are wrong:

A (2) - 2 is the floor result.

B (2.01) - ceil always returns a whole number.

C (2.1) - ceil returns whole numbers only.

P4.4 - Correct answer: C (Math.floor(Math.random() * 20) + 1)

Q: *Which expression rolls a random whole number from 1 to 20?*

Why: Twenty values (0-19), then +1 shifts to 1-20.

Why the other options are wrong:

A (Math.random() * 20) - Decimals, not whole numbers.

B (Math.floor(Math.random() * 20)) - Gives 0-19; the 20 is missing.

D (Math.ceil(Math.random()) * 20) - ceil(random()) is always 1, so this always gives 20.

P4.5 - Correct answer: D ("number")

Q: *What does typeof NaN evaluate to?*

Why: A famous quirk: typeof reports "number" for NaN, because NaN is a special numeric value.

Why the other options are wrong:

A ("nan") - "nan" is not a JavaScript type name.

B ("undefined") - "undefined" belongs to missing values.

C ("notanumber") - "notanumber" is just the meaning of the name, not the type.

P4.6 True - For positive numbers, floor and trunc agree: both drop the decimal to give 9. They differ only for negatives.

P4.7 Expected output: 9, then -5, then 9 again: max picks 9, min picks -5, and 3 squared is 9.

P4.8 Bug: The range is 0-5 because the +1 shift is missing. Fix: Math.floor(Math.random() * 6) + 1.

P4.9 Sample answer: Sample answer: Binary floating point cannot store 0.1 or 0.2 exactly, so the sum is 0.30000000000000004. Safe approaches: display with toFixed(2), or compare rounded/tolerant values.

P4.10 One possible solution:

```
let n = 23.678;
console.log(n.toFixed(2));
console.log(Math.sqrt(n).toFixed(1));
console.log(Math.abs(-23.678));
```

Chapter 5 - Making Decisions

P5.1 - Correct answer: C (age >= 13 && age <= 19)

Q: *Which expression correctly tests whether age is between 13 and 19 (inclusive)?*

Why: Both bounds must hold at once, so the two comparisons join with &&.

Why the other options are wrong:

A (age >= 13 || age <= 19) - || would be true even for age 5.

B (age === 13 && age === 19) - age cannot equal both 13 and 19 at once.

D (13 <= age <= 19) - Chained notation like that is not how JavaScript reads comparisons.

P5.2 - Correct answer: A (an error)

Q: *What does this program print? let x = 9; if (x % 2 === 0) { console.log("even"); } else { console.log("odd"); }*

Why: 9 % 2 is 1, which is not === to 0, so the else path prints odd.

Why the other options are wrong:

B (even) - even is wrong because 9 is not divisible by 2.

C (odd) - Correct: 9 % 2 is 1, so "odd" prints.

D (nothing) - One of the two blocks always runs.

P5.3 - Correct answer: D (true)

Q: *What does console.log(false || 3 > 2); print?*

Why: || needs at least one true side; 3 > 2 is true, so the expression is true.

Why the other options are wrong:

A (false) - The right side rescues the expression.

B (3) - || returns a boolean, not an operand value.

C (undefined) - Nothing is undefined here.

P5.4 - Correct answer: B (A switch with five cases and a default)

Q: *A program must print a badge color for a level number from 1 to 5, with a fallback for anything else. Which structure fits best?*

Why: One value against fixed labels plus a fallback is exactly switch with default.

Why the other options are wrong:

A (A chain of five separate if statements) - Separate ifs work but cannot share the fallback neatly.

C (One if with the || operator) - || compares facts, not menu choices.

D (Math.max on the level) - Math.max finds largest values, not labels.

P5.5 - Correct answer: A (false)

Q: *What does the expression !true || false evaluate to?*

Why: !true is false, and false || false is false.

Why the other options are wrong:

B (true) - The OR has no true side, so it fails.

C (true then false) - A single boolean results, not two.

D (An error) - Logical operators combine legally.

P5.6 False - A single = assigns 100 to score; comparisons use ===. This assignment also makes the condition truthy, so the block always runs - a sneaky bug.

P5.7 Expected output: Welcome. The strict comparison matches, so the if block runs and the else is skipped.

P5.8 Bug: The comparison points the wrong way: 31 > 30 is true, so it prints "Cold day". Fix the logic: temp > 30 should print "Hot day" (or the branches must be swapped).

P5.9 Sample answer: Sample answer: = assigns a value; == compares loosely with type conversion; === compares strictly with no conversion. The book uses === because predictable strictness avoids surprising bugs like "5" == 5.

P5.10 One possible solution:

```
let n = -4;
if (n > 0) {
    console.log("positive");
} else if (n < 0) {
    console.log("negative");
} else {
    console.log("zero");
}
```

Chapter 6 - Loops

P6.1 - Correct answer: A (A for loop with counter i from 1 to 100)

Q: *Which loop is the best fit for printing every number from 1 to 100?*

Why: A known, counted range is exactly what the for loop was made for.

Why the other options are wrong:

B (A while loop with a random condition) - while works but for expresses counting more directly.

C (Ten nested if statements) - Ifs do not repeat.

D (A switch with 100 cases) - switch matches labels; it does not count.

P6.2 - Correct answer: B (3)

Q: *How many rounds does for (let i = 3; i <= 9; i += 3) run?*

Why: i visits 3, 6, 9 - three rounds.

Why the other options are wrong:

A (2) - 2 would miss the last value.

C (4) - 4 would need one more value.

D (9) - 9 is the boundary, not the count.

P6.3 - Correct answer: C (10)

Q: *What does this loop print? let total = 0; for (let i = 1; i <= 4; i++) { total = total + i; } console.log(total);*

Why: The total collects 1 + 2 + 3 + 4 = 10.

Why the other options are wrong:

A (4) - 4 is just the last counter value.

B (6) - 6 is 1+2+3 - one round short.

D (24) - 24 multiplies; the loop adds.

P6.4 - Correct answer: D (while checking the input)
Q: *A loop must keep asking until the user types "stop". Which loop fits best?*

Why: The number of rounds is unknown, so the loop watches the condition: while.
Why the other options are wrong:

A (for with 10 rounds) - Ten rounds is a guess, not a rule.

B (break without a loop) - break needs a loop to live in.

C (switch on the input length) - switch does not repeat.

P6.5 - Correct answer: D (6 lines like 1-1, 1-2, ... 2-3)
Q: *What does this nested loop print in total? for (let r = 1; r <= 2; r++) { for (let c = 1; c <= 3; c++) { console.log(r + "-" + c); } }*

Why: 2 outer rounds times 3 inner rounds = 6 printed pairs.
Why the other options are wrong:

A (5 lines) - 6 pairs, not 5.

B (2 lines) - The inner loop runs fully per outer round.

C (3 lines) - The inner loop alone runs 3 times per round.

P6.6 False - That is break. continue skips only the rest of the current round and starts the next one.

P6.7 Expected output: 2, 4, 6, 8, then Done. The counter steps by 2 and stops before 10.

P6.8 Bug: The update line is missing: n never shrinks, so n > 0 stays true. Add n = n - 1; inside the loop body.

P6.9 Sample answer: Sample answer: break stops the whole loop - right when the search target is found and further checking is wasted. continue skips one uninteresting item (say, an even number) and keeps searching the rest.

P6.10 One possible solution:

```
for (let i = 1; i <= 5; i++) {
    console.log("3 x " + i + " = " + 3 * i);
}
```

Chapter 7 - Functions

P7.1 - Correct answer: A (function hello() { console.log("Hi"); } followed by hello();)
Q: *Which program correctly defines AND uses a function?*

Why: Define first, then call: both parts are needed.
Why the other options are wrong:

B (function hello() { console.log("Hi"); } (nothing else)) - Definition without a call prints nothing.

C (hello(); written alone with no definition) - Calling an undefined name crashes.

D (console.log(hello);) - Printing the function name does not run it.

P7.2 - Correct answer: C (10)
Q: *What does this program print? function add(a, b) { return a + b; } console.log(add(2, 3) * 2);*

Why: add(2, 3) returns 5, and 5 * 2 is 10.
Why the other options are wrong:

A (25) - 25 would be 5 squared.

B (add) - The call returns a number, not the name.

D (5) - 5 is the inner result before doubling.

P7.3 - Correct answer: D (Only A prints; the function returns 5)
Q: *A function contains: console.log("A"); return 5; console.log("B"); What happens when it runs?*

Why: return ends the function at once, so the B line is unreachable.
Why the other options are wrong:

A (A and B print, returns 5) - B never runs - return already left.

B (Only B prints) - B sits after the return, unreachable.

C (Nothing happens) - A definitely prints first.

P7.4 - Correct answer: A (Pass data in via parameters and out via return)
Q: *Which design keeps functions independent and safe?*

Why: Parameters and return values are the clean data roads.
Why the other options are wrong:
B (Share one big global variable for everything) - Global sharing invites interference.
C (Use console.log for all data transfer) - Printing sends text to humans, not to code.
D (Avoid naming functions) - Names are required and helpful.

P7.5 - Correct answer: B (GO!)
Q: *What does this print? const shout = (w) => w.toUpperCase() + "!"; console.log(shout("go")); (function declarations are also fine)*

Why: toUpperCase capitalizes, then ! is glued on: GO!.
Why the other options are wrong:
A (go!) - The capitalization applies first.
C (GO) - The exclamation mark is part of the result.
D (go) - Nothing stays lowercase.

P7.6 True - Function declarations are hoisted in JavaScript - the definition is registered before the code runs, so calls earlier in the file work. (const arrow functions are NOT hoisted.)

P7.7 Expected output: big, then small. For 5 the first return fires; for 2 the early return is skipped and "small" comes back.

P7.8 Bug: The returned value is dropped: double(10) computes 20 but nobody catches it. Fix: console.log(double(10)); or store it in a variable.

P7.9 Sample answer: Sample answer: sides is the parameter - the named slot in the definition; 6 is the argument - the actual value supplied at call time and stored in the slot.

P7.10 One possible solution:

```
/**
 * Computes rectangle area.
 * @param {number} width
 * @param {number} height
 * @return {number} width times height
 */
function areaOfRect(width, height) {
    return width * height;
}
console.log(areaOfRect(4, 3));
```

Chapter 8 - Arrays

P8.1 - Correct answer: B ([5, 10, 15])
Q: *Which array holds exactly four items?*

Why: Careful reading: count the top-level items. Option B has four - a, b, c, d - and is the correct choice. Option A has three; D has three too (the inner array counts as one).
Why the other options are wrong:
A ([1, [2, 3], 4]) - Three items: 1, the inner array, and 4.
C (["a", "b", "c", "d"]) - Correct: four top-level items.
D ([]) - An empty array holds zero.

P8.2 - Correct answer: D ("moon")
Q: *What does console.log(x[1]); print for x = ["sun", "moon", "star"]?*

Why: Index 1 is the second item: "moon".
Why the other options are wrong:
A ("sun") - "sun" sits at index 0.
B ("star") - "star" is at index 2.
C (undefined) - Index 1 exists here.

P8.3 - Correct answer: C ([0, 1, 2])
Q: *After let q = [1, 2]; q.unshift(0); what is q?*

Why: unshift adds to the FRONT: 0 joins ahead of 1.
Why the other options are wrong:
A ([2, 1]) - Nothing was removed.
B ([0]) - The original items survive.
D ([1, 2, 0]) - That is push's work.

P8.4 - Correct answer: A (arr[arr.length - 1])
Q: *Which expression gives the LAST item of arr, whatever its length?*

Why: The last index is length - 1.
Why the other options are wrong:
B (arr[arr.length]) - arr[length] is one past the end: undefined.
C (arr.last()) - No last() method exists.
D (arr[-1]) - Negative indexes do not work in JavaScript arrays.

P8.5 - Correct answer: C (8 and 16)
Q: *What does this loop print? let nums = [4, 8]; for (let n of nums) { console.log(n * 2); }*

Why: for...of visits each value; doubling gives 8 then 16.
Why the other options are wrong:
A (0 and 1) - Those are indexes, not values.
B (8) - Both items are visited.
D (4 and 8) - That is the unchanged list.

P8.6 False - pop returns the REMOVED ITEM. push and unshift return the new length - a famous pair to confuse.

P8.7 Expected output: 4, then clip. The push adds a fourth item; index 3 holds the newcomer.

P8.8 Bug: colors.length is 3, and index 3 does not exist (valid indexes are 0-2). Fix: colors[colors.length - 1].

P8.9 Sample answer: Sample answer: Valid indexes run from 0 to length - 1, so length itself is always one past the end. Subtracting one lands on the final item no matter how long the array is.

P8.10 One possible solution:

```
let subjects = ["Math", "Science", "History"];
console.log(subjects[0]);
subjects[2] = "Art";
console.log(subjects);
```

Chapter 9 - Objects

P9.1 - Correct answer: C (let car = { brand: "Fiat", year: 2020 };)
Q: *Which object literal is written correctly?*

Why: Keys use colons, pairs use commas, and the whole thing sits in braces.
Why the other options are wrong:
A (let car = ["brand: Fiat", "year: 2020"];) - That is an array of strings, not an object.
B (let car = (brand: "Fiat");) - Parentheses do not build objects.
D (let car = { brand = "Fiat", year = 2020 };) - Equal signs belong to assignments, not literals.

P9.2 - Correct answer: D (412)
Q: *For let book = { title: "Dune", pages: 412 };, what does book.pages return?*

Why: Dot access returns the value stored under that key.
Why the other options are wrong:
A ("pages") - That is the key name, not its value.
B ("Dune") - That is a different property.
C (undefined) - The property exists.

P9.3 - Correct answer: A (tea then L)
Q: *What does this print? let menu = { drink: "tea", size: "L" }; for (let k in menu) { console.log(k); }*

Why: for...in visits KEYS only; the values would need menu[k].

Why the other options are wrong:

B (drink then size) - Correct: the key names.

C (drink = tea, size = L) - Printing the values too needs menu[k] inside.

D (An error) - for...in on objects is legal.

P9.4 - Correct answer: B (record[key])

Q: *A variable key holds "score". Which line reads that property of record?*

Why: Dynamic keys require brackets: record[key] uses the variable's value.

Why the other options are wrong:

A (record.key) - Looks for a property literally named "key".

C (record("score")) - Parentheses call functions.

D (key.record) - Objects and keys are reversed.

P9.5 - Correct answer: C ("[1, 2, 3]")

Q: *What does JSON.stringify([1, 2, 3]) return?*

Why: stringify always returns a STRING, and arrays stringify in bracket form.

Why the other options are wrong:

A ("1,2,3") - That is join(",").

B (6) - No math happens.

D ([1, 2, 3]) - The array itself is not returned - text is.

P9.6 False - Objects grow freely: assigning to a new key adds the property on the spot.

P9.7 Expected output: cat, then 4. Dot access reads kind; bracket access reads age, and 3 + 1 is 4.

P9.8 Bug: The dot looks for a property literally named "key". Use brackets: student[key].

P9.9 Sample answer: Sample answer: student.name reads the property called name. student[name] uses the VALUE of the variable name as the key - if that value is "Maya", it reads student.Maya, which is undefined.

P9.10 One possible solution:

```
let game = {
    title: "Chess",
    hours: 12,
    fun: true
};
console.log(game.title);
game.platform = "board";
console.log(game);
```

Chapter 10 - Files, Modules, and Real Programs

P10.1 - Correct answer: A (const double = require math-tools;)

Q: *Which line correctly imports the double function from a local module?*

Why: Careful reading: the standard ES-module import is import { double } from "./math-tools.js" - that is option B. Option C mixes require (an older system) with wrong syntax.

Why the other options are wrong:

B (include double from "math-tools";) - include is not JavaScript.

C (import { double } from "./math-tools.js";) - Correct: named import from a relative path.

D (import math-tools.double;) - Import lists need braces and a from clause.

P10.2 - Correct answer: C (11)

Q: *A file story.txt contains "Hello\nWorld". After reading with "utf-8", what is text.length?*

Why: The \n is ONE character: 5 + 1 + 5 = 11.

Why the other options are wrong:

A (2) - 2 counts only the words.

B (5) - 5 is just the first word.

D (12) - 12 would count the backslash and n separately - they are one character in the file.

P10.3 - Correct answer: D (appendFileSync("log.txt", "new entry", "utf-8");)
Q: *Which sequence adds "new entry" to the END of log.txt without erasing it?*

Why: appendFileSync is the end-adder.
Why the other options are wrong:

A (writeFileSync("log.txt", "new entry", "utf-8");) - That wipes the file first.

B (readFileSync("log.txt", "new entry");) - Reading does not take content to write.

C (export "new entry" to "log.txt";) - export ships code, not file text.

P10.4 - Correct answer: C (process.cwd() - the terminal's working directory)
Q: *Your program says "ENOENT: no such file" for "data.txt" even though the file exists next to your .js file. What is the FIRST thing to check?*

Why: Relative paths resolve from the working directory, not the script location.
Why the other options are wrong:

A (The file's font) - Fonts never affect file lookup.

B (Your import statements) - Imports are unrelated to reading paths.

D (Reinstall Node.js) - Node is fine.

P10.5 - Correct answer: C (safe)
Q: *What does this print? import { readFileSync } from "node:fs"; try { console.log(readFileSync("missing.txt", "utf-8")); } catch (err) { console.log("safe"); }*

Why: The missing file throws inside try; catch prints "safe".
Why the other options are wrong:

A (An uncaught error) - The try / catch catches it.

B (undefined) - Nothing undefined reaches the console.

D (The file contents) - There is no file to read.

P10.6 True - Both write and append create missing files - creation is built in.

P10.7 Expected output: Day 1 then Day 2 on separate lines: the file was created, two entries appended in order, then read back.

P10.8 Bug: writeFileSync replaces the whole file. To accumulate, use appendFileSync (or read-modify-write the JSON data).

P10.9 Sample answer: Sample answer: Save with JSON.stringify to turn the array into text for writeFileSync; on load, readFileSync returns TEXT, so JSON.parse converts it back into a live array with real numbers.

P10.10 One possible solution:

```
import { readFileSync } from "node:fs";

try {
    let text = readFileSync("names.txt", "utf-8");
    console.log("Characters: " + text.length);
} catch (err) {
    console.log("Could not read names.txt: " +
        err.message);
}
```

Mixed Quiz Answers

Chapter 1 - Hello, JavaScript

Q1.1 - Correct answer: C (respond to the user)
Q: *JavaScript was created so that web pages could...*

Why: JavaScript was invented in 1995 to make web pages interactive and responsive to what the user does.
Why the other options are wrong:

A (replace HTML completely) - HTML still exists and was not replaced.

B (run only inside databases) - Databases are unrelated to why JavaScript was created.

D (stay offline forever) - Web pages never needed to stay offline.

Q1.2 - Correct answer: D (console.log("Hi");)
Q: *Which is the correct way to write the print statement?*

Why: The name is lowercase console.log and the text must be in quotes inside parentheses.
Why the other options are wrong:

A (Console.log("Hi");) - Capital C does not exist; JavaScript is case-sensitive.

B (console.log(Hi);) - Without quotes, Hi is treated as a name the computer does not know, causing an error.

C (print.log("Hi");) - print.log is not a JavaScript function.

Q1.3 - Correct answer: B (2)
Q: *How many statements are in this program? console.log("A"); // console.log("B"); console.log("C");*

Why: Careful reading: comments are not statements. The program has two real statements (A and C lines); the B line is a comment.
Why the other options are wrong:

A (1) - Only counting one line misses the last statement.

C (3) - 3 would count the commented line, which the computer ignores.

D (4) - 4 overcounts; there is no fourth line.

Q1.4 - Correct answer: B (It reports an error and the program stops at that point)
Q: *What happens when JavaScript meets a statement it cannot understand?*

Why: Errors stop execution at the broken statement; statements before it have run, statements after it do not run.
Why the other options are wrong:

A (It asks the user to fix it before stopping) - The console shows the error; it does not block and ask.

C (It rewrites the statement automatically) - JavaScript never silently rewrites your code.

D (It skips it and continues normally forever) - The program does not continue past a syntax error.

Q1.5 - Correct answer: D (/* like this */)
Q: *Which line is a multi-line comment?*

Why: Slash-star to star-slash wraps a comment of any length.
Why the other options are wrong:

A (// like this) - Double slashes make a single-line comment only.

B (# like this) - Hash marks are not JavaScript comment syntax.

C (<!-- like this -->) - Angle-bracket dashes are HTML comment syntax.

Q1.6 - Correct answer: B (a note for humans that the computer ignores)
Q: *A comment is best described as...*

Why: Comments exist purely for human readers; the machine skips them.
Why the other options are wrong:

A (a way to print hidden text) - Nothing hidden is printed by a comment.

C (an instruction the computer runs quietly) - Comments are never executed.

D (a type of error message) - Errors are reported, not written by you as comments.

Q1.7 - Correct answer: A (Book on one line, then Pen on the next line)
Q: *What does this print? console.log("Book"); console.log("Pen");*

Why: Statements run top to bottom and console.log ends its output with a new line, so Book appears before Pen.
Why the other options are wrong:

B (BookPen on one line) - Each console.log call ends with a line break; they do not glue together.

C (An error, because order is wrong) - Book before Pen is the correct order, so no error exists.

D (Pen on one line, then Book on the next line) - That is the reverse of the execution order.

Q1.8 - Correct answer: C (One statement per line, helpful comments, consistent spacing)

Q: *Which habit makes code easiest for humans to read?*

Why: Clean code is one statement per line with meaningful comments and consistent spacing.

Why the other options are wrong:

A (Deleting all comments to keep files small) - Comments help readers; deleting them hurts.

B (Mixing capital and small letters randomly) - Random casing breaks names like console.log.

D (Writing everything on one long line) - Long lines are hard to scan and easy to misread.

Q1.9 True - Execution is top-to-bottom; the error only stops things at the point where it happens.

Q1.10 True - F12 opens developer tools in Chrome, Edge, and Firefox, and the Console tab accepts JavaScript.

Q1.11 False - A /* ... */ comment can cover any number of lines, from part of one line to many.

Q1.12 Sample answer: Sample answer: A statement is an instruction the computer executes, like console.log("Go"); A comment is a note the computer ignores, like // check this later.

Q1.13 Sample answer: Sample answer: Typing builds muscle memory for punctuation and spelling, and it forces you to read every character, so you notice details like case-sensitivity and quotes.

Q1.14 One possible solution:

```
console.log("JavaScript");
console.log(1995);
```

Q1.15 One possible solution:

```
// print the first two days
console.log("Day 1");
console.log("Day 2");
```

Chapter 2 - Variables and Values

Q2.1 - Correct answer: A (let temperature = 21;)

Q: *Which statement creates a variable correctly?*

Why: The order is keyword, name, =, value: let temperature = 21;

Why the other options are wrong:

B (temperature let = 21;) - The keyword must come first.

C (let = temperature 21;) - Both = and the value are missing their correct places.

D (let temperature: 21;) - JavaScript uses = for assignment, not a colon.

Q2.2 - Correct answer: C (3)

Q: *What does 23 % 10 evaluate to?*

Why: 23 divided by 10 is 2 with 3 left over; % returns the left-over 3.

Why the other options are wrong:

A (2) - 2 is the whole-number quotient, not the remainder.

B (2.3) - 2.3 is the decimal quotient 23 / 10.

D (23) - 23 would be a concatenation "23" - there are no strings here.

Q2.3 - Correct answer: B (10)

Q: *What does console.log("5" * 2); print? (Hint: * behaves differently from +.)*

Why: Unlike +, the * operator converts the string "5" to the number 5 and multiplies, printing 10.

Why the other options are wrong:

A ("55") - "55" describes what + would do with strings, not *.

C ("10") - The output is a number, printed without quotes.

D (An error) - JavaScript converts numeric-looking strings for arithmetic - no error.

Q2.4 - Correct answer: C (50)

Q: *After these statements, what is in score? const score = 50; // score = 60;*

Why: Careful reading: the reassignment line is a comment, so nothing happens and score stays 50.

Why the other options are wrong:

A (undefined) - score was assigned 50 when created.

B (A TypeError stops the program) - A commented-out assignment cannot raise errors.

D (60) - 60 is inside a comment; comments do nothing.

Q2.5 - Correct answer: C ((2 + 3) * 4)

Q: *Which expression uses parentheses to force addition before multiplication?*

Why: Parentheses group 2 + 3, so the addition happens first.

Why the other options are wrong:

A (2 * 3 + 4) - Multiplication naturally runs first here anyway.

B (2 + (3 * 4)) - These parentheses wrap the multiplication, which already had priority.

D (2 + 3 * 4) - Here multiplication still runs first: 14.

Q2.6 - Correct answer: B (string)

Q: *What is the type of the value "true" (with quotes)?*

Why: Quotes make it text: the value is the four-character string "true", and typeof reports "string".

Why the other options are wrong:

A (boolean) - Without quotes, true would be a boolean.

C (undefined) - undefined belongs to never-assigned variables.

D (number) - Only digits make numeric-looking strings; this is text.

Q2.7 - Correct answer: A (Go!)

Q: *What does this print? let msg = "Go"; msg = msg + "!"; console.log(msg); (assume a normal semicolon)*

Why: msg + "!" concatenates to "Go!" and the print shows Go!.

Why the other options are wrong:

B (Go) - The reassignment changed msg before printing.

C ("Go!") - Quotes are syntax and are never printed.

D (An error) - Reassigning a let variable is legal.

Q2.8 - Correct answer: D (Descriptive names make code easier for humans to understand and maintain)

Q: *Why do programmers prefer descriptive names like remainingLives instead of short ones like x?*

Why: Code is read far more often than it is written; clear names prevent confusion for humans.

Why the other options are wrong:

A (Longer names make programs run faster) - Name length has no effect on execution speed.

B (Short names are illegal in JavaScript) - Short names like x are perfectly legal.

C (The computer requires at least 5 letters) - JavaScript imposes no minimum name length.

Q2.9 True - Assignment updates an existing let variable. If score was const, the assignment fails; if it never existed, it creates a global - a bad practice.

Q2.10 False - With + and a string on one side, JavaScript concatenates: the result is the string "105".

Q2.11 True - typeof reports the type name as a string, and quoted text is a string.

Q2.12 Sample answer: Sample answer: Assigning to a const throws a TypeError because const values cannot be reassigned. Fix: declare total with let if it must change, or compute a new const such as const newTotal = total + 5;

Q2.13 Sample answer: Sample answer: "7" is a string; 7 is a number. With +, "7" + 7 concatenates to "77", while 7 + 7 adds to 14.

Q2.14 One possible solution:

```
let rows = 12;
let columns = 8;
console.log(rows * columns);
console.log("Area: " + rows * columns);
```

Q2.15 One possible solution:

```
let number = 12345;
console.log(number % 10);
console.log((number - (number % 10)) / 10);
```

Chapter 3 - Working with Strings

Q3.1 - Correct answer: A (`)

Q: *Which character starts and ends a template literal?*

Why: Backticks wrap template literals and allow ${...} placeholders.

Why the other options are wrong:

B (") - Double quotes make a normal string.

C (') - Single quotes make a normal string.

D (/) - The slash starts comments or regex, not strings.

Q3.2 - Correct answer: B (tab tab (with a tab gap))

Q: *What does "tab\ttab" look like when printed?*

Why: \t inserts a tab character between the words on the same line.

Why the other options are wrong:

A (tabttab) - The t is consumed by the escape; no literal t remains.

C (tab then a new line, then tab) - A new line requires \n.

D (An error) - Escape sequences are valid - no error.

Q3.3 - Correct answer: B (6)

Q: *What does "robots".length evaluate to?*

Why: r-o-b-o-t-s is six characters.

Why the other options are wrong:

A (5) - One short - count again.

C ("6") - length returns a number, not a string.

D (7) - One too many.

Q3.4 - Correct answer: C (2)

Q: *What does "school".indexOf("o") return?*

Why: Positions: s=0, c=1, h=2, o=3. The first o is at position 3.

Why the other options are wrong:

A (-1) - -1 means not found; o exists here.

B (1) - 1 is the c.

D (3) - Correct: the first o sits at position 3.

Q3.5 - Correct answer: C (true)

Q: *What does "abracadabra".includes("cad") return?*

Why: The letters c-a-d appear in the middle, so includes answers true.

Why the other options are wrong:

A ("cad") - includes answers with a boolean, not text.

B (4) - It never returns a position.

D (false) - Scan the middle: abra-cad-abra.

Q3.6 - Correct answer: A ("ef")

Q: *What does "abcdef".slice(-2) return?*

Why: Negative positions count from the end: -2 starts at e and runs to the end, giving "ef".

Why the other options are wrong:

B ("ab") - "ab" is the start of the string.

C ("de") - "de" would be slice(-3, -1).

D ("f") - "f" would be slice(-1).

Q3.7 - Correct answer: D ("I like tea")

Q: *What does "I like cake".replace("cake", "tea") return?*

Why: replace returns a new string with the first (only) match swapped.

Why the other options are wrong:

A ("I like cake") - That is the original, unchanged.

B ("I liketea") - replace keeps the surrounding spaces.

C (true) - replace returns a string, not a boolean.

Q3.8 - Correct answer: B (It removes spaces from both ends only)

Q: *Which statement about .trim() is TRUE?*

Why: trim cleans the two ends; inner spaces always survive.

Why the other options are wrong:

A (It removes ALL spaces everywhere in the string) - Inner spaces survive trim.

C (It converts spaces to tabs) - Tabs are untouched; nothing is converted.

D (It returns a number of spaces removed) - It returns the cleaned string, not a count.

Q3.9 True - The \\ escape pair produces one real backslash character when printed.

Q3.10 False - "A\nB" holds three characters (A, the newline, B), while "AB" holds only two.

Q3.11 True - charAt returns the character at the given position; position 0 is the first character.

Q3.12 Sample answer: Sample answer: \t inserts a tab, pushing the following text to the next tab stop, which lines up the second column on each row.

Q3.13 Sample answer: Sample answer: Inside double quotes, an apostrophe is a normal character and needs no escape. You must escape it (\') only when the string itself is wrapped in single quotes.

Q3.14 One possible solution:

```
let word = "javascript";
console.log(word.toUpperCase());
console.log(word.length);
console.log(word.slice(4));
```

Q3.15 One possible solution:

```
console.log("Quote: \"Learn\"\nTab\tseparated");
```

Chapter 4 - Numbers and Math

Q4.1 - Correct answer: B (3)

Q: *What does Math.round(3.49) return?*

Why: 3.49 is nearer to 3 than to 4, so round gives 3.

Why the other options are wrong:

A (0) - round never returns 0 for a value near 3.

C (3.5) - round returns whole numbers.

D (4) - 4 would need 3.5 or more.

Q4.2 - Correct answer: C (0)

Q: *What does Math.ceil(-0.5) return?*

Why: ceil rounds UP toward positive infinity: the next whole number above -0.5 is 0 (technically -0, which prints as 0).

Why the other options are wrong:

A (-1) - -1 is floor's answer.

B (-0.5) - ceil returns whole numbers.

D (1) - 1 skips past the number entirely.

Q4.3 - Correct answer: A (-0.2)

Q: *What does Math.min(4, 4.5, -0.2) return?*

Why: The smallest of the three is -0.2.

Why the other options are wrong:

B (0) - 0 is larger than -0.2.

C (4) - 4 is beaten by both others.

D (4.5) - 4.5 is the largest.

Q4.4 - Correct answer: C (25)
Q: *What does console.log(Math.pow(5, 2)); print?*

Why: pow(5, 2) means 5 squared: 25.
Why the other options are wrong:

A (7) - 7 is 5 + 2.

B (10) - 10 is 5 * 2.

D (52) - 52 is just the digits glued - not an operation here.

Q4.5 - Correct answer: B (It returns a decimal from 0 up to but never including 1)
Q: *Which statement about Math.random() is TRUE?*

Why: The half-open range [0, 1) is Math.random's contract.
Why the other options are wrong:

A (It returns the same value every time) - Each call returns a fresh value.

C (It returns whole numbers from 1 to 100) - Whole numbers require floor.

D (It can return exactly 1) - 1 itself is never returned.

Q4.6 - Correct answer: A (NaN)
Q: *What does Number("3.5 apples") return?*

Why: Number demands the WHOLE text be numeric; trailing junk makes it NaN. (parseFloat would give 3.5.)
Why the other options are wrong:

B (3.5) - Number is stricter than parseInt/parseFloat.

C ("3.5 apples") - The result is a number value or NaN, never the input text.

D (0) - No zero fallback exists.

Q4.7 - Correct answer: D ("9.9")
Q: *What does console.log((9.876).toFixed(1)); print?*

Why: toFixed(1) rounds to one decimal: 9.876 becomes "9.9" as a string.
Why the other options are wrong:

A ("9.8") - 9.8 would be truncation, not rounding.

B (9.9 as a number) - The value is a string, not a number.

C ("10.0") - Rounding to one decimal never reaches 10.0 here.

Q4.8 - Correct answer: A (round makes the end values half as likely, biasing the roll)
Q: *Why is Math.floor preferred over Math.round for the random-range recipe?*

Why: With round, 0 and the top value each get only half a slot - floor keeps every value equally likely.
Why the other options are wrong:

B (floor is faster to type) - Speed is not the reason.

C (floor works with negative ranges only) - floor handles any range.

D (round cannot be used with Math.random) - round does work with random - it is just biased.

Q4.9 False - floor rounds DOWN toward negative infinity: -2.5 goes to -3. Truncation would give -2.

Q4.10 True - Number demands the whole text be numeric, so "12abc" becomes NaN, and Number.isNaN reports true. (parseInt would give 12.)

Q4.11 True - Zero's square root is zero - a legal, exact case.

Q4.12 Sample answer: Sample answer: Math.floor(Math.random() * 30) + 1. random()*30 stretches to [0,30), floor makes a whole number 0-29, and +1 shifts to 1-30.

Q4.13 Sample answer: Sample answer: Floating-point sums can be off by tiny amounts (0.1+0.2 style), so exact equality may fail. Instead round with toFixed for display or compare with a small tolerance.

Q4.14 One possible solution:

```
let d1 = Math.floor(Math.random() * 6) + 1;
let d2 = Math.floor(Math.random() * 6) + 1;
console.log(d1);
```

```
console.log(d2);
console.log("Total: " + (d1 + d2));
```

Q4.15 One possible solution:

```
let raw = "97";
let value = Number(raw);
console.log(value + 3);
console.log(Number.isNaN(Number("abc")));
```

Chapter 5 - Making Decisions

Q5.1 - Correct answer: C (false)

Q: *What does 10 !== 10 evaluate to?*

Why: 10 is exactly equal to 10, so the "not strictly equal" test is false.

Why the other options are wrong:

A (10) - Comparisons return booleans.

B (NaN) - Nothing here is NaN.

D (true) - The values match, so !== fails.

Q5.2 - Correct answer: C (true)

Q: *What does console.log("b" > "a"); print?*

Why: Strings compare alphabetically, and b comes after a.

Why the other options are wrong:

A (An error) - String comparison is legal.

B ("b") - The result is boolean, not text.

D (false) - The letters really are in order.

Q5.3 - Correct answer: B (Grade: B)

Q: *In the grade chain (>=90 A, >=75 B, >=60 C), what does score = 75 print?*

Why: 90 fails, 75 succeeds: first-match branching picks B and skips the rest.

Why the other options are wrong:

A (Grade: A) - 75 is not >= 90.

C (Grade: C) - The chain stops at the first match.

D (Nothing) - One branch always runs.

Q5.4 - Correct answer: D (true)

Q: *What does true && !false evaluate to?*

Why: !false is true, and true && true is true.

Why the other options are wrong:

A (false) - The NOT flips the second side to true.

B (undefined) - No undefined appears.

C (An error) - Mixing ! with && is normal.

Q5.5 - Correct answer: A (false)

Q: *What does the expression 5 > 3 && 2 > 7 evaluate to?*

Why: The left side is true but 2 > 7 is false; && needs both.

Why the other options are wrong:

B (true) - One false side is enough to fail AND.

C (5) - Booleans result, not operands.

D (2) - No numbers leak out.

Q5.6 - Correct answer: A (6 or 7)

Q: *Which value of dayNumber prints "Weekend" in this switch? switch (dayNumber) { case 6: case 7: console.log("Weekend"); break; default: console.log("Weekday"); }*

Why: The grouped empty case 6 falls through into case 7's shared body on purpose.

Why the other options are wrong:

B (Only 6) - Grouping shares the body with 7.

C (Only 7) - Same sharing applies.

D (Any number) - default catches everything else.

Q5.7 - Correct answer: C (falsy)

Q: *What does this print? let n = 0; if (n) { console.log("truthy"); } else { console.log("falsy"); }*

Why: The number 0 is falsy in JavaScript, so the else branch runs.

Why the other options are wrong:

A (0) - The value itself is never printed.

B (An error) - Conditions accept any value.

D (truthy) - 0 converts to false in conditions.

Q5.8 - Correct answer: D (Fall-through can be used on purpose to share code between labels)

Q: *Which statement about switch fall-through is TRUE?*

Why: Deliberate grouped cases are a known pattern; accidental fall-through is the bug.

Why the other options are wrong:

A (Fall-through always causes a SyntaxError) - It is legal, silent behavior.

B (Fall-through skips the default case automatically) - default runs on no match, unrelated to fall-through.

C (Fall-through happens only with string cases) - Fall-through ignores the case value type.

Q5.9 False - First-match branching runs only the FIRST true block, then skips the rest of the chain.

Q5.10 True - When used as conditions, 0 and "" convert to false - two famous falsy values.

Q5.11 False - Case labels are exact matches against one value; ranges need if / else if.

Q5.12 Sample answer: Sample answer: age >= 12 || height >= 150 - OR because either requirement alone qualifies.

Q5.13 Sample answer: Sample answer: Without braces, only the next single statement belongs to the if. Adding a second line later silently leaves it outside the branch, making it run unconditionally - a confusing bug braces prevent.

Q5.14 One possible solution:

```
let password = "abc-defg";
if (password.length >= 8 &&
    password.includes("-")) {
    console.log("strong");
} else {
    console.log("weak");
}
```

Q5.15 One possible solution:

```
let day = "tue";
switch (day) {
    case "sat":
        console.log("Saturday");
        break;
    case "sun":
        console.log("Sunday");
        break;
    case "tue":
        console.log("Tuesday");
        break;
    default:
        console.log("unknown day");
}
```

Chapter 6 - Loops

Q6.1 - Correct answer: A (for (let i = 1; i <= 8; i++))

Q: *Which line correctly starts a loop that runs exactly 8 times?*

Why: Starting at 1 with i <= 8 gives exactly the rounds 1 through 8.

Why the other options are wrong:

B (for (let i = 0; i <= 8; i++)) - 0 to 8 inclusive is nine rounds.

C (for (let i = 1; i < 8; i++)) - 1 to 7 is seven rounds.

D (for (let i = 8; i >= 1; i++)) - Missing the -1 update would run forever; with it, the order is backwards for counting up.

Q6.2 - Correct answer: C (10 7)

Q: *What does this print? let x = 10; while (x > 5) { console.log(x); x = x - 3; }*

Why: 10 prints and becomes 7; 7 prints and becomes 4; then 4 > 5 fails.

Why the other options are wrong:

A (10 8 6) - The steps subtract 3, not 2.

B (Nothing) - The condition is true at the start.

D (10 7 4) - 4 never prints because the loop stops first.

Q6.3 - Correct answer: C (30)

Q: *What is the value of total after this loop? let total = 0; for (let i = 2; i <= 10; i += 2) { total = total + i; }*

Why: The even numbers 2+4+6+8+10 add up to 30.

Why the other options are wrong:

A (10) - 10 is only the final counter value.

B (20) - 20 skips the 10.

D (45) - 45 is the odd full sum 1..9 - wrong range.

Q6.4 - Correct answer: B (1 3 5)

Q: *What does this loop print? for (let i = 1; i <= 6; i++) { if (i % 2 === 0) { continue; } console.log(i); }*

Why: Even rounds trigger continue before printing, so only odd values appear.

Why the other options are wrong:

A (1 3 5 6) - 6 is even, so it is skipped too.

C (1 2 3 4 5 6) - No - evens are skipped.

D (2 4 6) - That is the SKIP list, not the print list.

Q6.5 - Correct answer: D (Any for loop can be rewritten as a while loop and vice versa)

Q: *Which statement about for and while is TRUE?*

Why: They are interchangeable; the choice is about readability.

Why the other options are wrong:

A (for loops can count but while loops cannot) - while counts just fine with a manual update.

B (while loops always run at least once) - If the condition starts false, while runs zero times.

C (while loops cannot use break) - break works in every loop kind.

Q6.6 - Correct answer: C (10)

Q: *How many stars does this print in total? for (let r = 1; r <= 4; r++) { let line = ""; for (let c = 1; c <= r; c++) { line = line + "*"; } console.log(line); }*

Why: Row lengths 1 + 2 + 3 + 4 = 10 stars.

Why the other options are wrong:

A (4) - 4 is just the row count.

B (6) - 6 stops the sum one row early.

D (16) - 16 would be a full 4x4 square.

Q6.7 - Correct answer: B (The body never runs at all)

Q: *What happens when a condition is false at the very start of a while loop?*

Why: The check comes first, so zero rounds run - the loop is skipped entirely.

Why the other options are wrong:

A (JavaScript converts false to true) - No conversion happens.

C (The loop crashes) - False conditions are normal, not crashes.

D (The body runs once anyway) - That would be a do-while, not while.

Q6.8 - Correct answer: A (So each row starts empty instead of inheriting the last row's stars)

Q: *In the grid example, why does the inner loop rebuild line = "" every outer round?*

Why: Fresh row, fresh string - otherwise rows grow without bound.

Why the other options are wrong:

B (To comply with a language rule) - No rule forces it; logic does.

C (To make the stars different colors) - Colors are unrelated to variables.

D (Because console.log resets variables) - console.log prints; it resets nothing.

Q6.9 False - It exists only inside the loop; using it later gives an error or an undefined name.

Q6.10 True - break and continue are general loop-control statements, not tied to one loop kind.

Q6.11 True - Nesting multiplies: 3 x 7 = 21 inner executions.

Q6.12 Sample answer: Sample answer: The count is known and decreasing, so for (let i = 5; i >= 1; i--) matches the countdown, with console.log("Liftoff!") after the loop.

Q6.13 Sample answer: Sample answer: i <= 10 produces ten rounds; changing the boundary to i < 10 (or i <= 9) gives exactly nine. Off-by-one bugs live in the boundary operator.

Q6.14 One possible solution:

```
for (let i = 5; i <= 50; i += 5) {
    console.log(i);
}
```

Q6.15 One possible solution:

```
for (let r = 1; r <= 3; r++) {
    let line = "";
    for (let c = 1; c <= 3; c++) {
        line = line + "#";
    }
    console.log(line);
}
```

Chapter 7 - Functions

Q7.1 - Correct answer: A (Three times)

Q: *How many times does the body run? function f() { console.log("once"); } f(); f(); f();*

Why: Three calls, three runs - definitions do not count as runs.

Why the other options are wrong:

B (Zero) - Calls exist, so it runs.

C (Once) - Three calls, not one.

D (Forever) - Nothing loops here.

Q7.2 - Correct answer: B (Hi Sam *)

Q: *What does this print? function greet(name, emoji) { console.log("Hi " + name + " " + emoji); } greet("Sam", "*");*

Why: Arguments match by position: name gets "Sam", emoji gets "*".

Why the other options are wrong:

A (Hi * Sam) - Order is preserved, not reversed.

C (Hi Sam) - The emoji argument is included.

D (An error) - Two arguments fit two parameters.

Q7.3 - Correct answer: D (return total;)

Q: *A function needs to give its computed total back to the caller. Which statement belongs at the end?*

Why: return is how functions hand values back.

Why the other options are wrong:

A (console.log(total);) - That shows the value to humans only.

B (break total;) - break belongs to loops and switch.

C (export total;) - export belongs to modules - Chapter 10.

Q7.4 - Correct answer: C (4.5)

Q: *What does this print? const half = (n) => n / 2; console.log(half(9));*

Why: 9 / 2 is 4.5 - JavaScript numbers handle decimals natively.

Why the other options are wrong:

A (5) - No rounding happens in plain division.

B (undefined) - The arrow function returns a number.

D (4) - 4 would be Math.floor's answer.

Q7.5 - Correct answer: B (Only inside that function)

Q: *Where is the variable counter (declared with let inside a function) visible?*

Why: Function scope: born at declaration, gone when the function returns.

Why the other options are wrong:

A (Everywhere in the program) - That describes globals.

C (Only inside loops) - Loops have their own block scope, unrelated.

D (Everywhere after the function runs) - Locals do not survive the call.

Q7.6 - Correct answer: A (A description plus @param lines for each input and @return for the output)

Q: *What belongs directly above a function in a JSDoc comment?*

Why: JSDoc documents purpose, inputs, and output in one standard format.

Why the other options are wrong:

B (A list of every variable in the program) - Too much detail - describe THIS function.

C (The function's code, copied as text) - The code is already right there below the comment.

D (Nothing - JSDoc goes inside the body) - The comment sits above the definition.

Q7.7 - Correct answer: C (The text of the error, such as "Cannot divide by zero")

Q: *What does err.message contain inside a catch block?*

Why: err is the error object; its message property carries the human-readable description.

Why the other options are wrong:

A (The line number of the catch block) - Line numbers live elsewhere in the error object.

B (The program's output so far) - Output history is not stored in errors.

D (The name of the catch variable) - The variable name is whatever you chose.

Q7.8 - Correct answer: D (function area(w, h) { return w * h; })

Q: *Which function is best designed?*

Why: One clear job, inputs via parameters, output via return - the model function.

Why the other options are wrong:

A (function work() { /* prints AND returns AND changes a global */ }) - Mixing duties makes functions unpredictable.

B (function calc() { return 42; } // ignores its inputs) - Ignoring inputs is suspicious design.

C (function f(a, b, c, d, e, f2) { /* does everything */ }) - Doing everything is the opposite of design.

Q7.9 False - Without parentheses the name refers to the function itself; nothing runs.

Q7.10 False - Arrow functions take parameters in parentheses exactly like regular functions: (a, b) => a + b.

Q7.11 True - Throwing inside try is exactly the flow catch is designed for.

Q7.12 Sample answer: Sample answer: The parameter makes one function serve many callers: greet("Maya"); greet("Omar"); - no duplicated functions, and the message adapts to the data.

Q7.13 Sample answer: Sample answer: Printing as a side effect surprises callers who wanted silent computation (or silent display). A calculation used inside expressions should only return; a user-facing reporter should only print.

Q7.14 One possible solution:

```
function celsiusToF(c) {
    return c * 9 / 5 + 32;
}
console.log(celsiusToF(0));
console.log(celsiusToF(37));
console.log(celsiusToF(100));
```

Q7.15 One possible solution:

```
function safeDivide(a, b) {
    if (b === 0) {
        throw new Error("divide by zero");
    }
    return a / b;
}
try {
    console.log(safeDivide(9, 3));
    console.log(safeDivide(9, 0));
} catch (err) {
    console.log("Caught: " + err.message);
}
```

Chapter 8 - Arrays

Q8.1 - Correct answer: B (3)

Q: *What does [7, 8, 9].length evaluate to?*

Why: Three items, length 3.

Why the other options are wrong:

A (2) - 2 is the last index.

C (8) - Values do not set the length.

D (9) - 9 is a value, not the count.

Q8.2 - Correct answer: B (["a", "b", "c"])

Q: *After let s = ["a"]; s.push("b"); s.push("c"); what is s?*

Why: push appends in call order: b then c after a.

Why the other options are wrong:

A (["b", "c"]) - The "a" remains at the front.

C (["c", "b", "a"]) - Nothing reverses the list.

D (["a", "c"]) - Nothing was removed.

Q8.3 - Correct answer: D ("z")

Q: *What does let last = ["x", "y", "z"].pop(); store in last?*

Why: pop removes and returns the final item: "z".

Why the other options are wrong:

A ("x") - "x" is the first item.

B (3) - 3 is what push would return.

C (undefined) - The item really exists.

Q8.4 - Correct answer: A (Both A and B work)

Q: *Which expression tests whether "cat" is anywhere in pets?*

Why: includes answers directly; indexOf >= 0 is the classic equivalent.

Why the other options are wrong:

B (pets.indexOf("cat") >= 0) - Works, but it is the long way.

C (pets.includes("cat")) - Works too - hence "both".

D (cat in pets) - The in operator tests object keys, not array values.

Q8.5 - Correct answer: B ([1, 2, 3])

Q: *What does [3, 1, 2].sort((a, b) => a - b) return?*

Why: The comparator subtracts, sorting numbers ascending.

Why the other options are wrong:

A (undefined) - sort returns the (same) array.

C ([3, 2, 1]) - That is the b - a comparator.

D ([3, 1, 2]) - The comparator forces a change.

Q8.6 - Correct answer: C (["c", "d"])

Q: *What does ["a", "b", "c", "d"].slice(2) return?*

Why: slice(2) copies from index 2 to the end.

Why the other options are wrong:

A (["b", "c"]) - That is slice(1, 3).

B (["d"]) - That is slice(3).

D (["a", "b"]) - That is slice(0, 2).

Q8.7 - Correct answer: D (for (let [i, p] of pens.entries()))

Q: *Which loop prints every item with its position, like "0: pen"?*

Why: entries() supplies the index-value pair.

Why the other options are wrong:

A (for (let p of pens)) - Values only - no index.

B (for (let i of pens.length)) - Looping over a number errors.

C (pens.forEach(pen)) - forEach can get indexes with (item, index) - but as written, it only receives values.

Q8.8 - Correct answer: A ("red & green")

Q: *What does ["red", "green"].join(" & ") produce?*

Why: The separator string sits between the joined items.

Why the other options are wrong:

B ("redgreen") - The separator was specified, so it appears.

C (["red", " & ", "green"]) - join returns one string, not an array.

D ("red&&green") - The separator is used exactly once between items.

Q8.9 False - includes answers true/false. indexOf returns the index (or -1).

Q8.10 True - Arrays mix value types freely, though tidy programs usually keep one kind per list.

Q8.11 False - sort modifies the array in place and returns a reference to it. Copy with slice() first when the original order matters.

Q8.12 Sample answer: Sample answer: for...of with entries() plus (i + 1), or a counted for loop - both expose the index needed for human numbering; plain for...of cannot.

Q8.13 Sample answer: Sample answer: list[list.length] is one past the end and returns undefined. The correct recipe is list[list.length - 1].

Q8.14 One possible solution:

```
let nums = [];
nums.push(5);
nums.push(10);
nums.push(15);
console.log(nums);
console.log(nums.length);
console.log(nums[1]);
```

Q8.15 One possible solution:

```
let letters = ["d", "a", "c", "b"];
let sorted = letters.slice().sort();
console.log(sorted);
console.log(letters);
```

Chapter 9 - Objects

Q9.1 - Correct answer: B (5)

Q: *What does this print? let o = { a: 1, b: 2 }; o.c = 3; console.log(o.b + o.c);*

Why: b is 2 and the newly added c is 3: 2 + 3 = 5.

Why the other options are wrong:

A (3) - 3 is only o.c - add o.b as well.

C (23) - 23 glues text; these are numbers.

D (undefined) - Both properties exist by print time.

Q9.2 - Correct answer: A (Keys are usually names, read with dot or bracket access)

Q: *Which statement about object keys is TRUE?*

Why: Keys name their values; both access styles read them.

Why the other options are wrong:

B (Keys must be numbers) - Keys are usually text names.

C (Keys cannot be added after creation) - Objects grow on demand.

D (Keys must match their values) - Key and value are independent.

Q9.3 - Correct answer: D (user: adm and port: 80)

Q: *What does this loop print? let conf = { user: "adm", port: 80 }; for (let k in conf) { console.log(k + ": " + conf[k]); }*

Why: The loop visits keys; conf[k] reads each value.

Why the other options are wrong:

A (adm and 80) - That skips the key names.

B (user and port) - That skips the values.

C (An error) - for...in is legal on objects.

Q9.4 - Correct answer: A ("B")

Q: *For roster = [{n: "A", s: 9}, {n: "B", s: 7}];, what does roster[1].n return?*

Why: Index 1 is the second record; .n reads its name.

Why the other options are wrong:

B ("A") - "A" lives in the first record.

C (9) - 9 is that record's s.

D (undefined) - The property exists.

Q9.5 - Correct answer: C (Kira!)

Q: *What does this print? let hero = { name: "Kira", show() { return this.name + "!"; } }; console.log(hero.show());*

Why: this inside show() is hero, so this.name is "Kira".

Why the other options are wrong:

A (hero!) - The variable name is not the object.

B (undefined!) - The method returns a real string.

D (name!) - this resolves to the object, not the word name.

Q9.6 - Correct answer: D ('{"k": 1}')

Q: *Which JSON text is valid?*

Why: Double-quoted key, no trailing comma - proper JSON.

Why the other options are wrong:

A ("{'k': 1}") - Single quotes are illegal in JSON.

B ('{k: 1}') - Keys need quotes in JSON.

C ('{"k": 1,}') - Trailing commas are forbidden.

Q9.7 - Correct answer: B ("string")

Q: *After let t = JSON.stringify(o); what is typeof t?*

Why: stringify produces text - a string.

Why the other options are wrong:

A ("object") - The object was converted away.

C ("json") - "json" is not a JavaScript type.

D ("undefined") - The conversion succeeded.

Q9.8 - Correct answer: D (Invalid JSON throws an error that would crash the program)
Q: *Why wrap JSON.parse in try / catch for outside data?*

Why: Malformed text throws; catching it lets you show a friendly message.
Why the other options are wrong:

A (Parsing is slow) - Speed is not the issue.

B (JSON files are always broken) - Only malformed ones break.

C (try / catch makes parsing faster) - Catching adds a tiny cost, not a gain.

Q9.9 True - Values can be anything - including arrays and even other objects.

Q9.10 True - Array indexes are keys too, so for...in visits them - though for...of or a counted loop is the clearer choice for arrays.

Q9.11 False - Properties whose value is undefined are dropped during stringify - a subtle but real rule.

Q9.12 Sample answer: Sample answer: this points at whichever object the method was called on, so the method keeps working for copies and renames of the object.

Q9.13 Sample answer: Sample answer: Save with JSON.stringify(data, null, 2) to turn data into text, write the text to a file; load with JSON.parse(fileText) to rebuild the live object.

Q9.14 One possible solution:

```
let movie = {
    title: "Up",
    year: 2009,
    ratings: [5, 4, 5]
};
console.log(movie.title);
let total = 0;
for (let r of movie.ratings) {
    total = total + r;
}
console.log(total / movie.ratings.length);
```

Q9.15 One possible solution:

```
let text = '{"city": "Cairo",
    "population": 10000000}';
let data = JSON.parse(text);
console.log(data.population.toLocaleString());
```

Chapter 10 - Files, Modules, and Real Programs

Q10.1 - Correct answer: B (A module can export several things at once)
Q: *Which statement about modules is TRUE?*

Why: export { a, b, c } shares as much as you label.
Why the other options are wrong:

A (export works only with functions) - Any value can be exported.

C (Import must grab everything a module offers) - Importers choose only what they need.

D (Built-in modules need the ./ prefix) - ./ is for YOUR files; built-ins use node:.

Q10.2 - Correct answer: D (How bytes become characters)
Q: *What does "utf-8" control?*

Why: Encoding is the byte-to-character dictionary.
Why the other options are wrong:

A (The file's size limit) - No size is set by encoding.

B (The sort order of lines) - Sorting is a program decision.

C (The file's permissions) - Permissions are an operating system matter.

Q10.3 - Correct answer: A (A then B, on two lines)
Q: *After these calls, what does log.txt contain? writeFileSync("log.txt", "A\n", "utf-8"); appendFileSync("log.txt", "B\n", "utf-8");*

Why: The write creates A; the append adds B after it.
Why the other options are wrong:
B (B only) - The append adds after, not instead.
C (A only) - The append succeeded too.
D (An error) - Both calls are valid.

Q10.4 - Correct answer: B (readFileSync inside try / catch with a fallback)
Q: *Which read is SAFEST for a file that may not exist?*

Why: Catch the error, report calmly, and use a default.
Why the other options are wrong:
A (readFileSync(name, "utf-8") with no protection) - A missing file would crash the program.
C (Reading it twice to be sure) - Reading twice reads nothing new.
D (Using writeFileSync first to create it) - Pre-creating with write would overwrite data!

Q10.5 - Correct answer: C (A relative path in the same folder)
Q: *What does the ./ in "./logData.js" signal?*

Why: ./ anchors the path to the current folder.
Why the other options are wrong:
A (A hidden file) - Hidden files start with dots in names, not paths.
B (A TypeScript file) - The extension here is .js.
D (A built-in module) - Built-ins use node:.

Q10.6 - Correct answer: A (The data rebuilt as live objects/arrays)
Q: *What does JSON.parse(readFileSync("d.json", "utf-8")) give you?*

Why: Read returns text; parse revives the data.
Why the other options are wrong:
B (The raw file text) - parse already converted the text.
C (The number of properties) - You can count properties afterward.
D (A promise) - No promises - readFileSync is synchronous.

Q10.7 - Correct answer: D (It normally runs cleanup whether the try worked or the catch handled an error)
Q: *Why is finally useful around file operations?*

Why: finally is the always-tidy-up block (normally - a killed process skips it).
Why the other options are wrong:
A (It retries the file operation) - No retries happen automatically.
B (It deletes the file) - Deleting files is your own code's job.
C (It makes errors disappear) - Errors are handled in catch.

Q10.8 - Correct answer: A (Small modules by job plus a main file that wires them together)
Q: *Which program structure matches the lesson's recommendation?*

Why: Small focused modules plus one conductor scale best.
Why the other options are wrong:
B (One giant file with everything) - Giants are hard to navigate.
C (Every function in its own folder) - Folders per function is overkill.
D (No main file at all) - Something must coordinate.

Q10.9 **True** - Without an encoding, Node hands back raw bytes - which is why "utf-8" belongs on every text read in this book.

Q10.10 **False** - Appends always land at the END, in call order.

Q10.11 **True** - File errors happen during execution - exactly what catch is for.

Q10.12 **Sample answer:** Sample answer: \n. Without a trailing newline, entries glue together into one long line instead of one entry per line.

Q10.13 **Sample answer:** Sample answer: write replaces the whole file (good for saving a fresh complete version); append adds to the end (good for logs and diaries that grow).

Q10.14 **One possible solution:**

```
import { writeFileSync, readFileSync }
    from "node:fs";

writeFileSync("numbers.json",
    JSON.stringify([3, 6, 9]), "utf-8");
let loaded = JSON.parse(
    readFileSync("numbers.json", "utf-8"));
console.log(loaded[1]);
```

Q10.15 **One possible solution:**

```
import { readFileSync } from "node:fs";

function loadOrEmpty(name) {
    try {
        return JSON.parse(
            readFileSync(name, "utf-8"));
    } catch (err) {
        console.log("Starting fresh: " +
            err.message);
        return [];
    }
}
```

Challenge Solutions

Each solution shows the thinking steps first, then the complete program. Try the challenge yourself before reading - a solution understood is worth ten copied.

Chapter 1 - Hello, JavaScript

Challenge 1.1 - solution

1. Plan the four output lines: 3, 2, 1, Go!.
2. Write one console.log statement per line, in order.
3. Add a comment above the block explaining that statements run top to bottom, so swapping lines would swap the countdown.

```
// Lines run top to bottom, so the countdown prints in
// the exact order written here.
console.log(3);
console.log(2);
console.log(1);
console.log("Go!");
```

Challenge 1.2 - solution

1. Read line by line and separate real statements from comments.
2. Write the output of each real statement in order.
3. Check that commented lines contribute nothing.

```
// The program being analyzed:
console.log("Start");
// console.log("Hidden");
/* multi-line
   comment */
console.log("End");
// Output: Start, then End. Two comments print nothing.
```

Challenge 1.3 - solution

1. Line 1: Console.log has a capital C - change to console.log.
2. Line 2: consle.log is misspelled - change to console.log.
3. Line 3: the closing quote is missing - add it before the semicolon.

```
// Broken version:
// Console.log("A");
// consle.log("B");
// console.log("C);
// Fixed version:
console.log("A");
console.log("B");
console.log("C");
```

Challenge 1.4 - solution

1. Decide the picture rows: "*", "**", "***".
2. Write one console.log per row, in top-to-bottom order.
3. Add a comment naming the picture for the reader.

```
// A small triangle drawn with text rows
console.log("*");
console.log("**");
console.log("***");
```

Challenge 1.5 - solution

1. Write four console.log statements for the routine (wake up, brush teeth, eat breakfast, pack bag).
2. Copy the program and swap, for example, lines 2 and 3.
3. Compare outputs: the printed order swaps in the same way, because execution is strictly top to bottom.

```
// Version A
console.log("Wake up");
console.log("Brush teeth");
console.log("Eat breakfast");
console.log("Pack bag");
// Version B swaps lines 2 and 3:
// Wake up, Eat breakfast, Brush teeth, Pack bag
```

Chapter 2 - Variables and Values

Challenge 2.1 - solution

1. Create the temp variable and copy a into it.
2. Copy b into a.
3. Copy temp (the old a) into b.
4. Print a and b to verify the swap.

```
let a = 1;
let b = 2;
let temp = a;
a = b;
b = temp;
console.log(a);
console.log(b);
```

Challenge 2.2 - solution

1. Hundreds digit: Math.floor(987 / 100).
2. Ones digit: 987 % 10.
3. Tens digit: remove the ones (987 - 987 % 10 = 980), divide by 10 to get 98, then take % 10 to get 8.

```
let number = 987;
let hundreds = Math.floor(number / 100);
let ones = number % 10;
let tens = Math.floor(number / 10) % 10;
console.log(hundreds);
console.log(tens);
console.log(ones);
```

Challenge 2.3 - solution

1. Compute hours = Math.floor(137 / 60).
2. Compute minutes = 137 % 60.
3. Print both with clear labels.

```
const TOTAL = 137;
let hours = Math.floor(TOTAL / 60);
let minutes = TOTAL % 60;
console.log(hours + " hours");
console.log(minutes + " minutes");
```

Challenge 2.4 - solution

1. Build the string piece by piece.
2. Insert quantity, the "x", the item name, and the computed total.
3. Run it and compare character by character with the target line.

```
let item = "Notebook";
let price = 4.5;
let quantity = 3;
console.log(quantity + " x " + item + " = " +
    price * quantity);
```

Challenge 2.5 - solution

1. Line 1: pure number addition.
2. Line 2: string concatenation of digits.
3. Line 3: left-to-right evaluation decides addition vs joining.
4. Line 4: typeof reveals the type of the "mystery" value.

```
console.log(6 + 6);
console.log("6" + "6");
console.log(1 + 2 + "3");
console.log(typeof (1 + 2 + "3"));
// Expected: 12, "66", "33", "string"
```

Chapter 3 - Working with Strings

Challenge 3.1 - solution

1. Get the first initial with charAt(0).
2. Find the space with indexOf(" ").
3. The last initial is charAt(spaceIndex + 1).
4. Print them joined with dots.

```
let name = "Ada Lovelace";
let first = name.charAt(0);
let space = name.indexOf(" ");
let last = name.charAt(space + 1);
console.log(first + "." + last + ".");
```

Challenge 3.2 - solution

1. Start a counter at 1.
2. Walk through positions with a loop - or, simpler for now, count the spaces by hand into a constant.
3. Print counter. (A loop version arrives in Chapter 6.)

```
const SPACES = 4;  // counted in the sentence
let words = SPACES + 1;
console.log(words);  // 5
```

Challenge 3.3 - solution

1. Write the sentence as one string.
2. Swap each space for \n.
3. Print with a single console.log.

```
console.log("escape\nsequences\nare\nfun");
```

Challenge 3.4 - solution

1. Trim the edges.
2. Chain toUpperCase on the trimmed result.
3. Store or print the final value.

```
let title = "  the HOBBIT  ";
let tidy = title.trim().toUpperCase();
console.log(tidy);
```

Challenge 3.5 - solution

1. Take each needed piece with slice or charAt.
2. Assemble: s + s + e + r + t + s + e + d backwards.
3. Print the rebuilt string.

```
let word = "stressed";
let back = word.charAt(7) + word.charAt(6) +
    word.charAt(5) + word.charAt(4) + word.charAt(3) +
    word.charAt(2) + word.charAt(1) + word.charAt(0);
console.log(back);  // "desserts"
```

Chapter 4 - Numbers and Math

Challenge 4.1 - solution

1. Compute used = 4521.7 - 4370.2.
2. Print used.toFixed(1).
3. Compute cost = used * 1.25 and print cost.toFixed(2).

```
let now = 4521.7;
let before = 4370.2;
let used = now - before;
let cost = used * 1.25;
console.log(used.toFixed(1) + " units");
console.log(cost.toFixed(2) + " cost");
```

Challenge 4.2 - solution

1. hours = Math.floor(5000 / 3600).
2. leftover = 5000 % 3600.
3. minutes = Math.floor(leftover / 60); seconds = leftover % 60.

```
const TOTAL = 5000;
let hours = Math.floor(TOTAL / 3600);
let leftover = TOTAL % 3600;
let minutes = Math.floor(leftover / 60);
let seconds = leftover % 60;
console.log(hours + "h " + minutes + "m " +
    seconds + "s");
```

Challenge 4.3 - solution

1. Build the expression once and reuse it.
2. Print ten labeled lines (ten console.log calls for now - loops arrive next chapter).
3. Check every line shows 0 or 1 only.

```
console.log("Flip 1: " + Math.floor(Math.random() * 2));
console.log("Flip 2: " + Math.floor(Math.random() * 2));
// ... repeat up to Flip 10 ...
console.log("Flip 10: " + Math.floor(Math.random() * 2));
// Each line is 0 (heads) or 1 (tails).
```

Challenge 4.4 - solution

1. Line 1: round to nearest.
2. Line 2: floor a negative - down means more negative.
3. Line 3: ceil a negative - up means toward zero.
4. Line 4: trunc chops toward zero.
5. Line 5: abs removes the sign.

```
console.log(Math.round(2.5));   // 3
console.log(Math.floor(-2.5));  // -3
console.log(Math.ceil(-2.5));   // -2
console.log(Math.trunc(-2.9));  // -2
console.log(Math.abs(-2.5));    // 2.5
```

Challenge 4.5 - solution

1. Store the number and parse the string.
2. Compute Math.abs(a - b).
3. Compare the difference with < 0.001 and print the boolean.

```
let a = 87.654;
let b = parseFloat("87.654");
let diff = Math.abs(a - b);
console.log(diff < 0.001);  // true
```

Chapter 5 - Making Decisions

Challenge 5.1 - solution

1. Test divisibility with the % operator.
2. Join the rules with && and || carefully.
3. Run the expression for each of the three years.

```
let year = 2024;
console.log((year % 4 === 0 && year % 100 !== 0) ||
    year % 400 === 0);          // true
year = 1900;
console.log((year % 4 === 0 && year % 100 !== 0) ||
    year % 400 === 0);          // false
year = 2000;
console.log((year % 4 === 0 && year % 100 !== 0) ||
    year % 400 === 0);          // true
```

Challenge 5.2 - solution

1. Compute bmi = 70 / (1.75 * 1.75).
2. Print bmi.toFixed(1).
3. Branch with the thresholds in increasing order.

```
let weight = 70;
let height = 1.75;
let bmi = weight / (height * height);
console.log(bmi.toFixed(1));
if (bmi < 18.5) {
    console.log("underweight");
} else if (bmi < 25) {
    console.log("healthy");
} else if (bmi < 30) {
    console.log("overweight");
} else {
    console.log("obese");
}
```

Challenge 5.3 - solution

1. Order the conditions: safe check first.
2. Compute 100 / x on the right side.
3. Call console.log with the full expression twice.

```
let x = 20;
console.log(x !== 0 && 100 / x > 4);  // true
x = 0;
console.log(x !== 0 && 100 / x > 4);  // false,
// and the division never runs
```

Challenge 5.4 - solution

1. Declare code and quantity.
2. switch on code, setting price in each case (with breaks).
3. After the switch, branch on whether price was set, then print the total.

```
let code = "B";
let quantity = 3;
let price = 0;
switch (code) {
    case "A": price = 3.5; break;
    case "B": price = 2.0; break;
    case "C": price = 1.5; break;
    default:
        console.log("Not on the menu");
}
if (price > 0) {
    console.log("Total: " + price * quantity);
    }
```

Challenge 5.5 - solution

1. Handle the below-zero case first.
2. Climb the bands with else if (t <= 15), (t <= 25), (t <= 35).
3. Let the final else catch "extreme".

```
let t = -3;
if (t < 0) {
    console.log("freezing");
} else if (t <= 15) {
    console.log("cold");
} else if (t <= 25) {
    console.log("mild");
} else if (t <= 35) {
    console.log("hot");
} else {
    console.log("extreme");
}
// Repeat with t = 20 (mild) and t = 40 (extreme)
```

Chapter 6 - Loops

Challenge 6.1 - solution

1. Loop 1 to 15.
2. Check i % 15 === 0 first and print FizzBuzz.
3. Else check % 3, then % 5, then print the plain number.

```
for (let i = 1; i <= 15; i++) {
    if (i % 15 === 0) {
        console.log("FizzBuzz");
    } else if (i % 3 === 0) {
        console.log("Fizz");
    } else if (i % 5 === 0) {
        console.log("Buzz");
    } else {
        console.log(i);
    }
}
```

Challenge 6.2 - solution

1. Start total at 0.
2. While n > 0: add n % 10 to total, then set n = Math.floor(n / 10).
3. Print the total.

```
let n = 4729;
let total = 0;
while (n > 0) {
    total = total + (n % 10);
    n = Math.floor(n / 10);
}
console.log(total);  // 22
```

Challenge 6.3 - solution

1. Assume prime = true.
2. Loop d from 2 to Math.floor(Math.sqrt(29)).
3. If 29 % d === 0, set prime = false and break.
4. Print the verdict after the loop.

```
let number = 29;
let isPrime = true;
for (let d = 2; d <= Math.floor(Math.sqrt(number));
        d++) {
    if (number % d === 0) {
        isPrime = false;
        break;
    }
}
console.log(number + " prime? " + isPrime);
```

Challenge 6.4 - solution

1. Outer loop r from 1 to 5.
2. Inner loop runs r times, gluing r onto line.
3. Print line at the end of each outer round.

```
for (let r = 1; r <= 5; r++) {
    let line = "";
    for (let c = 1; c <= r; c++) {
        line = line + r;
    }
    console.log(line);
}
```

Challenge 6.5 - solution

1. Loop i from 1 to 10 normally.
2. Print 11 - i to flip the order.
3. Print Liftoff! after the loop.

```
for (let i = 1; i <= 10; i++) {
    console.log(11 - i);
}
console.log("Liftoff!");
```

Chapter 7 - Functions

Challenge 7.1 - solution

1. Declare makeCounter with a local count = 0.
2. Return an arrow function that adds 1 to count and prints it.
3. Create two counters and call each a few times - they count independently.

```
function makeCounter() {
    let count = 0;
    return () => {
        count = count + 1;
        console.log(count);
    };
}
let a = makeCounter();
let b = makeCounter();
a(); a(); b(); a();
// prints 1, 2, 1, 3
```

Challenge 7.2 - solution

1. Check length first with .length.
2. Loop characters: numbers include digits (use >= "0" and <= "9" comparisons); use toUpperCase check for capitals.
3. Return true only when all three flags hold.

```
function isStrong(pw) {
    if (pw.length < 8) return false;
    let hasDigit = false;
    let hasUpper = false;
    for (let ch of pw) {
        if (ch >= "0" && ch <= "9") {
            hasDigit = true;
        }
        if (ch === ch.toUpperCase() &&
            ch !== ch.toLowerCase()) {
            hasUpper = true;
        }
    }
    return hasDigit && hasUpper;
}
console.log(isStrong("abc12345"));
console.log(isStrong("Abcdefg1"));
console.log(isStrong("short1A"));
```

Challenge 7.3 - solution

1. Guard with typeof checks, throwing the type message.
2. Guard the zero case.
3. Return a / b; wrap the calls in try / catch.

```
function divide(a, b) {
    if (typeof a !== "number" ||
        typeof b !== "number") {
        throw new Error("Inputs must be numbers");
    }
    if (b === 0) {
        throw new Error("Cannot divide by zero");
    }
    return a / b;
}
try {
    console.log(divide(9, 3));
    console.log(divide(9, 0));
    console.log(divide("x", 2));
} catch (err) {
    console.log("Caught: " + err.message);
}
```

Challenge 7.4 - solution

1. Define the three arrows.
2. switch on p, returning square(n), cube(n), or fourth(n).
3. default throws "unknown power".

```
const square = (n) => n * n;
const cube = (n) => n * n * n;
const fourth = (n) => n * n * n * n;
function power(n, p) {
    switch (p) {
        case 2: return square(n);
        case 3: return cube(n);
        case 4: return fourth(n);
        default:
            throw new Error("unknown power");
    }
}
console.log(power(3, 2));
console.log(power(3, 3));
console.log(power(2, 4));
```

Challenge 7.5 - solution

1. Print the entry line first.
2. Compute the result into a variable.
3. Print the leaving line, then return the variable.

```
function double(n) {
    console.log("double called with " + n);
    let result = n * 2;
    console.log("double returning " + result);
    return result;
}
console.log(double(5));
```

Chapter 8 - Arrays

Challenge 8.1 - solution

1. Start both trackers at -Infinity.
2. For each n: if n > best, shift best into second and update best; else if n > second, update second.
3. Print second.

```
let nums = [12, 45, 2, 41, 31, 8];
let best = -Infinity;
let second = -Infinity;
for (let n of nums) {
    if (n > best) {
        second = best;
        best = n;
    } else if (n > second) {
        second = n;
    }
}
console.log(second);  // 41
```

Challenge 8.2 - solution

1. Loop i from 0 to Math.floor(length / 2).
2. Compare list[i] with list[length - 1 - i].
3. Any mismatch breaks the loop; report the verdict.

```
let list = [1, 2, 3, 2, 1];
let ok = true;
for (let i = 0;
        i < Math.floor(list.length / 2); i++) {
    if (list[i] !== list[list.length - 1 - i]) {
        ok = false;
        break;
    }
}
console.log(ok ? "palindrome" : "not");
```

Challenge 8.3 - solution

1. Print the original.
2. let last = list.pop().
3. list.unshift(last) and print again.

```
let list = [1, 2, 3, 4, 5];
console.log(list);
let last = list.pop();
list.unshift(last);
console.log(list);
```

Challenge 8.4 - solution

1. Start result = [].
2. Loop over both arrays in turn.
3. Push only values that includes does not already know.

```
let a = [1, 2, 3];
let b = [3, 4, 5];
let result = [];
for (let n of a) {
    if (!result.includes(n)) {
        result.push(n);
    }
}
for (let n of b) {
    if (!result.includes(n)) {
        result.push(n);
    }
}
console.log(result);
```

Challenge 8.5 - solution

1. Write the comparison with the two standard recipes.
2. Run it on both arrays.
3. Confirm the expression never mentions a fixed number.

```
function sandwichish(arr) {
    return arr[0] === arr[arr.length - 1];
}
console.log(sandwichish(
    ["bread", "cheese", "bread"]));  // true
console.log(sandwichish(
    ["bread", "cheese", "ham"]));    // false
```

Chapter 9 - Objects

Challenge 9.1 - solution

1. Build the object literal inside the function.
2. average() loops this.scores, sums, divides by length.
3. Create two students and call average() on each.

```
function makeStudent(name, scores) {
    return {
        name: name,
        scores: scores,
        average: function () {
            let total = 0;
            for (let s of this.scores) {
                total = total + s;
            }
            return total / this.scores.length;
        }
    };
}
let a = makeStudent("Lina", [90, 80, 100]);
let b = makeStudent("Sam", [60, 70, 65]);
console.log(a.average());
console.log(b.average());
```

Challenge 9.2 - solution

1. Loop the array with for...of.
2. Print name, stock, and price per line.
3. Sum price * stock into a total; print it after the loop.

```
let products = [
    { name: "Pen", price: 2, stock: 100 },
    { name: "Book", price: 9, stock: 40 },
    { name: "Bag", price: 15, stock: 12 }
];
let total = 0;
for (let p of products) {
    console.log(p.name + " x" + p.stock +
        " - " + p.price);
    total = total + p.price * p.stock;
}
console.log("Total value: " + total);
```

Challenge 9.3 - solution

1. Start an empty result array.
2. Push each key during the for...in loop.
3. Return the array; print result.join(", ").

```
function keysOf(obj) {
    let result = [];
    for (let k in obj) {
        result.push(k);
    }
    return result;
}
let keys = keysOf({ a: 1, b: 2, c: 3 });
console.log(keys.join(", "));
```

Challenge 9.4 - solution

1. Spot the trailing comma after the last property.
2. Remove it in the fixed text.
3. Parse and print data.age.

```
let bad = '{"name": "Ada", "age": 36,}';
// JSON forbids the trailing comma.
let good = '{"name": "Ada", "age": 36}';
let data = JSON.parse(good);
console.log(data.age);
```

Challenge 9.5 - solution

1. Create the outer object with an inner array.
2. Print school.name.
3. Loop school.students and print name and grade per record.

```
let school = {
    name: "Nile School",
    students: [
        { name: "Maya", grade: "A" },
        { name: "Omar", grade: "B" }
    ]
};
console.log(school.name);
for (let s of school.students) {
    console.log(s.name + " - " + s.grade);
}
```

Chapter 10 - Files, Modules, and Real Programs

Challenge 10.1 - solution

1. Read the file with utf-8.
2. Split on "\n" to get an array of lines.
3. Count non-empty lines (or subtract the trailing empty one) and print.

```
import { readFileSync } from "node:fs";

let text = readFileSync("notes.txt", "utf-8");
let lines = text.split("\n");
let count = 0;
for (let line of lines) {
    if (line !== "") {
        count = count + 1;
    }
}
console.log(count + " lines");
```

Challenge 10.2 - solution

1. addEntry appends text + " - " + timestamp + "\n".
2. showEntries reads, splits on "\n".
3. Print non-empty lines with (i + 1) numbering via entries().

```
import { appendFileSync, readFileSync }
    from "node:fs";

function addEntry(text) {
    let stamp = new Date().toLocaleString();
    appendFileSync("diary.txt",
        text + " - " + stamp + "\n", "utf-8");
}
function showEntries() {
    let text = readFileSync("diary.txt",
        "utf-8");
    for (let [i, line] of
            text.split("\n").entries()) {
        if (line !== "") {
            console.log((i + 1) + ". " + line);
        }
    }
}
addEntry("Started the diary");
showEntries();
```

Challenge 10.3 - solution

1. Load with a try / catch fallback to [].
2. push 97 and write the stringified array back.
3. Loop to find the maximum and print it.

```
import { readFileSync, writeFileSync }
    from "node:fs";

let scores = [];
try {
    scores = JSON.parse(
        readFileSync("scores.json", "utf-8"));
} catch (err) {
    console.log("Fresh start.");
}
scores.push(97);
writeFileSync("scores.json",
    JSON.stringify(scores), "utf-8");
let best = scores[0];
for (let s of scores) {
    if (s > best) {
        best = s;
    }
}
console.log("Best: " + best);
```

Challenge 10.4 - solution

1. Write stats.js with the function and export.
2. Write main.js with the import from "./stats.js".
3. Run main.js and confirm 100.

```
// stats.js
function totalMinutes(list) {
    let total = 0;
    for (let s of list) {
        total = total + s;
    }
    return total;
}
export { totalMinutes };

// main.js
import { totalMinutes as tm } from "./stats.js";
console.log(tm([30, 45, 25]));  // tm = totalMinutes
```

Challenge 10.5 - solution

1. Read the text successfully.
2. Wrap JSON.parse in its own try / catch.
3. Fall back to {} and print a recovery message.

```
import { readFileSync } from "node:fs";

let text = readFileSync("data.json", "utf-8");
let data;
try {
    data = JSON.parse(text);
} catch (err) {
    console.log("Damaged save: " + err.message);
    data = {};
}
console.log("Recovered with:", data);
```

Index

Topics, functions, and keywords with the lessons that teach them. "Ch 3, L2" means Chapter 3, Lesson 2.

Topic	
A	
abs (absolute value)	Ch 4, L1
and operator (&&)	Ch 5, L3
appendFileSync	Ch 10, L3
arguments	Ch 7, L2
arguments vs parameters	Ch 7, L2
arithmetic operators	Ch 2, L2
Array.isArray	Ch 8, L1
arrays	Ch 8, L1
arrays of objects	Ch 9, L2
arrow functions	Ch 7, L3
assignment (=)	Ch 2, L1
B	
backticks (template literals)	Ch 3, L1
booleans	Ch 2, L4
bracket access (objects)	Ch 9, L1
break	Ch 6, L3
built-in functions	Ch 1, L2
C	
callback	Ch 8, L5
camelCase names	Ch 2, L4
case sensitivity	Ch 1, L2

Topic	
catch	Ch 7, L4
ceil (rounding up)	Ch 4, L1
charAt	Ch 3, L4
class_ (naming around keywords)	Ch 9, L2
code blocks { }	Ch 5, L2
comments (// and /* */)	Ch 1, L4
comparator for sort	Ch 8, L2
comparisons (=== !== < >)	Ch 5, L1
concatenation (+)	Ch 2, L3
console.log	Ch 1, L2
const	Ch 2, L1
continue	Ch 6, L3
current working directory	Ch 10, L2
D	
dangling else / braces rule	Ch 5, L2
destructuring [i, item]	Ch 8, L3
destructuring objects { title }	Ch 9, L5
dot access (objects)	Ch 9, L1
E	
else / else if	Ch 5, L2
entries()	Ch 8, L3
errors, runtime vs syntax	Ch 7, L4
escape sequences (\n, \t)	Ch 3, L2
export	Ch 10, L1
expressions	Ch 2, L2
F	
false and true	Ch 2, L4
filter	Ch 8, L5
finally	Ch 7, L4
floating-point precision	Ch 4, L2
for loop	Ch 6, L2
for...in	Ch 9, L2
for...of	Ch 8, L3
function declarations	Ch 7, L1
functions	Ch 7, L1
G	
global variables	Ch 7, L3
H	
hoisting of function declarations	Ch 7, L2

Topic	
I	
if statements	Ch 5, L2
import	Ch 10, L1
includes (arrays)	Ch 8, L4
includes (strings)	Ch 3, L4
index (positions)	Ch 8, L1
Index - indexes start at 0	Ch 8, L1
indexOf (arrays)	Ch 8, L4
indexOf (strings)	Ch 3, L3
infinite loops	Ch 6, L1
J	
join	Ch 8, L4
JSDoc comments	Ch 7, L4
JSON.parse	Ch 9, L4
JSON.stringify	Ch 9, L4
L	
length property (arrays)	Ch 8, L1
length property (strings)	Ch 3, L3
let	Ch 2, L1
local variables	Ch 7, L3
logical operators (&& || !)	Ch 5, L3
loops	Ch 6, L1
M	
map	Ch 8, L5
Math object	Ch 4, L1
Math.abs	Ch 4, L1
Math.ceil	Ch 4, L1
Math.floor	Ch 4, L1
Math.max / Math.min	Ch 4, L2
Math.pow	Ch 4, L1
Math.random	Ch 4, L3
Math.round	Ch 4, L1
Math.sqrt	Ch 4, L1
methods (object functions)	Ch 9, L3
modules	Ch 10, L1
N	
NaN	Ch 4, L4
nested loops	Ch 6, L4
Node.js	Ch 1, L2

Topic	
not operator (!)	Ch 5, L3
Number() conversion	Ch 4, L4
Number.isNaN	Ch 4, L4
numbers	Ch 2, L2
O	
Object.entries	Ch 9, L5
Object.keys	Ch 9, L5
Object.values	Ch 9, L5
objects	Ch 9, L1
off-by-one errors	Ch 6, L2
optional chaining (? .)	Ch 9, L5
or operator (||)	Ch 5, L3
P	
parameters	Ch 7, L2
parseInt / parseFloat	Ch 4, L4
pop	Ch 8, L2
precedence of operators	Ch 2, L2
process.cwd()	Ch 10, L2
prompt() (browser)	Mini-Project 2
push	Ch 8, L2
R	
random numbers in a range	Ch 4, L3
readFileSync	Ch 10, L2
reduce	Ch 8, L5
remainder operator (%)	Ch 2, L2
replace	Ch 3, L4
return	Ch 7, L2
rounding functions	Ch 4, L1
S	
scope	Ch 7, L3
shift / unshift	Ch 8, L2
short-circuit evaluation	Ch 5, L3
slice (arrays)	Ch 8, L4
slice (strings)	Ch 3, L4
sort (arrays)	Ch 8, L2
splice	Ch 8, L5
statements, order of	Ch 1, L3
string immutability	Ch 3, L4
string methods	Ch 3, L3

Topic	
strings	Ch 2, L3
switch	Ch 5, L4
T	
template literals	Ch 3, L1
ternary operator (?:)	Ch 8, L5
this	Ch 9, L3
throw	Ch 7, L4
toFixed	Ch 4, L2
toLocaleString	Ch 9, L4
toLowerCase / toUpperCase	Ch 3, L3
trim	Ch 3, L3
true / false values	Ch 2, L4
truthy and falsy values	Ch 5, L3
typeof	Ch 2, L4
U	
undefined	Ch 2, L4
utf-8 encoding	Ch 10, L2
V	
variables	Ch 2, L1
W	
while loop	Ch 6, L1
writeFileSync	Ch 10, L3

What's Next? Your Road Ahead

Every course has a last page, but a programming course is different: its last page is really your first page. You began with a single line - console.log("Hello!") - and you now write programs that make decisions, repeat work, transform arrays with map and filter, model the world with objects, and store information in real files. Forty-two lessons, three mini-projects, and one final program stand between the reader you were and the programmer you are now. Nothing you learned here expires, and none of it will need to be thrown away later.

So what should you learn next? The short answer is the web. HTML gives a page its structure, CSS gives it style, and the DOM - the document object model - is the bridge that lets JavaScript reach inside a page: changing text, showing and hiding elements, and reacting when someone clicks or types. This book deliberately stopped at that door, because the language had to come first. Now that decisions, loops, functions, arrays, and objects feel like home, the DOM will greet you with familiar faces: document.querySelector is a built-in function, addEventListener takes a callback, and every element it hands you is an object with properties you can read and change.

The road ahead, in order

- HTML and CSS - the structure and style of web pages, learned best by building real ones.
- The DOM and browser events - the JavaScript of clicks, typing, and living pages.
- fetch and APIs - programs that ask servers for data, from weather reports to your own databases.
- Deeper Node.js - the huge standard library, npm packages, and your first small servers.
- A framework such as React - a way of organizing big projects, easiest once the language underneath feels natural.

TIP

Keep your final project folder. The Study Log is a real application, not an exercise: when you learn the DOM you can give it a web page, and when you learn fetch you can share its data. Programmers grow by growing their own projects, and yours is already under construction.

Whatever you study next, keep the habits that carried you here. Type code by hand instead of pasting it, break it on purpose, and read every error message until it feels like advice rather than an insult. Rebuild the mini-projects from memory, then rebuild them differently. When you get stuck - and you will, because everyone does - search, read, experiment, and sleep on it; the bug that looks impossible tonight often looks obvious tomorrow morning.

And when the road feels long, remember how you got this far: one small lesson at a time, with code you actually ran and mistakes you actually fixed. Every programmer you admire once sat exactly where you are sitting, holding a finished first course and wondering what to build next. Choose the next small thing, type the first line, and keep going. We will see your work on the web.

JavaScript Foundations

A Friendly Step-by-Step Course for Absolute Beginners - with lessons, checks, practice, quizzes, challenges, projects, a glossary, and a complete answer key.

01 **42 lessons** in 10 chapters, each one a small, complete idea

02 **168 Lesson Check questions** - four after every lesson

03 **100 practice exercises** in six styles, from MCQ to code writing

04 **15-question mixed quiz** closing every chapter

05 **50 challenges** with hints and step-by-step solutions

06 **3 mini-projects** and one grand final project with files and modules

07 **Every code example runnable** in your browser console or Node.js

08 **Common Mistakes boxes** that show beginner traps - and the fixes

09 **84-term glossary** and a full index that points to the exact lesson

10 **Answer Key for all 468 questions**, explaining every wrong option

> *"You do not need talent to start programming. You need curiosity, patience, and a first line of code - this book gives you all three."*
>
> FROM THE INTRODUCTION

DIRECTED BY

Mohamed Ahmed Fayad

AUDIENCE	LEVEL	EDITION
Students & Self-Learners	Beginner	First Edition